

Document Number: P0267R1
Date: 2016-03-21
Revises: P0267R0
Reply to: Michael B. McLaughlin
mikebmcl@gmail.com
Herb Sutter
Microsoft Inc.
hsutter@microsoft.com
Jason Zink
jzink_1@yahoo.com
Audience: LEWG

A Proposal to Add 2D Graphics Rendering and Display to C++

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomatting.

Contents

Contents	ii
List of Tables	v
List of Figures	vi
1 General	1
1.1 Scope	1
1.2 Normative references	1
1.3 Terms and definitions	1
1.4 Hello world example	6
2 Requirements	7
2.1 Namespaces and headers	7
2.2 Feature test macros	7
2.3 Native handles	7
2.4 IEC 559 floating point support	7
2.5 Exact width integer types	7
3 Error reporting	9
4 Header <experimental/io2d> synopsis	11
5 Error codes	14
5.1 Enum class <code>io2d_error</code>	14
5.2 Class <code>io2d_error_category</code>	16
6 Colors	18
6.1 Class <code>rgba_color</code>	18
6.2 <code>literals</code> namespace	36
7 Class <code>vector_2d</code>	37
7.1 <code>vector_2d</code> synopsis	37
7.2 <code>vector_2d</code> Description	37
7.3 <code>vector_2d</code> constructors and assignment operators	37
7.4 <code>vector_2d</code> modifiers	38
7.5 <code>vector_2d</code> observers	38
7.6 <code>vector_2d</code> member operators	38
7.7 <code>vector_2d</code> non-member operators	39
8 Class <code>rectangle</code>	40
8.1 <code>rectangle</code> synopsis	40
8.2 <code>rectangle</code> Description	40
8.3 <code>rectangle</code> constructors and assignment operators	40
8.4 <code>rectangle</code> modifiers	41
8.5 <code>rectangle</code> observers	42

9	Class <code>matrix_2d</code>	43
9.1	<code>matrix_2d</code> synopsis	43
9.2	<code>matrix_2d</code> Description	44
9.3	<code>matrix_2d</code> constructors and assignment operators	44
9.4	<code>matrix_2d</code> static factory functions	45
9.5	<code>matrix_2d</code> modifiers	45
9.6	<code>matrix_2d</code> observers	47
9.7	<code>matrix_2d</code> member operators	48
9.8	<code>matrix_2d</code> non-member operators	48
10	Paths	49
10.1	Path geometries	49
10.2	Enum class <code>path_data_type</code>	55
10.3	Class <code>path_data_item</code>	56
10.4	Class <code>path</code>	82
10.5	Class <code>path_factory</code>	83
11	Fonts	91
11.1	Enum class <code>font_slant</code>	91
11.2	Enum class <code>font_weight</code>	92
11.3	Enum class <code>subpixel_order</code>	92
11.4	Class <code>font_options</code>	93
11.5	Class <code>font_face</code>	94
11.6	Class <code>simple_font_face</code>	95
11.7	Class <code>font_extents</code>	97
11.8	Class <code>text_extents</code>	99
12	Brushes	103
12.1	Gradient brushes	103
12.2	Enum class <code>extend</code>	107
12.3	Enum class <code>filter</code>	108
12.4	Enum class <code>brush_type</code>	109
12.5	Class <code>brush</code>	110
12.6	Class <code>solid_color_brush_factory</code>	114
12.7	Class <code>linear_brush_factory</code>	115
12.8	Class <code>radial_brush_factory</code>	117
12.9	Class <code>surface_brush_factory</code>	120
13	Surfaces	122
13.1	Enum class <code>antialias</code>	122
13.2	Enum class <code>content</code>	123
13.3	Enum class <code>fill_rule</code>	123
13.4	Enum class <code>line_cap</code>	124
13.5	Enum class <code>line_join</code>	125
13.6	Enum class <code>compositing_operator</code>	126
13.7	Enum class <code>format</code>	131
13.8	Enum class <code>scaling</code>	133
13.9	Enum class <code>refresh_rate</code>	136
13.10	Class <code>device</code>	138
13.11	Class <code>surface</code>	140
13.12	Class <code>image_surface</code>	170

13.13	Class <code>display_surface</code>	173
13.14	Class <code>mapped_surface</code>	190
14	Standalone functions	194
14.1	Standalone functions synopsis	194
14.2	<code>format_stride_for_width</code>	194
14.3	<code>make_display_surface</code>	194
14.4	<code>make_image_surface</code>	195
A	Bibliography	196
	Index	197
	Index of implementation-defined behavior	198

List of Tables

1	<code>io2d_error</code> enumerator meanings	14
2	<code>native_geometry_collection</code> state data	49
3	<code>path_data_type</code> enumerator meanings	55
4	<code>font_slant</code> enumerator meanings	91
5	<code>font_weight</code> enumerator meanings	92
6	<code>subpixel_order</code> enumerator meanings	93
7	<code>extend</code> enumerator meanings	108
8	<code>filter</code> enumerator meanings	109
9	<code>brush_type</code> enumerator meanings	110
10	<code>antialias</code> enumerator meanings	122
11	<code>content</code> value meanings	123
12	<code>fill_rule</code> enumerator meanings	124
13	<code>line_cap</code> enumerator meanings	125
14	<code>line_join</code> enumerator meanings	125
15	<code>compositing_operator</code> basic enumerator meanings	128
16	<code>compositing_operator</code> blend enumerator meanings	129
17	<code>compositing_operator</code> hsl enumerator meanings	131
18	<code>format</code> enumerator meanings	132
19	<code>scaling</code> enumerator meanings	134
20	<code>refresh_rate</code> value meanings	137
21	Surface observable state	146
22	<code>surface</code> rendering and composing operations	149
23	Point transformations	152
24	Display surface observable state	176

List of Figures

1 General

[io2d.general]

1.1 Scope

[io2d.general.scope]

- ¹ This Technical Specification specifies requirements for implementations of an interface that computer programs written in the C++ programming language may use to render and display 2D computer graphics.

1.2 Normative references

[io2d.general.refs]

- ¹ The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
- (1.1) — ISO/IEC 14882, *Programming languages — C++*
 - (1.2) — ISO/IEC 10646-1:1993, *Information technology — Universal Multiple-Octet Coded Character Set (UCS) — Part 1: Architecture and Basic Multilingual Plane*
 - (1.3) — ISO/IEC TR 19769:2004, *Information technology — Programming languages, their environments and system software interfaces — Extensions for the programming language C to support new character data types*
 - (1.4) — ISO 15076-1, *Image technology colour management — Architecture, profile format and data structure — Part 1: Based on ICC.1:2004-10*
 - (1.5) — IEC 61966-2-1, *Colour Measurement and Management in Multimedia Systems and Equipment - Part 2-1: Default RGB Colour Space - sRGB*
 - (1.6) — ISO 32000-1:2008, *Document management — Portable document format — Part 1: PDF 1.7*
 - (1.7) — ISO/IEC 9541-1:2012, *Information technology — Font information interchange — Part 1: Architecture*
 - (1.8) — Tantek Çelik et al., *CSS Color Module Level 3 — W3C Recommendation 07 June 2011*, Copyright © 2011 W3C[®] (MIT, ERCIM, Keio)
- ² The library described in ISO/IEC TR 19769:2004 is hereinafter called the *C Unicode TR*.
- ³ The document CSS Color Module Level 3 — W3C Recommendation 07 June 2011 is hereinafter called the *CSS Colors Specification*.

1.3 Terms and definitions

[io2d.general.defns]

- ¹ For the purposes of this document, the following definitions apply.

1.3.1

[io2d.general.defns.standardcoordinatespace]

standard coordinate space

a Euclidean plane described by a Cartesian coordinate system where the first coordinate is measured along a horizontal axis, called the *x* axis, oriented from left to right, the second coordinate is measured along a vertical axis, called the *y* axis, oriented from top to bottom, and rotation of a point around the origin by a positive value expressed in radians is clockwise

1.3.2

[io2d.general.defns.visualdata]

visual data

data representing color, transparency, or some combination thereof

1.3.3 [io2d.general.defs.channel]
channel

a bounded set of homogeneously-spaced real numbers in the range $[0, 1]$

1.3.4 [io2d.general.defns.visualdataformat]
visual data format

a specification of visual data channels which defines a total bit size for the format and each channel's role, bit size, and location relative to the upper (high-order) bit [*Note*: The total bit size may be larger than the sum of the bit sizes of all of the channels of the format. — *end note*]

1.3.5 [io2d.general.defns.visualdataelement]
visual data element

an item of visual data with a defined visual data format

1.3.6 [io2d.general.defns.alpha]
alpha

visual data representing transparency

1.3.7 [io2d.general.defns.pixel]
pixel

a discrete, rectangular visual data element

1.3.8 [io2d.general.defns.alias]
aliasing

the presence of visual artifacts in the results of rendering due to sampling imperfections

1.3.9 [io2d.general.defns.artifact]
artifact

an error in the results of the application of a composing operation

1.3.10 [io2d.general.defns.antialias]
anti-aliasing

the application of a function or algorithm while rendering to reduce aliasing [*Note*: Certain algorithms can produce “better” results, i.e. results with less artifacts or with less pronounced artifacts, when rendering text with anti-aliasing due to the nature of text rendering. As such, it often makes sense to provide the ability to choose one type of anti-aliasing for text rendering and another for all other rendering and to provide different sets of anti-aliasing types to choose from for each of the two operations. — *end note*]

1.3.11 [io2d.general.defns.aspectratio]
aspect ratio

the ratio of the width to the height of a rectangular area

1.3.12 [io2d.general.defns.closedpathgeometry]
closed path geometry

a path geometry with one or more path segments where the initial path segment's start point is used to define the end point of the final path segment

1.3.13 [io2d.general.defns.colormodel]
color model

an ideal, mathematical representation of colors which often uses color channels

1.3.14 [io2d.general.defns.additivecolor]
additive color
 a color defined by the emissive intensity of its color channels

1.3.15 [io2d.general.defns.rgbcolormodel]
RGB color model
 an additive color model using red, green, and blue color channels

1.3.16 [io2d.general.defns.rgbacolormodel]
RGBA color model
 the RGB color model with an alpha channel [*Note:* RGBA is not a proper color model; it is a convenient way to refer to the RGB color model to which an alpha channel has been added.

The interpretation of the alpha channel and its effect on the interpretation of the RGB color channels is intentionally not defined here because it is context-dependent. — *end note*]

1.3.17 [io2d.general.defns.colorsapce]
color space
 an unambiguous mapping of values to colorimetric colors [*Note:* The difference between a color model and a color space is often obscured, and sometimes the terms themselves are mistakenly used interchangeably.

A color model defines color mathematically without regard to how humans actually perceive color. Color models are useful for working with color computationally but, since they deal in ideal colors rather than perceived colors, they fail to provide the information necessary to allow for the uniform display of their colors on different output devices (e.g. LCD monitors, CRT TVs, and printers).

A color space, by contrast, maps unambiguous values to perceived colors. Since the perception of color varies from person to person, color spaces use the science of colorimetry to define those perceived colors in order to obtain uniformity. As such, the uniform display of the colors in a color space on different output devices is possible. — *end note*]

1.3.18 [io2d.general.defns.srgbcolorsapce]
sRGB color space
 an additive color space defined in IEC 61966-2-1 that is based on the RGB color model

1.3.19 [io2d.general.defns.bezier]
cubic Bézier curve
 a curve defined by the equation $f(t) = (1 - t)^3 \times P_0 + 3 \times t \times (1 - t)^2 \times P_1 + 3 \times t^2 \times (1 - t) \times P_2 + t^3 \times P_3$ where $0.0 \leq t \leq 1.0$, P_0 is the starting point, P_1 is the first control point, P_2 is the second control point, and P_3 is the ending point

1.3.20 [io2d.general.defns.currentpoint]
current point
 a point established by various operations used in creating a path geometry [*Note:* A new path geometry has no current point except as otherwise specified. — *end note*]

1.3.21 [io2d.general.defns.degeneratepathgeometry]
degenerate path geometry
 a path geometry with only one path segment [*Note:* The path segment is not required to be a degenerate path segment. — *end note*]

1.3.22 [io2d.general.defns.degeneratepathsegment]
degenerate path segment
 a path segment which has the same value for its start point and its end point is a *degenerate path segment*

1.3.23 [io2d.general.defns.filter] filter

a mathematical function that determines the visual data value of a point for a graphics data graphics resource

1.3.24 [io2d.general.defns.finalpathsegment] final path segment

a path segment whose end point shall not be used to define the start point of any other path segment [*Note*: It is possible for the initial path segment and final path segment to be the same path segment. — *end note*]

1.3.25 [io2d.general.defns.graphicsdata] graphics data

<graphics data> visual data stored in an unspecified form

1.3.26 [io2d.general.defns.graphics.raster] graphics data

<raster graphics data> visual data stored as pixels that is accessible as if it was an array of rows of pixels beginning with the pixel at the integral point (0, 0)

1.3.27 [io2d.general.defns.graphicsresource] graphics resource

<graphics resource> an object of unspecified type used by an implementation [*Note*: By its definition a graphics resource is an implementation detail. Often it will be a graphics subsystem object (e.g. a graphics device or a render target) or an aggregate composed of multiple graphics subsystem objects. However the only requirement placed upon a graphics resource is that the implementation is able to use it to provide the functionality required of the graphics resource. — *end note*]

1.3.28 [io2d.general.defns.graphicsresource.graphicsdata] graphics resource

<graphics data graphics resource> an object of unspecified type used by an implementation to provide access to and allow manipulation of visual data

1.3.29 [io2d.general.defns.graphicsresource.pathgeometry] graphics resource

<path geometry graphics resource> an object of unspecified type used by an implementation to store a collection of zero or more path geometries in an unspecified format

1.3.30 [io2d.general.defns.pixmap] pixmap

a raster graphics data graphics resource

1.3.31 [io2d.general.defns.point] point

<point> a coordinate designated by a floating point x axis value and a floating point y axis value within the standard coordinate space

1.3.32 [io2d.general.defns.point.integral] point

<integral point> a coordinate designated by an integral x axis value and an integral y axis value within the standard coordinate space

1.3.33 [io2d.general.defns.premultipliedformat] **premultiplied format**

a format with color and alpha where each color channel is normalized and then multiplied by the normalized alpha channel value [*Example:* Given the 32-bit non-premultiplied RGBA pixel with 8 bits per channel {255, 0, 0, 127} (half-transparent red), when normalized it would become {1.0, 0.0, 0.0, 0.5}. As such, in premultiplied, normalized format it would become {0.5, 0.0, 0.0, 0.5} as a result of multiplying each of the three color channels by the alpha channel value. — *end example*]

1.3.34 [io2d.general.defns.graphicsstatedata] **graphics state data**

data which specify how some part of the process of rendering or of a composing operation shall be performed in part or in whole

1.3.35 [io2d.general.defns.initialpathsegment] **initial path segment**

a path segment whose start point is not defined as being the end point of another path segment [*Note:* It is possible for the initial path segment and final path segment to be the same path segment. — *end note*]

1.3.36 [io2d.general.defns.graphicssubsystem] **graphics subsystem**

collection of unspecified operating system and library functionality used to render and display 2D computer graphics

1.3.37 [io2d.general.defns.lastmovetopoint] **last-move-to point**

the point in a path geometry that is the start point of the initial path segment

1.3.38 [io2d.general.defns.normalize] **normalize**

to map a closed set of evenly spaced values in the range $[0, x]$ to an evenly spaced sequence of floating point values in the range $[0, 1]$ [*Note:* The definition of normalize given is the definition for normalizing unsigned input. Signed normalization, i.e. the mapping of a closed set of evenly spaced values in the range $[-x, x]$ to an evenly spaced sequence of floating point values in the range $[-1, 1]$, also exists but is not used in this Technical Specification. — *end note*]

1.3.39 [io2d.general.defns.openpathgeometry] **open path geometry**

a path geometry with one or more path segments where the initial path segment's start point is not used to define the end point of the final path segment [*Note:* Even if the start point of the initial path segment and the end point of the final path segment are assigned the same coordinates, the path geometry is still an open path geometry since the final path segment's end point is not defined as being the start point of the initial segment but instead merely happens to have the same value as that point. — *end note*]

1.3.40 [io2d.general.defns.pathgeometry] **path geometry**

a collection of path segments where the end point of each path segment, except the final path segment, shall be used to define the start point of exactly one other path segment in the collection

1.3.41 [io2d.general.defns.pathsegment] **path segment**

is a line or a curve, each of which has a start point and an end point

1.3.42 [io2d.general.defns.render]
render

to transform path geometries or text into graphics data in the manner specified by a set of graphics state data

1.3.43 [io2d.general.defns.renderingoperation]
rendering operation

an operation that performs rendering

1.3.44 [io2d.general.defns.compose]
compose

to combine part or all of a source graphics data graphics resource with a destination graphics data graphics resource in the manner specified by a composition algorithm

1.3.45 [io2d.general.defns.composingoperation]
composing operation

an operation that performs composing

1.3.46 [io2d.general.defns.compositionalgorithm]
composition algorithm

an algorithm that combines a source visual data element and a destination visual data element producing a visual data element that has the same visual data format as the destination visual data element

1.3.47 [io2d.general.defns.renderingandcomposingop]
rendering and composing operation

an operation that is either a composing operation or a rendering operation followed by a composing operation

1.3.48 [io2d.general.defns.sample]
sample

to use a filter to obtain the visual data for a given point from a graphics data graphics resource

1.3.49 [io2d.general.defns.colorstop]
color stop

a tuple composed of a floating point offset value in the range [0,1] and a color value

1.4 Hello world example [io2d.general.helloworld]

- ¹ The following is an example of a complete "hello world" program written using this library:

```
#include <experimental/io2d>
using namespace std::experimental::io2d;

int main() {
    auto ds = make_display_surface(640, 480, format::argb32);
    ds.draw_callback([](display_surface& ds) {
        ds.paint(rgba_color::cornflower_blue());
        ds.render_text("Hello world!", { 50.0, 50.0 }, rgba_color::black());
    });
    return ds.show();
}
```

2 Requirements

[io2d.req]

2.1 Namespaces and headers

[io2d.req.namespace]

- ¹ The components described in this technical specification are experimental and not part of the C++ standard library. All components described in this technical specification are declared in namespace `std::experimental::io2d::v1` or a sub-namespace thereof unless otherwise specified. The header described in this technical specification shall import the contents of `std::experimental::io2d::v1` into `std::experimental::io2d` as if by

2

```
namespace std {
    namespace experimental {
        namespace io2d {
            inline namespace v1 { }
        }
    }
}
```

- ³ Unless otherwise specified, references to other entities described in this Technical Specification are assumed to be qualified with `std::experimental::io2d::v1::`, and references to entities described in the C++ standard are assumed to be qualified with `std::`.

2.2 Feature test macros

[io2d.req.macros]

- ¹ This macro allows users to determine which version of this Technical Specification is supported by header `<experimental/io2d>`.
- ² Header `<experimental/io2d>` shall supply the following macro definition:
- ³ `#define __cpp_lib_experimental_io2d 201603`
- ⁴ [*Note:* The value of macro `__cpp_lib_experimental_io2d` is `yyyymm` where `yyyy` is the year and `mm` the month when the version of the Technical Specification was completed. — *end note*]

2.3 Native handles

[io2d.req.native]

- ¹ Several classes described in this Technical Specification have members `native_handle_type` and `native_handle`. The presence of these members and their semantics is implementation-defined. [*Note:* These members allow implementations to provide access to implementation details. Their names are specified to facilitate portable compile-time detection. Actual use of these members is inherently non-portable. — *end note*]

2.4 IEC 559 floating point support

[io2d.req.iec559]

- ¹ Certain requirements of this Technical Specification assume that `numeric_limits<double>::is_iec559` evaluates to `true`. The behavior of these requirements is implementation-defined if `numeric_limits<double>::is_iec559` evaluates to `false`.

2.5 Exact width integer types

[io2d.req.cstdintopttypes]

- ¹ In order to implement this Technical Specification, the implementation shall provide the following optional integer types from the `<cstdint>` header file:

(1.1) — `uint8_t`

(1.2) — `uint16_t`

(1.3) — `uint32_t`

(1.4) — `uint64_t`

3 Error reporting

[io2d.err.report]

- ¹ 2D graphics library functions often provide two overloads, one that throws an exception to report graphics subsystem errors, and another that sets an **error_code**.
- ² [*Note*: This supports two common use cases:
 - (2.1) — Uses where graphics subsystem errors are truly exceptional and indicate a serious failure. Throwing an exception is the most appropriate response. This is the preferred default for most everyday programming.
 - (2.2) — Uses where graphics subsystem errors are routine and do not necessarily represent failure. Returning an error code is the most appropriate response. This allows application specific error handling, including simply ignoring the error.

— *end note*]
- ³ Functions **not** having an argument of type **error_code&** report errors as follows, unless otherwise specified:
 - (3.1) — When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications and the cause of the error is described in the function's *Error conditions* description:
 - (3.1.1) — If the description calls for **errc::argument_out_of_domain** or **io2d_error::invalid_index**, the exception type shall be **out_of_range** constructed with an implementation-defined **what_arg** argument value.
 - (3.1.2) — If the description calls for **errc::invalid_argument**, the exception type shall be **invalid_argument** constructed with an implementation-defined **what_arg** argument value.
 - (3.1.3) — If the description calls for **errc::not_enough_memory**, the error shall be reported by throwing an exception as described in C++ 2014 §17.6.5.12 [res.on.exception.handling].
 - (3.1.4) — In all other cases the exception type shall be **system_error** constructed with an **ec** argument value formed by passing the specified enumerator value to **make_error_code** and an implementation-defined **what_arg** argument value, unless otherwise specified.
 - (3.2) — When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications and the cause of the error is **not** described in the function's *Error conditions* description and is not a failure to allocate storage, an exception of type **system_error** shall be thrown constructed with its **error_code** argument set as appropriate for the specific operating system dependent error. Implementations shall document the cause, enumerator value, **error_category**, and exception type for each of these additional error conditions.
 - (3.3) — Failure to allocate storage is reported by throwing an exception as described in C++ 2014 §17.6.5.12 [res.on.exception.handling].
 - (3.4) — Destructors throw nothing.
- ⁴ Functions taking an argument of type **error_code&** report errors as follows, unless otherwise specified:
 - (4.1) — When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications and the cause of the error is described in the function's *Error conditions* description, the **error_code&** argument is set as appropriate for the specified enumerator.

- (4.2) — When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications and the cause of the error is **not** described in the function's *Error conditions* description and is not a failure to allocate storage, the **error_code&** argument is set as appropriate for the specific operating system dependent error. Implementations should document these errors where possible.
- (4.3) — If a failure to allocate storage occurs, the **error_code&** argument shall be set to **make_error_code(errc::not_enough_memory)**.
- (4.4) — Otherwise, **clear()** is called on the **error_code&** argument.

4 Header <experimental/io2d> synopsis [syn]

```

namespace std { namespace experimental {
    // From C++ Extensions for Library Fundamentals, N4335
    struct nullopt_t;
    constexpr nullopt_t nullopt{ implementation-defined };

    namespace io2d { inline namespace v1 {

        typedef tuple<vector<double>, double> dashes;

        enum class io2d_error;
        enum class antialias;
        enum class content;
        enum class fill_rule;
        enum class line_cap;
        enum class line_join;
        enum class compositing_operator;
        enum class format;
        enum class extend;
        enum class filter;
        enum class brush_type;
        enum class font_slant;
        enum class font_weight;
        enum class subpixel_order;
        enum class path_data_type;
        enum class scaling;

        class io2d_error_category;
        const error_category& io2d_category() noexcept;

        class rectangle;

        class rgba_color;

        inline namespace literals {
            double operator""ubyte(unsigned long long value);
            double operator""unorm(long double value);
        }

        class vector_2d;
        bool operator==(const vector_2d& lhs, const vector_2d& rhs) noexcept;
        bool operator!=(const vector_2d& lhs, const vector_2d& rhs) noexcept;
        vector_2d operator+(const vector_2d& lhs) noexcept;
        vector_2d operator+(const vector_2d& lhs, const vector_2d& rhs) noexcept;
        vector_2d operator-(const vector_2d& lhs) noexcept;
        vector_2d operator-(const vector_2d& lhs, const vector_2d& rhs) noexcept;
        vector_2d operator*(const vector_2d& lhs, double rhs) noexcept;
        vector_2d operator*(double lhs, const vector_2d& rhs) noexcept;
    }
}

```

```

class font_extents;
class text_extents;

class matrix_2d;

matrix_2d operator*(const matrix_2d& lhs, const matrix_2d& rhs);
matrix_2d& operator*=(matrix_2d& lhs, const matrix_2d& rhs);
bool operator==(const matrix_2d& lhs, const matrix_2d& rhs);
bool operator!=(const matrix_2d& lhs, const matrix_2d& rhs);

class path_data_item;
class path_data_item::arc;
class path_data_item::arc_negative;
class path_data_item::change_matrix;
class path_data_item::change_origin;
class path_data_item::close_path;
class path_data_item::curve_to;
class path_data_item::line_to;
class path_data_item::move_to;
class path_data_item::new_sub_path;
class path_data_item::path_data;
class path_data_item::rel_curve_to;
class path_data_item::rel_line_to;
class path_data_item::rel_move_to;

class path;
class path_factory;

class device;

class font_options;

class font_face;
class simple_font_face;

class brush;
class solid_color_brush_factory;
class linear_brush_factory;
class radial_brush_factory;
class surface_brush_factory;

class surface;
class image_surface;
class display_surface;
class mapped_surface;

int format_stride_for_width(format format, int width) noexcept;
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat,
    scaling scl = scaling::letterbox);
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat, error_code& ec,
    scaling scl = scaling::letterbox) noexcept;
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat, int preferredDisplayWidth,

```

```

        int preferredDisplayHeight, scaling scl = scaling::letterbox);
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat, int preferredDisplayWidth,
    int preferredDisplayHeight, ::std::error_code& ec,
    scaling scl = scaling::letterbox) noexcept;
image_surface make_image_surface(format format, int width, int height);
image_surface make_image_surface(format format, int width, int height,
    error_code& ec) noexcept;
} } } }

namespace std {
    template<>
    struct is_error_condition_enum<experimental::io2d::io2d_error>
        : public std::true_type{ };

    template<>
    struct is_error_code_enum<implementation-defined>
        : public true_type{ }; // exposition only

    std::error_condition make_error_condition(experimental::io2d::io2d_error e)
        noexcept;

    std::error_code make_error_code(experimental::io2d::io2d_error e) noexcept;
}

```

5 Error codes

[errorcodes]

- ¹ The `io2d_error` enum class and the `io2d_error_category` class are provided by this Technical Specification to report errors from the graphics subsystem, excluding certain errors which shall be reported in other ways as per the requirements of (3).

5.1 Enum class `io2d_error`

[io2derror]

5.1.1 `io2d_error` Summary

[io2derror.summary]

- ¹ The `io2d_error` enum class is an enumeration holding error condition values which are used with the `io2d_error_category` class. See Table 1 for the meaning of each error condition value.

5.1.2 `io2d_error` Synopsis

[io2derror.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class io2d_error {
        success,
        invalid_restore,
        no_current_point,
        invalid_matrix,
        invalid_status,
        null_pointer,
        invalid_string,
        invalid_path_data,
        read_error,
        write_error,
        surface_finished,
        invalid_dash,
        invalid_index,
        clip_not_representable,
        invalid_stride,
        user_font_immutable,
        user_font_error,
        invalid_clusters,
        device_error,
        invalid_mesh_construction,
    };
} } }

template<>
struct is_error_condition_enum<experimental::io2d::io2d_error>
: public std::true_type{ };
}
```

5.1.3 `io2d_error` Enumerators

[io2derror.enumerators]

Table 1 — `io2d_error` enumerator meanings

Enumerator	Meaning
success	The operation completed successfully.

Table 1 — `io2d_error` enumerator meanings (continued)

Enumerator	Meaning
<code>invalid_restore</code>	A call was made to <code>surface::restore</code> for which no prior call to <code>surface::save</code> was made.
<code>no_current_point</code>	The operation requires a current point but no current point was set. This is the result of an improper call to <code>path_factory::rel_curve_to</code> , <code>path_factory::rel_line_to</code> , or <code>path_factory::rel_move_to</code> or is the result of a call to <code>path_factory::append</code> with malformed data. See
<code>invalid_matrix</code>	A <code>matrix_2d</code> value that the operation depends on is invalid. Except as otherwise specified, this means that the <code>matrix_2d</code> value is not invertible.
<code>invalid_status</code>	An internal error has occurred. The conditions and circumstances under which this <code>io2d_error</code> value occurs are implementation-defined. [<i>Note</i> : This value should only be used when no other <code>io2d_error</code> value is appropriate. It signifies that an implementation-specific error occurred such as passing a bad native handle as an argument. — <i>end note</i>]
<code>null_pointer</code>	A null pointer value was unexpectedly encountered. The conditions and circumstances under which this <code>io2d_error</code> value occurs are implementation-defined.
<code>invalid_string</code>	A UTF-8 string value was expected but the string is not a valid UTF-8 string.
<code>invalid_path_data</code>	Invalid data was encountered in a <code>path</code> or a <code>path_factory</code> object. [<i>Note</i> : This status value should only occur when a user creates invalid path data and appends it to a path. — <i>end note</i>]
<code>read_error</code>	An error occurred while attempting to read data from an input stream.
<code>write_error</code>	An error occurred while attempting to write data to an output stream.
<code>surface_finished</code>	An attempt was made to use or manipulate a <code>surface</code> object or <code>surface</code> -derived object which is no longer valid. [<i>Note</i> : This can occur due to a previous call to <code>surface::finish</code> or as a result of erroneous usage of a native handle. — <i>end note</i>]
<code>invalid_dash</code>	An invalid dash value was specified in a call to <code>surface::set_dashes</code> .
<code>invalid_index</code>	An index value was specified in a call to a function which is outside the range of index values that are currently valid.
<code>clip_not_representable</code>	A call was made to <code>surface::get_clip_rectangles</code> when the <code>surface</code> object's current clipping region could not be represented with rectangles.
<code>invalid_stride</code>	An invalid stride value was used. Surface formats may require padding at the end of each row of pixel data depending on the implementation and the current graphics chipset, if any. Use <code>format_stride_for_width</code> to obtain the correct stride value.

Table 1 — `io2d_error` enumerator meanings (continued)

Enumerator	Meaning
<code>user_font_immutable</code>	User font immutable. [<i>Note</i> : Reserved. — <i>end note</i>]
<code>user_font_error</code>	User font error. [<i>Note</i> : Reserved. — <i>end note</i>]
<code>invalid_clusters</code>	A call was made to <code>surface::show_text_glyphs</code> with a <code>std::vector<text_clusters></code> argument that does not properly map the UTF-8 <code>std::string</code> code points to the <code>std::vector<glyph></code> glyphs.
<code>device_error</code>	The operation failed. The device encountered an error. [<i>Note</i> : The conditions and circumstances in which this <code>io2d_error</code> value occurs are implementation-defined. — <i>end note</i>]

5.2 Class `io2d_error_category`

[io2derrcat]

5.2.1 `io2d_error_category` synopsis

[io2derrcat.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class io2d_error_category : public std::error_category {
    public:
        // 5.2.3, observers:
        virtual const char* name() const noexcept override;
        virtual string message(int errVal) const override;
        virtual bool equivalent(int code,
            const error_condition& condition) const noexcept override;
        virtual bool equivalent(const error_code& ec,
            int condition) const noexcept override;
    };

    // 5.2.4, non-member functions:
    const error_category& io2d_category() noexcept;
} } }

error_condition make_error_condition(
    experimental::io2d::io2d_error e) noexcept;
error_code make_error_code(experimental::io2d::io2d_error e) noexcept;
}

```

5.2.2 `io2d_error_category` Description

[io2derrcat.intro]

- ¹ The `io2d_error_category` class derives from `error_category` in order to provide a custom error category for use by this library.

5.2.3 `io2d_error_category` observers

[io2derrcat.observers]

```
virtual const char* name() const noexcept override;
```

- ¹ *Returns:* A pointer to the string "io2d".

```
virtual string message(int errVal) const override;
```

- ² *Returns:* When `errVal` has the same value as the integer value of an `io2d_error` enumerator, the corresponding meaning text in Table 1 shall be part of the `string` returned by this function for that value. If there is no corresponding enumerator, the return value is implementation-defined. [*Note*:

When `errVal` has the same value as the integer value of an `io2d_error` enumerator, implementations should include any additional meaningful diagnostic information in the `string` returned by this function. When no equivalent value enumerator exists, implementations should return string diagnostic information provided by the underlying rendering and presentation technologies as well as any additional meaningful diagnostic information in the `string` returned by this function. — *end note*

```
virtual bool equivalent(int code,
    const error_condition& condition) const noexcept override;
```

- 3 *Returns:* True if `condition.category() == *this` and the implementation-defined error code value `code` equates to `static_cast<io2d_error>(condition.value())`. [*Note:* Because of the variations in rendering and presentation technologies available for use on different platforms, the issue of equivalence between error codes and error conditions is one that must be determined by implementors. — *end note*]

```
virtual bool equivalent(const error_code& ec,
    int condition) const noexcept override;
```

- 4 *Returns:* True if `ec.category() == *this` and the implementation-defined error code value in `ec.value` equates to `static_cast<io2d_error>(condition)`. [*Note:* Because of the variations in rendering and presentation technologies available for use on different platforms, the issue of equivalence between error codes and error conditions is one that must be determined by implementors. — *end note*]

5.2.4 io2d_error_category non-member functions [io2derrcat.nonmembers]

```
const error_category& io2d_category() noexcept;
```

- 1 *Returns:* A reference to an object of a type derived from `error_category`. All calls to this function shall return references to the same object.
- 2 *Remarks:* The object's `default_error_condition` virtual function shall behave as specified for the class `error_category`. The object's `message` and `equivalent` virtual functions shall behave as specified for the class `io2d_error_category`. The object's `name` virtual function shall return a pointer to the string "io2d".

```
error_condition make_error_condition(experimental::io2d::io2d_error e) noexcept;
```

- 3 *Returns:* `error_condition(static_cast<int>(e), experimental::io2d::io2d_category())`;

```
error_code make_error_code(experimental::io2d::io2d_error e) noexcept;
```

- 4 *Returns:* `error_code(static_cast<int>(e), experimental::io2d::io2d_category())`;

6 Colors

[colors]

¹ This section is forthcoming in a future revision.

6.1 Class `rgba_color`

[rgbacolor]

¹ The class `rgba_color` describes a four channel color in premultiplied format.

² There are three color channels, red, green, and blue, each of which is a `double`.

³ There is also an alpha channel, which is a `double`.

⁴ Legal values for each channel are in the range `[0, 1]`.

⁵ The type predefines a set of named colors, for which each channel is an unsigned normalized 8-bit integer.

6.1.1 `rgba_color` synopsis

[rgbacolor.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class rgba_color {
        // 6.1.2, construct/copy/move/destroy:
        rgba_color() noexcept;
        rgba_color(double r, double g, double b, double a = 1.0);
        rgba_color(double r, double g, double b, error_code& ec) noexcept;
        rgba_color(double r, double g, double b, double a, error_code& ec) noexcept;
        rgba_color(const rgba_color& c) noexcept;
        rgba_color& operator=(const rgba_color& c) noexcept;
        rgba_color(rgba_color&& c) noexcept;
        rgba_color& operator=(rgba_color&& c) noexcept;

        // 6.1.3, modifiers:
        void r(double val);
        void r(double val, error_code& ec) noexcept;
        void g(double val);
        void g(double val, error_code& ec) noexcept;
        void b(double val);
        void b(double val, error_code& ec) noexcept;
        void a(double val);
        void a(double val, error_code& ec) noexcept;

        // 6.1.4, observers:
        double r() const noexcept;
        double g() const noexcept;
        double b() const noexcept;
        double a() const noexcept;

        // 6.1.5, static member functions:
        static const rgba_color& alice_blue() noexcept;
        static const rgba_color& antique_white() noexcept;
        static const rgba_color& aqua() noexcept;
        static const rgba_color& aquamarine() noexcept;
        static const rgba_color& azure() noexcept;
        static const rgba_color& beige() noexcept;
        static const rgba_color& bisque() noexcept;
```



```
static const rgba_color& black() noexcept;
static const rgba_color& blanched_almond() noexcept;
static const rgba_color& blue() noexcept;
static const rgba_color& blue_violet() noexcept;
static const rgba_color& brown() noexcept;
static const rgba_color& burly_wood() noexcept;
static const rgba_color& cadet_blue() noexcept;
static const rgba_color& chartreuse() noexcept;
static const rgba_color& chocolate() noexcept;
static const rgba_color& coral() noexcept;
static const rgba_color& cornflower_blue() noexcept;
static const rgba_color& cornsilk() noexcept;
static const rgba_color& crimson() noexcept;
static const rgba_color& cyan() noexcept;
static const rgba_color& dark_blue() noexcept;
static const rgba_color& dark_cyan() noexcept;
static const rgba_color& dark_goldenrod() noexcept;
static const rgba_color& dark_gray() noexcept;
static const rgba_color& dark_green() noexcept;
static const rgba_color& dark_grey() noexcept;
static const rgba_color& dark_khaki() noexcept;
static const rgba_color& dark_magenta() noexcept;
static const rgba_color& dark_olive_green() noexcept;
static const rgba_color& dark_orange() noexcept;
static const rgba_color& dark_orchid() noexcept;
static const rgba_color& dark_red() noexcept;
static const rgba_color& dark_salmon() noexcept;
static const rgba_color& dark_sea_green() noexcept;
static const rgba_color& dark_slate_blue() noexcept;
static const rgba_color& dark_slate_gray() noexcept;
static const rgba_color& dark_slate_grey() noexcept;
static const rgba_color& dark_turquoise() noexcept;
static const rgba_color& dark_violet() noexcept;
static const rgba_color& deep_pink() noexcept;
static const rgba_color& deep_sky_blue() noexcept;
static const rgba_color& dim_gray() noexcept;
static const rgba_color& dim_grey() noexcept;
static const rgba_color& dodger_blue() noexcept;
static const rgba_color& firebrick() noexcept;
static const rgba_color& floral_white() noexcept;
static const rgba_color& forest_green() noexcept;
static const rgba_color& fuchsia() noexcept;
static const rgba_color& gainsboro() noexcept;
static const rgba_color& ghost_white() noexcept;
static const rgba_color& gold() noexcept;
static const rgba_color& goldenrod() noexcept;
static const rgba_color& gray() noexcept;
static const rgba_color& green() noexcept;
static const rgba_color& green_yellow() noexcept;
static const rgba_color& grey() noexcept;
static const rgba_color& honeydew() noexcept;
static const rgba_color& hot_pink() noexcept;
static const rgba_color& indian_red() noexcept;
static const rgba_color& indigo() noexcept;
static const rgba_color& ivory() noexcept;
```

```
static const rgba_color& khaki() noexcept;
static const rgba_color& lavender() noexcept;
static const rgba_color& lavender_blush() noexcept;
static const rgba_color& lawn_green() noexcept;
static const rgba_color& lemon_chiffon() noexcept;
static const rgba_color& light_blue() noexcept;
static const rgba_color& light_coral() noexcept;
static const rgba_color& light_cyan() noexcept;
static const rgba_color& light_goldenrod_yellow() noexcept;
static const rgba_color& light_gray() noexcept;
static const rgba_color& light_green() noexcept;
static const rgba_color& light_grey() noexcept;
static const rgba_color& light_pink() noexcept;
static const rgba_color& light_salmon() noexcept;
static const rgba_color& light_sea_green() noexcept;
static const rgba_color& light_sky_blue() noexcept;
static const rgba_color& light_slate_gray() noexcept;
static const rgba_color& light_slate_grey() noexcept;
static const rgba_color& light_steel_blue() noexcept;
static const rgba_color& light_yellow() noexcept;
static const rgba_color& lime() noexcept;
static const rgba_color& lime_green() noexcept;
static const rgba_color& linen() noexcept;
static const rgba_color& magenta() noexcept;
static const rgba_color& maroon() noexcept;
static const rgba_color& medium_aquamarine() noexcept;
static const rgba_color& medium_blue() noexcept;
static const rgba_color& medium_orchid() noexcept;
static const rgba_color& medium_purple() noexcept;
static const rgba_color& medium_sea_green() noexcept;
static const rgba_color& medium_slate_blue() noexcept;
static const rgba_color& medium_spring_green() noexcept;
static const rgba_color& medium_turquoise() noexcept;
static const rgba_color& medium_violet_red() noexcept;
static const rgba_color& midnight_blue() noexcept;
static const rgba_color& mint_cream() noexcept;
static const rgba_color& misty_rose() noexcept;
static const rgba_color& moccasin() noexcept;
static const rgba_color& navajo_white() noexcept;
static const rgba_color& navy() noexcept;
static const rgba_color& old_lace() noexcept;
static const rgba_color& olive() noexcept;
static const rgba_color& olive_drab() noexcept;
static const rgba_color& orange() noexcept;
static const rgba_color& orange_red() noexcept;
static const rgba_color& orchid() noexcept;
static const rgba_color& pale_goldenrod() noexcept;
static const rgba_color& pale_green() noexcept;
static const rgba_color& pale_turquoise() noexcept;
static const rgba_color& pale_violet_red() noexcept;
static const rgba_color& papaya_whip() noexcept;
static const rgba_color& peach_puff() noexcept;
static const rgba_color& peru() noexcept;
static const rgba_color& pink() noexcept;
static const rgba_color& plum() noexcept;
```

```

static const rgba_color& powder_blue() noexcept;
static const rgba_color& purple() noexcept;
static const rgba_color& red() noexcept;
static const rgba_color& rosy_brown() noexcept;
static const rgba_color& royal_blue() noexcept;
static const rgba_color& saddle_brown() noexcept;
static const rgba_color& salmon() noexcept;
static const rgba_color& sandy_brown() noexcept;
static const rgba_color& sea_green() noexcept;
static const rgba_color& sea_shell() noexcept;
static const rgba_color& sienna() noexcept;
static const rgba_color& silver() noexcept;
static const rgba_color& sky_blue() noexcept;
static const rgba_color& slate_blue() noexcept;
static const rgba_color& slate_gray() noexcept;
static const rgba_color& slate_grey() noexcept;
static const rgba_color& snow() noexcept;
static const rgba_color& spring_green() noexcept;
static const rgba_color& steel_blue() noexcept;
static const rgba_color& tan() noexcept;
static const rgba_color& teal() noexcept;
static const rgba_color& thistle() noexcept;
static const rgba_color& tomato() noexcept;
static const rgba_color& transparent_black() noexcept;
static const rgba_color& turquoise() noexcept;
static const rgba_color& violet() noexcept;
static const rgba_color& wheat() noexcept;
static const rgba_color& white() noexcept;
static const rgba_color& white_smoke() noexcept;
static const rgba_color& yellow() noexcept;
static const rgba_color& yellow_green() noexcept;

private:
    double _Red;    // exposition only
    double _Green;  // exposition only
    double _Blue;   // exposition only
    double _Alpha;  // exposition only
};

// 6.1.6, non-member operators:
bool operator==(const rgba_color& lhs, const rgba_color& rhs) noexcept;
bool operator!=(const rgba_color& lhs, const rgba_color& rhs) noexcept;
} } } }

```

6.1.2 rgba_color constructors and assignment operators

[rgbacolor.cons]

```
rgba_color() noexcept;
```

1 *Effects:* Constructs an object of type `rgba_color`.

2 *Postconditions:* `_Red == 0.0`.

3 `_Green == 0.0`.

4 `_Blue == 0.0`.

5 `_Alpha == 1.0`.

6 *Note:* The resulting color is opaque black, which can also be obtained from `rgba_color::black()`.

```

    rgba_color(double r, double g, double b, double a = 1.0);
    rgba_color(double r, double g, double b, error_code& ec) noexcept;
    rgba_color(double r, double g, double b, double a, error_code& ec) noexcept;

```

Requires: $r \geq 0.0$ and $r \leq 1.0$ and $g \geq 0.0$ and $g \leq 1.0$ and $b \geq 0.0$ and $b \leq 1.0$. Where there is an `a` parameter, $a \geq 0.0$ and $a \leq 1.0$.

Effects: Constructs an object of type `rgba_color`.

Postconditions: Where there is an `a` parameter, `_Alpha = a`; otherwise `_Alpha = 1.0`.

`_Red = r * a.`

`_Green = g * a.`

`_Blue = b * a.`

Throws: As specified in Error reporting (3).

Remarks: In the event of a non-throwing error, the object shall be constructed to meet the following conditions:

`_Red == 1.0 && _Green == 0.0 && _Blue == 1.0 && _Alpha == 1.0.`

Error conditions: `errc::argument_out_of_domain` if the value of `r`, `g`, `b`, or `a` fails to meet the preconditions.

Notes: When an object is constructed despite the presence of one or more erroneous arguments, the resulting color is opaque magenta, which can also be obtained from `rgba_color::magenta()`.

This color was picked because it is a legal color value, it does not have all channels set to the same value (which would hide, e.g., erroneous saturation calculations), and in the hope that it will help highlight error visibility (both visually and in an examination of the data) in the event that the developer neglects to check the `error_code&` argument following construction of the object.

6.1.3 `rgba_color` modifiers

[`rbgacolor.modifiers`]

```

    void r(double val);
    void r(double val, error_code& ec) noexcept;

```

Requires: $val \geq 0.0$ and $val \leq 1.0$.

Throws: As specified in Error reporting (3).

Postconditions: `_Red == val.`

Remarks: In the event of an error, the object shall not be modified.

Error conditions: `errc::argument_out_of_domain` if the value of `val` fails to meet the preconditions.

```

    void g(double val);
    void g(double val, error_code& ec) noexcept;

```

Requires: $val \geq 0.0$ and $val \leq 1.0$.

Throws: As specified in Error reporting (3).

Postconditions: `_Green == val.`

Remarks: In the event of an error, the object shall not be modified.

Error conditions: `errc::argument_out_of_domain` if the value of `val` fails to meet the preconditions.

```

    void b(double val);
    void b(double val, error_code& ec) noexcept;

```

11 *Requires:* `val >= 0.0` and `val <= 1.0`.
 12 *Throws:* As specified in Error reporting (3).
 13 *Postconditions:* `_Blue == val`.
 14 *Remarks:* In the event of an error, the object shall not be modified.
 15 *Error conditions:* `errc::argument_out_of_domain` if the value of `val` fails to meet the preconditions.

```
void a(double val);
void a(double val, error_code& ec) noexcept;
```

16 *Requires:* `val >= 0.0` and `val <= 1.0`.
 17 *Throws:* As specified in Error reporting (3).
 18 *Postconditions:* `_Alpha == val`.
 `_Red = _Red * val`.
 `_Green = _Green * val`.
 `_Blue = _Blue * val`.
 19 *Remarks:* In the event of an error, the object shall not be modified.
 20 *Error conditions:* `errc::argument_out_of_domain` if the value of `val` fails to meet the preconditions.

6.1.4 `rgba_color` observers

[`rgbacolor.observers`]

```
double r() const noexcept;
```

1 *Returns:* `_Red`.

```
double g() const noexcept;
```

2 *Returns:* `_Green`.

```
double b() const noexcept;
```

3 *Returns:* `_Blue`.

```
double a() const noexcept;
```

4 *Returns:* `_Alpha`.

6.1.5 `rgba_color` static member functions

[`rgbacolor.statics`]

```
static const rgba_color& alice_blue() noexcept;
```

1 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 240ubyte, 248ubyte, 255ubyte, 255ubyte }`.

```
static const rgba_color& antique_white() noexcept;
```

2 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 250ubyte, 235ubyte, 215ubyte, 255ubyte }`.

```
static const rgba_color& aqua() noexcept;
```

3 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 255ubyte, 255ubyte, 255ubyte }`.

```
static const rgba_color& aquamarine() noexcept;
```

4 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 127ubyte, 255ubyte, 212ubyte, 255ubyte }`.

`static const rgba_color& azure() noexcept;`

5 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 240ubyte, 255ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& beige() noexcept;`

6 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 245ubyte, 245ubyte, 220ubyte, 255ubyte }`.

`static const rgba_color& bisque() noexcept;`

7 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 228ubyte, 196ubyte, 255ubyte }`.

`static const rgba_color& black() noexcept;`

8 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 0ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& blanched_almond() noexcept;`

9 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 235ubyte, 205ubyte, 255ubyte }`.

`static const rgba_color& blue() noexcept;`

10 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 0ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& blue_violet() noexcept;`

11 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 138ubyte, 43ubyte, 226ubyte, 255ubyte }`.

`static const rgba_color& brown() noexcept;`

12 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 165ubyte, 42ubyte, 42ubyte, 255ubyte }`.

`static const rgba_color& burly_wood() noexcept;`

13 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 222ubyte, 184ubyte, 135ubyte, 255ubyte }`.

`static const rgba_color& cadet_blue() noexcept;`

14 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 95ubyte, 158ubyte, 160ubyte, 255ubyte }`.

`static const rgba_color& chartreuse() noexcept;`

15 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 127ubyte, 255ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& chocolate() noexcept;`

16 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 210ubyte, 105ubyte, 30ubyte, 255ubyte }`.

`static const rgba_color& coral() noexcept;`

17 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 127ubyte, 80ubyte, 255ubyte }`.

`static const rgba_color& cornflower_blue() noexcept;`

18 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 100ubyte, 149ubyte, 237ubyte, 255ubyte }`.

`static const rgba_color& cornsilk() noexcept;`

19 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 248ubyte, 220ubyte, 255ubyte }`.

`static const rgba_color& crimson() noexcept;`

20 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 220ubyte, 20ubyte, 60ubyte, 255ubyte }`.

`static const rgba_color& cyan() noexcept;`

21 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 255ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& dark_blue() noexcept;`

22 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 0ubyte, 139ubyte, 255ubyte }`.

`static const rgba_color& dark_cyan() noexcept;`

23 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 139ubyte, 139ubyte, 255ubyte }`.

`static const rgba_color& dark_goldenrod() noexcept;`

24 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 184ubyte, 134ubyte, 11ubyte, 255ubyte }`.

`static const rgba_color& dark_gray() noexcept;`

25 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 169ubyte, 169ubyte, 169ubyte, 255ubyte }`.

`static const rgba_color& dark_green() noexcept;`

26 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 100ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& dark_grey() noexcept;`

27 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 169ubyte, 169ubyte, 169ubyte, 255ubyte }`.

`static const rgba_color& dark_khaki() noexcept;`

28 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 189ubyte, 183ubyte, 107ubyte, 255ubyte }`.

`static const rgba_color& dark_magenta() noexcept;`

29 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 139ubyte, 0ubyte, 139ubyte, 255ubyte }`.

`static const rgba_color& dark_olive_green() noexcept;`

30 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 85ubyte, 107ubyte, 47ubyte, 255ubyte }`.

`static const rgba_color& dark_orange() noexcept;`

31 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 140ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& dark_orchid() noexcept;`

32 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 153ubyte, 50ubyte, 204ubyte, 255ubyte }`.

`static const rgba_color& dark_red() noexcept;`

33 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 139ubyte, 0ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& dark_salmon() noexcept;`

34 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 233ubyte, 150ubyte, 122ubyte, 255ubyte }`.

`static const rgba_color& dark_sea_green() noexcept;`

35 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 143ubyte, 188ubyte, 143ubyte, 255ubyte }`.

`static const rgba_color& dark_slate_blue() noexcept;`

36 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 72ubyte, 61ubyte, 139ubyte, 255ubyte }`.

`static const rgba_color& dark_slate_gray() noexcept;`

37 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 47ubyte, 79ubyte, 79ubyte, 255ubyte }`.

`static const rgba_color& dark_slate_grey() noexcept;`

38 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 47ubyte, 79ubyte, 79ubyte, 255ubyte }`.

`static const rgba_color& dark_turquoise() noexcept;`

39 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 206ubyte, 209ubyte, 255ubyte }`.

`static const rgba_color& dark_violet() noexcept;`

40 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 148ubyte, 0ubyte, 211ubyte, 255ubyte }`.

`static const rgba_color& deep_pink() noexcept;`

41 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 20ubyte, 147ubyte, 255ubyte }`.

`static const rgba_color& deep_sky_blue() noexcept;`

42 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 191ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& dim_gray() noexcept;`

43 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 105ubyte, 105ubyte, 105ubyte, 255ubyte }`.

`static const rgba_color& dim_grey() noexcept;`

44 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 105ubyte, 105ubyte, 105ubyte, 255ubyte }`.

`static const rgba_color& dodger_blue() noexcept;`

45 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 30ubyte, 144ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& firebrick() noexcept;`

46 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 178ubyte, 34ubyte, 34ubyte, 255ubyte }`.

`static const rgba_color& floral_white() noexcept;`

47 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 250ubyte, 240ubyte, 255ubyte }`.

`static const rgba_color& forest_green() noexcept;`

48 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 34ubyte, 139ubyte, 34ubyte, 255ubyte }`.

`static const rgba_color& fuchsia() noexcept;`

49 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 0ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& gainsboro() noexcept;`

50 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 220ubyte, 220ubyte, 220ubyte, 255ubyte }`.

`static const rgba_color& ghost_white() noexcept;`

51 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 248ubyte, 248ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& gold() noexcept;`

52 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 215ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& goldenrod() noexcept;`

53 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 218ubyte, 165ubyte, 32ubyte, 255ubyte }`.

`static const rgba_color& gray() noexcept;`

54 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 128ubyte, 128ubyte, 128ubyte, 255ubyte }`.

`static const rgba_color& green() noexcept;`

55 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 128ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& green_yellow() noexcept;`

56 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 173ubyte, 255ubyte, 47ubyte, 255ubyte }`.

`static const rgba_color& grey() noexcept;`

57 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 128ubyte, 128ubyte, 128ubyte, 255ubyte }`.

`static const rgba_color& honeydew() noexcept;`

58 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 240ubyte, 255ubyte, 240ubyte, 255ubyte }`.

`static const rgba_color& hot_pink() noexcept;`

59 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 105ubyte, 180ubyte, 255ubyte }`.

`static const rgba_color& indian_red() noexcept;`

60 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 205ubyte, 92ubyte, 92ubyte, 255ubyte }`.

`static const rgba_color& indigo() noexcept;`

61 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 75ubyte, 0ubyte, 130ubyte, 255ubyte }`.

`static const rgba_color& ivory() noexcept;`

62 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 255ubyte, 240ubyte, 255ubyte }`.

`static const rgba_color& khaki() noexcept;`

63 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 240ubyte, 230ubyte, 140ubyte, 255ubyte }`.

`static const rgba_color& lavender() noexcept;`

64 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 230ubyte, 230ubyte, 250ubyte, 255ubyte }`.

static const `rgba_color&` lavender_blush() noexcept;

65 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 240ubyte, 245ubyte, 255ubyte }`.

static const `rgba_color&` lawn_green() noexcept;

66 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 124ubyte, 252ubyte, 0ubyte, 255ubyte }`.

static const `rgba_color&` lemon_chiffon() noexcept;

67 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 250ubyte, 205ubyte, 255ubyte }`.

static const `rgba_color&` light_blue() noexcept;

68 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 173ubyte, 216ubyte, 230ubyte, 255ubyte }`.

static const `rgba_color&` light_coral() noexcept;

69 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 240ubyte, 128ubyte, 128ubyte, 255ubyte }`.

static const `rgba_color&` light_cyan() noexcept;

70 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 224ubyte, 255ubyte, 255ubyte, 255ubyte }`.

static const `rgba_color&` light_goldenrod_yellow() noexcept;

71 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 250ubyte, 250ubyte, 210ubyte, 255ubyte }`.

static const `rgba_color&` light_gray() noexcept;

72 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 211ubyte, 211ubyte, 211ubyte, 255ubyte }`.

static const `rgba_color&` light_green() noexcept;

73 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 144ubyte, 238ubyte, 144ubyte, 255ubyte }`.

static const `rgba_color&` light_grey() noexcept;

74 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 211ubyte, 211ubyte, 211ubyte, 255ubyte }`.

static const `rgba_color&` light_pink() noexcept;

75 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 182ubyte, 193ubyte, 255ubyte }`.

static const `rgba_color&` light_salmon() noexcept;

76 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 160ubyte, 122ubyte, 255ubyte }`.

`static const rgba_color& light_sea_green() noexcept;`

77 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 32ubyte, 178ubyte, 170ubyte, 255ubyte }`.

`static const rgba_color& light_sky_blue() noexcept;`

78 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 135ubyte, 206ubyte, 250ubyte, 255ubyte }`.

`static const rgba_color& light_slate_gray() noexcept;`

79 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 119ubyte, 136ubyte, 153ubyte, 255ubyte }`.

`static const rgba_color& light_slate_grey() noexcept;`

80 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 119ubyte, 136ubyte, 153ubyte, 255ubyte }`.

`static const rgba_color& light_steel_blue() noexcept;`

81 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 176ubyte, 196ubyte, 222ubyte, 255ubyte }`.

`static const rgba_color& light_yellow() noexcept;`

82 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 255ubyte, 224ubyte, 255ubyte }`.

`static const rgba_color& lime() noexcept;`

83 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 255ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& lime_green() noexcept;`

84 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 50ubyte, 205ubyte, 50ubyte, 255ubyte }`.

`static const rgba_color& linen() noexcept;`

85 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 250ubyte, 240ubyte, 230ubyte, 255ubyte }`.

`static const rgba_color& magenta() noexcept;`

86 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 0ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& maroon() noexcept;`

87 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 128ubyte, 0ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& medium_aquamarine() noexcept;`

88 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 102ubyte, 205ubyte, 170ubyte, 255ubyte }`.

`static const rgba_color& medium_blue() noexcept;`

89 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 0ubyte, 205ubyte, 255ubyte }`.

`static const rgba_color& medium_orchid() noexcept;`

90 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 186ubyte, 85ubyte, 211ubyte, 255ubyte }`.

`static const rgba_color& medium_purple() noexcept;`

91 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 147ubyte, 112ubyte, 219ubyte, 255ubyte }`.

`static const rgba_color& medium_sea_green() noexcept;`

92 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 60ubyte, 179ubyte, 113ubyte, 255ubyte }`.

`static const rgba_color& medium_slate_blue() noexcept;`

93 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 123ubyte, 104ubyte, 238ubyte, 255ubyte }`.

`static const rgba_color& medium_spring_green() noexcept;`

94 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 250ubyte, 154ubyte, 255ubyte }`.

`static const rgba_color& medium_turquoise() noexcept;`

95 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 72ubyte, 209ubyte, 204ubyte, 255ubyte }`.

`static const rgba_color& medium_violet_red() noexcept;`

96 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 199ubyte, 21ubyte, 133ubyte, 255ubyte }`.

`static const rgba_color& midnight_blue() noexcept;`

97 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 25ubyte, 25ubyte, 112ubyte, 255ubyte }`.

`static const rgba_color& mint_cream() noexcept;`

98 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 245ubyte, 255ubyte, 250ubyte, 255ubyte }`.

`static const rgba_color& misty_rose() noexcept;`

99 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 228ubyte, 225ubyte, 255ubyte }`.

`static const rgba_color& moccasin() noexcept;`

100 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 228ubyte, 181ubyte, 255ubyte }`.

`static const rgba_color& navajo_white() noexcept;`

101 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 222ubyte, 173ubyte, 255ubyte }`.

`static const rgba_color& navy() noexcept;`

102 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 0ubyte, 128ubyte, 255ubyte }`.

`static const rgba_color& old_lace() noexcept;`

103 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 253ubyte, 245ubyte, 230ubyte, 255ubyte }`.

`static const rgba_color& olive() noexcept;`

104 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 128ubyte, 128ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& olive_drab() noexcept;`

105 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 107ubyte, 142ubyte, 35ubyte, 255ubyte }`.

`static const rgba_color& orange() noexcept;`

106 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 165ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& orange_red() noexcept;`

107 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 69ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& orchid() noexcept;`

108 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 218ubyte, 112ubyte, 214ubyte, 255ubyte }`.

`static const rgba_color& pale_goldenrod() noexcept;`

109 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 238ubyte, 232ubyte, 170ubyte, 255ubyte }`.

`static const rgba_color& pale_green() noexcept;`

110 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 152ubyte, 251ubyte, 152ubyte, 255ubyte }`.

`static const rgba_color& pale_turquoise() noexcept;`

111 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 175ubyte, 238ubyte, 238ubyte, 255ubyte }`.

`static const rgba_color& pale_violet_red() noexcept;`

112 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 219ubyte, 112ubyte, 147ubyte, 255ubyte }`.

static const `rgba_color&` `papaya_whip()` noexcept;

113 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 239ubyte, 213ubyte, 255ubyte }`.

static const `rgba_color&` `peach_puff()` noexcept;

114 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 218ubyte, 185ubyte, 255ubyte }`.

static const `rgba_color&` `peru()` noexcept;

115 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 205ubyte, 133ubyte, 63ubyte, 255ubyte }`.

static const `rgba_color&` `pink()` noexcept;

116 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 192ubyte, 203ubyte, 255ubyte }`.

static const `rgba_color&` `plum()` noexcept;

117 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 221ubyte, 160ubyte, 221ubyte, 255ubyte }`.

static const `rgba_color&` `powder_blue()` noexcept;

118 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 176ubyte, 224ubyte, 230ubyte, 255ubyte }`.

static const `rgba_color&` `purple()` noexcept;

119 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 128ubyte, 0ubyte, 128ubyte, 255ubyte }`.

static const `rgba_color&` `red()` noexcept;

120 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 0ubyte, 0ubyte, 255ubyte }`.

static const `rgba_color&` `rosy_brown()` noexcept;

121 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 188ubyte, 143ubyte, 143ubyte, 255ubyte }`.

static const `rgba_color&` `royal_blue()` noexcept;

122 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 65ubyte, 105ubyte, 225ubyte, 255ubyte }`.

static const `rgba_color&` `saddle_brown()` noexcept;

123 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 139ubyte, 69ubyte, 19ubyte, 255ubyte }`.

static const `rgba_color&` `salmon()` noexcept;

124 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 250ubyte, 128ubyte, 114ubyte, 255ubyte }`.

`static const rgba_color& sandy_brown() noexcept;`

125 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 244ubyte, 164ubyte, 96ubyte, 255ubyte }`.

`static const rgba_color& sea_green() noexcept;`

126 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 46ubyte, 139ubyte, 87ubyte, 255ubyte }`.

`static const rgba_color& sea_shell() noexcept;`

127 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 245ubyte, 238ubyte, 255ubyte }`.

`static const rgba_color& sienna() noexcept;`

128 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 160ubyte, 82ubyte, 45ubyte, 255ubyte }`.

`static const rgba_color& silver() noexcept;`

129 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 192ubyte, 192ubyte, 192ubyte, 255ubyte }`.

`static const rgba_color& sky_blue() noexcept;`

130 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 135ubyte, 206ubyte, 235ubyte, 255ubyte }`.

`static const rgba_color& slate_blue() noexcept;`

131 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 106ubyte, 90ubyte, 205ubyte, 255ubyte }`.

`static const rgba_color& slate_gray() noexcept;`

132 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 112ubyte, 128ubyte, 144ubyte, 255ubyte }`.

`static const rgba_color& slate_grey() noexcept;`

133 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 112ubyte, 128ubyte, 144ubyte, 255ubyte }`.

`static const rgba_color& snow() noexcept;`

134 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 250ubyte, 250ubyte, 255ubyte }`.

`static const rgba_color& spring_green() noexcept;`

135 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 255ubyte, 127ubyte, 255ubyte }`.

`static const rgba_color& steel_blue() noexcept;`

136 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 70ubyte, 130ubyte, 180ubyte, 255ubyte }`.

`static const rgba_color& tan() noexcept;`

137 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 210ubyte, 180ubyte, 140ubyte, 255ubyte }`.

`static const rgba_color& teal() noexcept;`

138 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 128ubyte, 128ubyte, 255ubyte }`.

`static const rgba_color& thistle() noexcept;`

139 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 216ubyte, 191ubyte, 216ubyte, 255ubyte }`.

`static const rgba_color& tomato() noexcept;`

140 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 99ubyte, 71ubyte, 255ubyte }`.

`static const rgba_color& transparent_black() noexcept;`

141 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 0ubyte, 0ubyte, 0ubyte, 0ubyte }`.

`static const rgba_color& turquoise() noexcept;`

142 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 64ubyte, 244ubyte, 208ubyte, 255ubyte }`.

`static const rgba_color& violet() noexcept;`

143 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 238ubyte, 130ubyte, 238ubyte, 255ubyte }`.

`static const rgba_color& wheat() noexcept;`

144 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 245ubyte, 222ubyte, 179ubyte, 255ubyte }`.

`static const rgba_color& white() noexcept;`

145 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 255ubyte, 255ubyte, 255ubyte }`.

`static const rgba_color& white_smoke() noexcept;`

146 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 245ubyte, 245ubyte, 245ubyte, 255ubyte }`.

`static const rgba_color& yellow() noexcept;`

147 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 255ubyte, 255ubyte, 0ubyte, 255ubyte }`.

`static const rgba_color& yellow_green() noexcept;`

148 *Returns:* a const reference to the static `rgba_color` object `rgba_color{ 154ubyte, 205ubyte, 50ubyte, 255ubyte }`.

6.1.6 rgba_color non-member operators**[rgbacolor.ops]**

```
bool operator==(const rgba_color& lhs, const rgba_color& rhs) noexcept;
```

¹ *Returns:* lhs.r() == rhs.r() && lhs.g() == rhs.g() && lhs.b() == rhs.b() && lhs.a() == rhs.a().

```
bool operator!=(const rgba_color& lhs, const rgba_color& rhs) noexcept;
```

² *Returns:* !(lhs == rhs)

6.2 literals namespace**[literals]****6.2.1 literals Synopsis****[literals.synopsis]**

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    inline namespace literals {
        double operator "" ubyte(unsigned long long int value);
        double operator "" unorm(long double value);
    }
} } } }
```

6.2.2 literals operators**[literals.operators]**

```
double operator "" ubyte(unsigned long long int value);
```

¹ *Returns:* max(0.0, min(1.0, static_cast<double>(value) / 255.0))

```
double operator "" unorm(long double value);
```

² *Returns:* nearbyint(max(0.0, min(1.0, static_cast<double>(value))) * 255.0) / 255.0

7 Class vector_2d

[vector2d]

7.1 vector_2d synopsis

[vector2d.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class vector_2d {
    public:
        // 7.3, construct/copy/move/destroy:
        vector_2d() noexcept;
        vector_2d(double x, double y) noexcept;
        vector_2d(const vector_2d&) noexcept;
        vector_2d& operator=(const vector_2d&) noexcept;
        vector_2d(vector_2d&&) noexcept;
        vector_2d& operator=(vector_2d&&) noexcept;

        // 7.4, modifiers:
        void x(double value) noexcept;
        void y(double value) noexcept;

        // 7.5, observers:
        double x() const noexcept;
        double y() const noexcept;
        double length() const noexcept;
        double dot(const vector_2d& other) const noexcept;
        vector_2d to_unit() const noexcept;

        // 7.6, member operators:
        vector_2d& operator+=(const vector_2d& rhs) noexcept;
        vector_2d& operator-=(const vector_2d& rhs) noexcept;
        vector_2d& operator*=(double rhs) noexcept;

    private:
        double _X; // exposition only
        double _Y; // exposition only
    };

    // 7.7, non-member operators:
    bool operator==(const vector_2d& lhs, const vector_2d& rhs) noexcept;
    bool operator!=(const vector_2d& lhs, const vector_2d& rhs) noexcept;
    vector_2d operator+(const vector_2d& lhs) noexcept;
    vector_2d operator+(const vector_2d& lhs, const vector_2d& rhs) noexcept;
    vector_2d operator-(const vector_2d& lhs) noexcept;
    vector_2d operator-(const vector_2d& lhs, const vector_2d& rhs) noexcept;
    vector_2d operator*(const vector_2d& lhs, double rhs) noexcept;
    vector_2d operator*(double lhs, const vector_2d& rhs) noexcept;
} } } }

```

7.2 vector_2d Description

[vector2d.intro]

- ¹ The class vector_2d describes an object that stores a two-dimensional Euclidean vector.

7.3 vector_2d constructors and assignment operators

[vector2d.cons]

```
vector_2d() noexcept;
```

1 *Effects:* Constructs an object of type `vector_2d`.

2 *Postconditions:* `_X == 0.0 && _Y == 0.0`.

```
vector_2d(double x, double y) noexcept;
```

3 *Effects:* Constructs an object of type `vector_2d`.

4 *Postconditions:* `_X == x && _Y == y`.

7.4 `vector_2d` modifiers

[vector2d.modifiers]

```
void x(double value) noexcept;
```

1 *Postconditions:* `_X == value`.

```
void y(double value) noexcept;
```

2 *Postconditions:* `_Y == value`.

7.5 `vector_2d` observers

[vector2d.observers]

```
double x() const noexcept;
```

1 *Returns:* `_X`.

```
double y() const noexcept;
```

2 *Returns:* `_Y`.

```
double length() const noexcept;
```

3 *Returns:* `sqrt(_X * _X + _Y * _Y)`.

```
double dot(const vector_2d& other) const noexcept;
```

4 *Returns:* `_X * other._X + _Y * other._Y`.

```
vector_2d to_unit() const noexcept;
```

5 *Returns:* `vector_2d{ _X / length(), _Y / length() }`.

7.6 `vector_2d` member operators

[vector2d.member.ops]

```
vector_2d& operator+=(const vector_2d& rhs) noexcept;
```

1 *Effects:* `*this = *this + rhs`.

2 *Returns:* `*this`.

```
vector_2d& operator-=(const vector_2d& rhs) noexcept;
```

3 *Effects:* `*this = *this - rhs`.

4 *Returns:* `*this`.

```
vector_2d& operator*=(double rhs) noexcept;
```

5 *Effects:* `*this = *this * rhs`.

6 *Returns:* `*this`.

7.7 vector_2d non-member operators**[vector2d.ops]**

```

    bool operator==(const vector_2d& lhs, const vector_2d& rhs) noexcept;
1    Returns: lhs.x() == rhs.x() && lhs.y() == rhs.y().

    bool operator!=(const vector_2d& lhs, const vector_2d& rhs) noexcept;
2    Returns: !(lhs == rhs).

    vector_2d operator+(const vector_2d& lhs) noexcept;
3    Returns: vector_2d(lhs).

    vector_2d operator+(const vector_2d& lhs, const vector_2d& rhs) noexcept;
4    Returns: vector_2d{ lhs.x() + rhs.x(), lhs.y() + rhs.y() }.

    vector_2d operator-(const vector_2d& lhs) noexcept;
5    Returns: vector_2d{ -lhs.x(), -lhs.y() }.

    vector_2d operator-(const vector_2d& lhs, const vector_2d& rhs) noexcept;
6    Returns: vector_2d{ lhs.x() - rhs.x(), lhs.y() - rhs.y() }.

    vector_2d operator*(const vector_2d& lhs, double rhs) noexcept;
7    Returns: vector_2d{ lhs.x() * rhs, lhs.y() * rhs }.

    vector_2d operator*(double lhs, const vector_2d& rhs) noexcept;
8    Returns: vector_2d{ lhs * rhs.x(), lhs * rhs.y() }.

```

8 Class rectangle

[rectangle]

8.1 rectangle synopsis

[rectangle.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class rectangle {
    public:
        // 8.3, construct/copy/move/destroy:
        rectangle() noexcept;
        rectangle(double x, double y, double width, double height) noexcept;
        rectangle(const vector_2d& tl, const vector_2d& br) noexcept;
        rectangle(const rectangle& r) noexcept;
        rectangle& operator=(const rectangle& r) noexcept;
        rectangle(rectangle&& r) noexcept;
        rectangle& operator=(rectangle&& r) noexcept;

        // 8.4, modifiers:
        void x(double value) noexcept;
        void y(double value) noexcept;
        void width(double value) noexcept;
        void height(double value) noexcept;
        void top_left(const vector_2d& value) noexcept;
        void bottom_right(const vector_2d& value) noexcept;

        // 8.5, observers:
        double x() const noexcept;
        double y() const noexcept;
        double width() const noexcept;
        double height() const noexcept;
        vector_2d top_left() const noexcept;
        vector_2d bottom_right() const noexcept;

    private:
        double _X;      // exposition only
        double _Y;      // exposition only
        double _Width;  // exposition only
        double _Height; // exposition only
    };
} } } }

```

8.2 rectangle Description

[rectangle.intro]

- 1 The class `rectangle` describes an object that represents a rectangle.

8.3 rectangle constructors and assignment operators

[rectangle.cons]

```
rectangle() noexcept;
```

- 1 *Effects:* Constructs an object of type `rectangle`.
- 2 *Postconditions:* `_X == 0.0`.
- 3 `_Y == 0.0`.
- 4 `_Width == 0.0`.

```

5      _Height == 0.0.

rectangle(double x, double y, double w, double h) noexcept;
6      Effects: Constructs an object of type rectangle.
7      Postconditions: _X == x.
8      _Y == y.
9      _Width == w.
10     _Height == h.

rectangle(const vector_2d& tl, const vector_2d& br) noexcept;
11     Effects: Constructs an object of type rectangle from a top-left coordinate and a bottom-right coordinate.
12     Postconditions: _X == tl.x().
13     _Y == tl.y().
14     _Width == max(0.0, br.x() - tl.x()).
15     _Height == max(0.0, br.y() - tl.y()).

```

8.4 rectangle modifiers

[rectangle.modifiers]

```

void x(double value) noexcept;
1     Postconditions: _X == value.

void y(double value) noexcept;
2     Postconditions: _Y == value.

void width(double value) noexcept;
3     Postconditions: _Width == value.

void height(double value) noexcept;
4     Postconditions: _Height == value.

void top_left(const vector_2d& value) noexcept;
5     Postconditions: _X == value.x().
6     _Y == value.y().

void bottom_right(const vector_2d& value) noexcept;
7     Postconditions: _Width == max(0.0, value.x() - _X).
8     _Height == max(0.0, value.y() - _Y).

```

8.5 rectangle observers

[rectangle.observers]

```
double x() const noexcept;  
1     Returns: _X.  
  
double y() const noexcept;  
2     Returns: _Y.  
  
double width() const noexcept;  
3     Returns: _Width.  
  
double height() const noexcept;  
4     Returns: _Height.  
  
vector_2d top_left() const noexcept;  
5     Returns: vector_2d{ _X, _Y }.  
  
vector_2d bottom_right() const noexcept;  
6     Returns: vector_2d{ _X + _Width, _Y + _Height }.
```


9 Class `matrix_2d`

[`matrix2d`]

9.1 `matrix_2d` synopsis

[`matrix2d.synopsis`]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class matrix_2d {
    public:
        // 9.3, construct/copy/move/destroy:
        matrix_2d() noexcept;
        matrix_2d(const matrix_2d& other) noexcept;
        matrix_2d& operator=(const matrix_2d& other) noexcept;
        matrix_2d(matrix_2d&& other) noexcept;
        matrix_2d& operator=(matrix_2d&& other) noexcept;
        matrix_2d(double v00, double v01, double v10, double v11,
            double v20, double v21) noexcept;

        // 9.4, static factory functions:
        static matrix_2d init_identity() noexcept;
        static matrix_2d init_translate(const vector_2d& value) noexcept;
        static matrix_2d init_scale(const vector_2d& value) noexcept;
        static matrix_2d init_rotate(double radians) noexcept;
        static matrix_2d init_shear_x(double factor) noexcept;
        static matrix_2d init_shear_y(double factor) noexcept;

        // 9.5, modifiers:
        void m00(double value) noexcept;
        void m01(double value) noexcept;
        void m10(double value) noexcept;
        void m11(double value) noexcept;
        void m20(double value) noexcept;
        void m21(double value) noexcept;
        matrix_2d& translate(const vector_2d& value) noexcept;
        matrix_2d& scale(const vector_2d& value) noexcept;
        matrix_2d& rotate(double radians) noexcept;
        matrix_2d& shear_x(double factor) noexcept;
        matrix_2d& shear_y(double factor) noexcept;
        matrix_2d& invert();
        matrix_2d& invert(error_code& ec) noexcept;

        // 9.6, observers:
        double m00() const noexcept;
        double m01() const noexcept;
        double m10() const noexcept;
        double m11() const noexcept;
        double m20() const noexcept;
        double m21() const noexcept;
        bool is_invertible const noexcept;
        double determinant() const;
        double determinant(error_code& ec) const noexcept;
        vector_2d transform_distance(const vector_2d& dist) const noexcept;
        vector_2d transform_point(const vector_2d& pt) const noexcept;
    };
};
}
}

```

```

// 9.7, matrix_2d member operators:
matrix_2d& operator*=(const matrix_2d& rhs) noexcept;

private:
    double _M00; // exposition only
    double _M01; // exposition only
    double _M10; // exposition only
    double _M11; // exposition only
    double _M20; // exposition only
    double _M21; // exposition only
};

// 9.8, matrix_2d non-member operators:
matrix_2d operator*(const matrix_2d& lhs, const matrix_2d& rhs) noexcept;
bool operator==(const matrix_2d& lhs, const matrix_2d& rhs) noexcept;
bool operator!=(const matrix_2d& lhs, const matrix_2d& rhs) noexcept;
} } } }

```

9.2 matrix_2d Description

[matrix2d.intro]

- 1 The `matrix_2d` class represents a two-dimensional, three row by three column, row-major matrix. Its purpose is to perform affine transformations.
- 2 Mathematically, regardless of the operations performed on a `matrix_2d`, the third column will always have the column vector value of $[0.0, 0.0, 1.0]$. As such, it is not included in the observable data of the matrix.
- 3 The performance of any mathematical operation upon a `matrix_2d` shall be carried out as if the omitted third column data members were present with the values prescribed in the previous paragraph.
- 4 [Note: If the third column's data members were included, they would be:
 - (4.1) — `m02`, a `double` which would follow `m01` in the same row and would be assigned a value of `0.0`.
 - (4.2) — `m12`, a `double` which would follow `m11` in the same row and would be assigned a value of `0.0`.
 - (4.3) — `m22`, a `double` which would follow `m21` in the same row and would be assigned a value of `1.0`.
- 5 The layout of the resulting matrix would be as such:


```

[ [ m00 m01 m02 ] ]
[ [ m10 m11 m12 ] ]
[ [ m20 m21 m22 ] ] — end note

```

9.3 matrix_2d constructors and assignment operators

[matrix2d.cons]

- ```

 matrix_2d() noexcept;

```
- 1 *Effects:* Constructs an object of type `matrix_2d`.
  - 2 *Postconditions:* `_M00 == 1.0`.
  - 3 `_M01 == 0.0`.
  - 4 `_M10 == 0.0`.
  - 5 `_M11 == 1.0`.
  - 6 `_M20 == 0.0`.
  - 7 `_M21 == 0.0`.
  - 8 *Note:* The resulting matrix is the identity matrix, which can also be obtained from `matrix_2d::init_identity()`.

```
matrix_2d(double v00, double v01, double v10, double v11,
double v20, double v21) noexcept;
```

9 *Effects:* Constructs an object of type `matrix_2d`.

10 *Postconditions:* `_M00 == v00`.

11 `_M01 == v01`.

12 `_M10 == v10`.

13 `_M11 == v11`.

14 `_M20 == v20`.

15 `_M21 == v21`.

#### 9.4 `matrix_2d` static factory functions

[`matrix2d.staticfactories`]

```
static matrix_2d init_identity() noexcept;
```

1 *Returns:* `matrix(1.0, 0.0, 0.0, 1.0, 0.0, 0.0)`.

```
static matrix_2d init_translate(const vector_2d& value) noexcept;
```

2 *Returns:* `matrix(1.0, 0.0, 0.0, 1.0, value.x(), value.y())`.

```
static matrix_2d init_scale(const vector_2d& value) noexcept;
```

3 *Returns:* `matrix(value.x(), 0.0, 0.0, value.y(), 0.0, 0.0)`.

```
static matrix_2d init_rotate(double radians) noexcept;
```

4 *Returns:* `matrix(cos(radians), sin(radians), -sin(radians), cos(radians), 0.0, 0.0)`.

```
static matrix_2d init_shear_x(double factor) noexcept;
```

5 *Returns:* `matrix(1.0, 0.0, factor, 1.0, 0.0, 0.0)`.

```
static matrix_2d init_shear_y(double factor) noexcept;
```

6 *Returns:* `matrix{ 1.0, factor, 0.0, 1.0, 0.0, 0.0 }`

#### 9.5 `matrix_2d` modifiers

[`matrix2d.modifiers`]

```
void m00(double value) noexcept;
```

1 *Postconditions:* `_M00 == value`.

```
void m01(double value) noexcept;
```

2 *Postconditions:* `_M01 == value`.

```
void m10(double value) noexcept;
```

3 *Postconditions:* `_M10 == value`.

```
void m11(double value) noexcept;
```

4 *Postconditions:* `_M11 == value`.

```
void m20(double value) noexcept;
```

5 *Postconditions:* `_M20 == value`.

```

 void m21(double value) noexcept;
6 Postconditions: _M21 == value.

matrix_2d& translate(const vector_2d& value) noexcept;
7 Effects: *this = init_translate(value) * (*this).
8 Returns: *this.

matrix_2d& scale(const vector_2d& value) noexcept;
9 Effects: *this = init_scale(value) * (*this).
10 Returns: *this.

matrix_2d& rotate(double radians) noexcept;
11 Effects: *this = init_rotate(radians) * (*this).
12 Returns: *this.

matrix_2d& shear_x(double factor) noexcept;
13 Effects: *this = init_shear_x(factor) * (*this).
14 Returns: *this.

matrix_2d& shear_y(double factor) noexcept;
15 Effects: *this = init_shear_y(factor) * (*this).
16 Returns: *this.

matrix_2d& invert();
matrix_2d& invert(error_code& ec) noexcept;
17 Effects:

 const auto det = _M00 * _M11 - _M01 * _M10;
 const auto inverseDet = 1.0 / det;

 const auto cM02 = 0.0;
 const auto cM12 = 0.0;
 const auto cM22 = 1.0;

 const auto adjugateM00 = _M11 * cM22 - cM12 * _M21;
 const auto adjugateM01 = -(_M01 * cM22 - cM02 * _M21);
 const auto adjugateM10 = -(_M10 * cM22 - cM12 * _M20);
 const auto adjugateM11 = _M00 * cM22 - cM02 * _M20;
 const auto adjugateM20 = _M10 * _M21 - _M11 * _M20;
 const auto adjugateM21 = -(_M00 * _M21 - _M01 * _M20);

 _M00 = inverseDet * adjugateM00;
 _M01 = inverseDet * adjugateM01;
 _M10 = inverseDet * adjugateM10;
 _M11 = inverseDet * adjugateM11;
 _M20 = inverseDet * adjugateM20;
 _M21 = inverseDet * adjugateM21;

18 Returns: *this.
19 Throws: As specified in Error reporting (3).
20 Error conditions: io2d_error::invalid_matrix if this->is_invertible() == false.
21 Remark: If an error occurs, this function shall have no effects.

```

**9.6 matrix\_2d observers****[matrix2d.observers]**

```

 double m00() const noexcept;
1 Returns: _M00.

 double m01() const noexcept;
2 Returns: _M01.

 double m10() const noexcept;
3 Returns: _M10.

 double m11() const noexcept;
4 Returns: _M11.

 double m20() const noexcept;
5 Returns: _M20.

 double m21() const noexcept;
6 Returns: _M21.

 bool is_invertible const noexcept;
7 Returns: true if all of the following are true:
(7.1) — isfinite(_M00)
(7.2) — isfinite(_M01)
(7.3) — isfinite(_M10)
(7.4) — isfinite(_M11)
(7.5) — isfinite(_M20)
(7.6) — isfinite(_M21)
(7.7) — (_M00 * _M11 - _M01 * M10) != 0.0
8 Otherwise returns false.

 double determinant() const;
 double determinant(error_code& ec) const noexcept;
9 Returns: _M00 * _M11 - _M01 * M10.
10 In the event of a non-throwing error, the function shall return numeric_limits<double>::quiet_-
 NaN().
11 Throws: As specified in Error reporting (3).
12 Error conditions: io2d_error::invalid_matrix if !isfinite(_M00) || !isfinite(_M01) || !isfinite(_-
 M10) || !isfinite(_M11) || !isfinite(_M20) || !isfinite(_M21).

 vector_2d transform_distance(const vector_2d& dist) const noexcept;
13 Returns: vector_2d(m00() * dist.x() + m10() * dist.y(), m01() * dist.x() + m11() * dist.y()).
14 Note: This function ignores the translation component of *this. If the translation component, m20()
 and m21(), of *this is set to (0.0, 0.0), the return value of this function and the return value of
 transform_point(dist) will be identical when given the same input.

 vector_2d transform_point(const vector_2d& pt) const noexcept;
15 Returns: vector_2d((m00() * dist.x() + m10() * dist.y()) + m20(), (m01() * dist.x() +
 m11() * dist.y()) + m21()).

```

**9.7 matrix\_2d member operators****[matrix2d.member.ops]**

```
matrix_2d& operator*=(const matrix_2d& rhs) noexcept;
```

1 *Effects:* \*this = \*this \* rhs

2 *Returns:* \*this

**9.8 matrix\_2d non-member operators****[matrix2d.ops]**

```
matrix_2d operator*(const matrix_2d& lhs, const matrix_2d& rhs) noexcept;
```

1 *Returns:* matrix\_2d{

```
 lhs.m00() * rhs.m00() + lhs.m01() * rhs.m10(),
 lhs.m00() * rhs.m01() + lhs.m01() * rhs.m11(),
 lhs.m10() * rhs.m00() + lhs.m11() * rhs.m10(),
 lhs.m10() * rhs.m01() + lhs.m11() * rhs.m11(),
 lhs.m20() * rhs.m00() + lhs.m21() * rhs.m10() + lhs.m20(),
 lhs.m20() * rhs.m01() + lhs.m21() * rhs.m11() + lhs.m21()
}
```

```
bool operator==(const matrix_2d& lhs, const matrix_2d& rhs) noexcept;
```

2 *Returns:* lhs.m00() == rhs.m00() && lhs.m01() == rhs.m01() && lhs.m10() == rhs.m10() &&  
lhs.m11() == rhs.m11() && lhs.m20() == rhs.m20() && lhs.m21() == rhs.m21().

```
bool operator!=(const matrix_2d& lhs, const matrix_2d& rhs) noexcept;
```

3 *Returns:* !(lhs == rhs).

## 10 Paths

[paths]

- <sup>1</sup> Paths define geometric objects which can be stroked (Table 22), filled, masked, and used to define or modify a Clip Area (Table 21).
- <sup>2</sup> Paths are created using a `path_factory` object.
- <sup>3</sup> They provide vector graphics functionality and as such are particularly useful in situations where an application is intended to run on a variety of platforms whose output devices (13.13.2) span a large gamut of sizes, both physical and in terms of pixel dimensions.

### 10.1 Path geometries

[pathgeometries]

#### 10.1.1 Overview of path geometries

[pathgeometries.overview]

- <sup>1</sup> Path geometries are most easily formed using a `path_factory` object.
- <sup>2</sup> They may also be formed by directly creating and manipulating a `vector<path_data_item>` object.
- <sup>3</sup> A path geometry may contain degenerate path segments.
- <sup>4</sup> There are special rules concerning the rendering of degenerate path segments. As such they shall be added to a path geometry when requested and shall not be removed from a path geometry when processing it.
- <sup>5</sup> A `path` object is an immutable resource wrapper containing a path geometry graphics resource (10.4).

#### 10.1.2 Processing path geometries

[pathgeometries.processing]

- <sup>1</sup> 10.1.2 describes how a properly formed `vector<path_data_item>` object shall be interpreted by an implementation in order to convert it into a path geometry graphics resource.
- <sup>2</sup> The `native_geometry_collection` class, described below, is used for expository purposes only. It provides a hypothetical interface for forming a path geometry graphics resource object. It is not normative.
- <sup>3</sup> The `native_geometry_collection` class has the following state data, the types of which are unspecified:

Table 2 — `native_geometry_collection` state data

| Name                  | Use                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current Point         | The start point for a path segment that is added to the Current Path Geometry.                                                                                                                |
| Close Point           | The start point for the initial path segment in the Current Path Geometry.                                                                                                                    |
| Current Path Geometry | The path geometry to which path segments are added.                                                                                                                                           |
| Collection            | The collection of all path geometries added to the <code>native_geometry_collection</code> object. A new path geometry that is added to the collection is added to the end of the collection. |

```
// This class is exposition only
class native_geometry_collection {
public:
 void current_point(const vector_2d& pt) noexcept;
 void close_point(const vector_2d& pt) noexcept;
```

```

void line_to(const vector_2d& pt) noexcept;
void curve_to(const vector_2d& control1, const vector_2d& control2,
 const vector_2d& endPt) noexcept;
void close_path() noexcept;
};

```

```

void current_point(const vector_2d& pt) noexcept;

```

4     *Effects:* If the last member function called was not `current_point` or if the Collection contains no path geometries, a new path geometry is created, added to the Collection, and set as the Current Path Geometry.

5     If a new path geometry is created and the Collection contained at least one path geometry prior to this member function being called, then unless the last member function called was `close_path`, the previous Current Path Geometry shall be an open path geometry.

6     Sets `pt` as the Current Point.

```

void close_point(const vector_2d& pt) noexcept;

```

7     *Requires:* There is a Current Path Geometry.

8     *Effects:* Sets `pt` as the Close Point.

```

void line_to(const vector_2d& pt) noexcept;

```

9     *Requires:* There is a Current Path Geometry.

10    *Effects:* Creates a line segment from the Current Point to `pt` and adds it to the Current Path Geometry.

```

void curve_to(const vector_2d& cpt1, const vector_2d& cpt2,
 const vector_2d& endPt) noexcept;

```

11    *Requires:* There is a Current Path Geometry.

12    *Effects:* Creates a cubic Bézier curve from the Current Point to `endPt` using `cpt1` as the first control point and `cpt2` as the second control point and adds it to the Current Path Geometry.

```

void close_path() noexcept;

```

13    *Requires:* There is a Current Path Geometry.

14    *Effects:* Creates a line segment from the Current Point to the Close Point.

15    The Current Path Geometry becomes a closed path geometry.

16 [ *Note:* A path geometry graphics resource that only supports rendering triangles is possible. The triangles would be used to form lines and to approximate curves. This description assumes the existence of a path geometry graphics resource that performs those actions where needed. — *end note* ]

17 The following code shows how to properly process `vector<path_data_item> p` and store the results into `native_geometry_collection n`:

```

const double pi = 3.1415926535897932384626433832795;
const double halfpi = 1.57079632679489661923132169163985;
const double twopi = 6.283185307179586476925286766559;
matrix_2d m;
vector_2d origin;
vector_2d currentPoint;
bool hasCurrentPoint = false;
vector_2d closePoint;

```



```

for (const auto& item : p) {
 switch(item.type()) {
 case path_data_type::move_to:
 {
 currentPoint = item.get<path_data_item::move_to>().to();
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.current_point(pt);
 hasCurrentPoint = true;
 closePoint = pt;
 n.close_point(pt);
 } break;
 case path_data_type::line_to:
 {
 currentPoint = item.get<path_data_item::line_to>().to();
 auto pt = m.transform_point(currentPoint - origin) + origin;
 if (hasCurrentPoint) {
 n.line_to(pt);
 }
 else {
 n.current_point(pt);
 hasCurrentPoint = true;
 closePoint = pt;
 n.close_point(pt);
 }
 } break;
 case path_data_type::curve_to:
 {
 auto cd = item.get<path_data_item::curve_to>();
 auto pt1 = m.transform_point(cd.control_point_1() - origin) + origin;
 auto pt2 = m.transform_point(cd.control_point_2() - origin) + origin;
 auto pt3 = m.transform_point(cd.end_point() - origin) + origin;
 if (!hasCurrentPoint) {
 currentPoint = cd.control_point_1();
 n.current_point(pt1);
 hasCurrentPoint = true;
 closePoint = pt1;
 n.close_point(pt1);
 }
 n.curve_to(pt1, pt2, pt3);
 currentPoint = cd.end_point();
 } break;
 case path_data_type::new_sub_path:
 {
 hasCurrentPoint = false;
 } break;
 case path_data_type::close_path:
 {
 if (!hasCurrentPoint) {
 break;
 }
 n.close_path();
 n.current_point(closePoint);
 // Invert can error so use correct overload; here is the throw version.
 auto invM = matrix_2d{m}.invert();
 // Need the untransformed value for currentPoint.

```

```

 currentPoint = invM.transform_point(closePoint - origin) + origin;
} break;
case path_data_type::rel_move_to:
{
 // If !hasCurrentPoint, error is io2d_error::no_current_point;
 currentPoint = item.get<path_data_item::rel_move_to>().to() + currentPoint;
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.current_point(pt);
 hasCurrentPoint = true;
 closePoint = pt;
 n.close_point(pt);
} break;
case path_data_type::rel_line_to:
{
 // If !hasCurrentPoint, error is io2d_error::no_current_point;
 currentPoint = item.get<path_data_item::rel_line_to>().to() + currentPoint;
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.line_to(pt);
} break;
case path_data_type::rel_curve_to:
{
 // If !hasCurrentPoint, error is io2d_error::no_current_point;
 auto cd = item.get<path_data_item::rel_curve_to>();
 auto pt1 = m.transform_point(cd.control_point_1() + currentPoint -
 origin) + origin;
 auto pt2 = m.transform_point(cd.control_point_2() + currentPoint -
 origin) + origin;
 auto pt3 = m.transform_point(cd.end_point() + currentPoint - origin) +
 origin;
 n.curve_to(pt1, pt2, pt3);
 currentPoint = cd.end_point() + currentPoint;
} break;
case path_data_type::arc:
{
 auto ad = item.get<path_data_item::arc>();
 auto ctr = ad.center();
 auto rad = ad.radius();
 auto ang1 = ad.angle_1();
 auto ang2 = ad.angle_2();
 while(ang2 < ang1) {
 ang2 += twopi;
 }
 vector_2d pt0, pt1, pt2, pt3;
 int bezCount = 1;
 double theta = ang2 - ang1;
 double phi;
 while (theta >= halfpi) {
 theta /= 2.0;
 bezCount += bezCount;
 }
 phi = theta / 2.0;
 auto cosPhi = cos(phi);
 auto sinPhi = sin(phi);
 pt0.x(cosPhi);
 pt0.y(-sinPhi);

```

```

 pt3.x(pt0.x());
 pt3.y(-pt0.y());
 pt1.x((4.0 - cosPhi) / 3.0);
 pt1.y(-(((1.0 - cosPhi) * (3.0 - cosPhi)) / (3.0 * sinPhi)));
 pt2.x(pt1.x());
 pt2.y(-pt1.y());
 phi = -phi;
 auto rotCwFn = [](const vector_2d& pt, double a) -> vector_2d {
 return { pt.x() * cos(a) + pt.y() * sin(a),
 -(pt.x() * -(sin(a)) + pt.y() * cos(a)) };
 };
 pt0 = rotCwFn(pt0, phi);
 pt1 = rotCwFn(pt1, phi);
 pt2 = rotCwFn(pt2, phi);
 pt3 = rotCwFn(pt3, phi);

 auto currTheta = ang1;
 const auto startPt =
 ctr + rotCwFn({ pt0.x() * rad, pt0.y() * rad }, currTheta);
 if (hasCurrentPoint) {
 currentPoint = startPt;
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.line_to(pt);
 }
 else {
 currentPoint = startPt;
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.current_point(pt);
 hasCurrentPoint = true;
 closePt = pt;
 n.close_point(pt);
 }
 for (; bezCount > 0; bezCount--) {
 auto cpt1 = ctr + rotCwFn({ pt1.x() * rad, pt1.y() * rad }, currTheta);
 auto cpt2 = ctr + rotCwFn({ pt2.x() * rad, pt2.y() * rad }, currTheta);
 auto cpt3 = ctr + rotCwFn({ pt3.x() * rad, pt3.y() * rad }, currTheta);
 currentPoint = cpt3;
 cpt1 = m.transform_point(cpt1 - origin) + origin;
 cpt2 = m.transform_point(cpt2 - origin) + origin;
 cpt3 = m.transform_point(cpt3 - origin) + origin;
 n.curve_to(cpt1, cpt2, cpt3);
 currTheta += theta;
 }
} break;
case path_data_type::arc_negative:
{
 auto ad = item.get<path_data_item::arc_negative>();
 auto ctr = ad.center();
 auto rad = ad.radius();
 auto ang1 = ad.angle_1();
 auto ang2 = ad.angle_2();
 while(ang2 > ang1) {
 ang2 -= twopi;
 }
 vector_2d pt0, pt1, pt2, pt3;

```

```

int bezCount = 1;
double theta = ang1 - ang2;
double phi;
while (theta >= halfpi) {
 theta /= 2.0;
 bezCount += bezCount;
}
phi = theta / 2.0;
auto cosPhi = cos(phi);
auto sinPhi = sin(phi);
pt0.x(cosPhi);
pt0.y(-sinPhi);
pt3.x(pt0.x());
pt3.y(-pt0.y());
pt1.x((4.0 - cosPhi) / 3.0);
pt1.y(-(((1.0 - cosPhi) * (3.0 - cosPhi)) / (3.0 * sinPhi)));
pt2.x(pt1.x());
pt2.y(-pt1.y());
auto rotCwFn = [](const vector_2d& pt, double a) -> vector_2d {
 return { pt.x() * cos(a) + pt.y() * sin(a),
 -(pt.x() * -(sin(a)) + pt.y() * cos(a)) };
};
pt0 = rotCwFn(pt0, phi);
pt1 = rotCwFn(pt1, phi);
pt2 = rotCwFn(pt2, phi);
pt3 = rotCwFn(pt3, phi);
auto shflPt = pt3;
pt3 = pt0;
pt0 = shflPt;
shflPt = pt2;
pt2 = pt1;
pt1 = shflPt;
auto currTheta = ang1;
const auto startPt =
 ctr + rotCwFn({ pt0.x() * rad, pt0.y() * rad }, currTheta);
if (hasCurrentPoint) {
 currentPoint = startPt;
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.line_to(pt);
}
else {
 currentPoint = startPt;
 auto pt = m.transform_point(currentPoint - origin) + origin;
 n.current_point(pt);
 hasCurrentPoint = true;
 closePt = pt;
 n.close_point(pt);
}
for (; bezCount > 0; bezCount--) {
 auto cpt1 = ctr + rotCwFn({ pt1.x() * rad, pt1.y() * rad }, currTheta);
 auto cpt2 = ctr + rotCwFn({ pt2.x() * rad, pt2.y() * rad }, currTheta);
 auto cpt3 = ctr + rotCwFn({ pt3.x() * rad, pt3.y() * rad }, currTheta);
 currentPoint = cpt3;
 cpt1 = m.transform_point(cpt1 - origin) + origin;
 cpt2 = m.transform_point(cpt2 - origin) + origin;

```

```

 cpt3 = m.transform_point(cpt3 - origin) + origin;
 n.curve_to(cpt1, cpt2, cpt3);
 currTheta -= theta;
 }
} break;
case path_data_type::change_matrix:
{
 m = item.get<path_data_item::change_matrix>().matrix();
} break;
case path_data_type::change_origin:
{
 origin = item.get<path_data_item::change_origin>().origin();
} break;
}
}

```

## 10.2 Enum class path\_data\_type [pathdatatype]

### 10.2.1 path\_data\_type Summary [pathdatatype.summary]

- <sup>1</sup> The path\_data\_type enum class specifies the polymorphic type of a path\_data object. See Table 3 for the meaning of each path\_data\_type enumerator.

### 10.2.2 path\_data\_type Synopsis [pathdatatype.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 enum class path_data_type {
 move_to,
 line_to,
 curve_to,
 new_sub_path,
 close_path,
 rel_move_to,
 rel_line_to,
 rel_curve_to,
 arc,
 arc_negative,
 change_matrix,
 change_origin
 };
} } } }

```

### 10.2.3 path\_data\_type Enumerators [pathdatatype.enumerators]

Table 3 — path\_data\_type enumerator meanings

| Enumerator   | Meaning                             |
|--------------|-------------------------------------|
| move_to      | The object is of type move_to.      |
| line_to      | The object is of type line_to.      |
| curve_to     | The object is of type curve_to.     |
| new_sub_path | The object is of type new_sub_path. |
| close_path   | The object is of type close_path.   |
| rel_move_to  | The object is of type rel_move_to.  |
| rel_line_to  | The object is of type rel_line_to.  |
| rel_curve_to | The object is of type rel_curve_to. |

Table 3 — `path_data_type` enumerator meanings (continued)

| Enumerator                 | Meaning                                            |
|----------------------------|----------------------------------------------------|
| <code>arc</code>           | The object is of type <code>arc</code> .           |
| <code>arc_negative</code>  | The object is of type <code>arc_negative</code> .  |
| <code>change_matrix</code> | The object is of type <code>change_matrix</code> . |
| <code>change_origin</code> | The object is of type <code>change_origin</code> . |

### 10.3 Class `path_data_item`

[pathdataitem]

#### 10.3.1 `path_data_item` synopsis

[pathdataitem.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item {
 public:
 class path_data;
 class arc;
 class arc_negative;
 class change_matrix;
 class change_origin;
 class close_path;
 class curve_to;
 class line_to;
 class move_to;
 class new_sub_path;
 class rel_curve_to;
 class rel_line_to;
 class rel_move_to;
```

// 10.3.3, construct/copy/move/destroy:

```
path_data_item() noexcept;
path_data_item(const path_data_item& other) noexcept;
path_data_item& operator=(const path_data_item& other) noexcept;
path_data_item(path_data_item&& other) noexcept;
path_data_item& operator=(path_data_item&& other) noexcept;
path_data_item(const path_data_item::arc& value) noexcept;
path_data_item(const path_data_item::arc_negative& value) noexcept;
path_data_item(const path_data_item::change_matrix& value) noexcept;
path_data_item(const path_data_item::change_origin& value) noexcept;
path_data_item(const path_data_item::close_path& value) noexcept;
path_data_item(const path_data_item::curve_to& value) noexcept;
path_data_item(const path_data_item::rel_curve_to& value) noexcept;
path_data_item(const path_data_item::new_sub_path& value) noexcept;
path_data_item(const path_data_item::line_to& value) noexcept;
path_data_item(const path_data_item::move_to& value) noexcept;
path_data_item(const path_data_item::rel_line_to& value) noexcept;
path_data_item(const path_data_item::rel_move_to& value) noexcept;
```

// 10.3.4, modifiers:

```
void assign(const path_data_item::arc& value) noexcept;
void assign(const path_data_item::arc_negative& value) noexcept;
void assign(const path_data_item::change_matrix& value) noexcept;
void assign(const path_data_item::change_origin& value) noexcept;
void assign(const path_data_item::close_path& value) noexcept;
void assign(const path_data_item::curve_to& value) noexcept;
```

```

void assign(const path_data_item::rel_curve_to& value) noexcept;
void assign(const path_data_item::new_sub_path& value) noexcept;
void assign(const path_data_item::line_to& value) noexcept;
void assign(const path_data_item::move_to& value) noexcept;
void assign(const path_data_item::rel_line_to& value) noexcept;
void assign(const path_data_item::rel_move_to& value) noexcept;

// 10.3.5, observers:
bool has_data() const noexcept;
path_data_type type() const;
path_data_type type(error_code& ec) const noexcept;

template <class T>
T get() const;
template <class T>
T get(error_code& ec) const noexcept;

private:
bool _Has_data; // exposition only
union {
 struct {
 double centerX;
 double centerY;
 double radius;
 double angle1;
 double angle2;
 } arc;
 struct {
 double m00;
 double m01;
 double m10;
 double m11;
 double m20;
 double m21;
 } matrix;
 struct {
 double cpt1x;
 double cpt1y;
 double cpt2x;
 double cpt2y;
 double eptx;
 double epty;
 } curve;
 struct {
 double x;
 double y;
 } point;
} _Data; // exposition only

path_data_type _Type; // exposition only
};
} } }

```

### 10.3.2 path\_data\_item Description

[pathdataitem.intro]

- 1 The class `path_data_item` describes an opaque container capable of storing and retrieving an object of a type derived from `path_data`.

### 10.3.3 path\_data\_item constructors and assignment operators

[pathdataitem.cons]

```
path_data_item() noexcept;
```

- 1 *Effects:* Constructs an object of type `path_data_item`.

- 2 *Postconditions:* `_Has_data == false`.

```
path_data_item(const path_data_item::arc& value) noexcept;
```

- 3 *Effects:* Constructs an object of type `path_data_item`.

- 4 *Postconditions:* `_Has_data == true`.

- 5 `_Type == path_data_type::arc`.

- 6 `_Data.arc.centerX == value.center().x()`.

- 7 `_Data.arc.centerY == value.center().y()`.

- 8 `_Data.arc.radius == value.radius()`.

- 9 `_Data.arc.angle1 == value.angle_1()`.

- 10 `_Data.arc.angle2 == value.angle_2()`.

```
path_data_item(const path_data_item::arc_negative& value) noexcept;
```

- 11 *Effects:* Constructs an object of type `path_data_item`.

- 12 *Postconditions:* `_Has_data == true`.

- 13 `_Type == path_data_type::arc_negative`.

- 14 `_Data.arc.centerX == value.center().x()`.

- 15 `_Data.arc.centerY == value.center().y()`.

- 16 `_Data.arc.radius == value.radius()`.

- 17 `_Data.arc.angle1 == value.angle_1()`.

- 18 `_Data.arc.angle2 == value.angle_2()`.

```
path_data_item(const path_data_item::change_matrix& value) noexcept;
```

- 19 *Effects:* Constructs an object of type `path_data_item`.

- 20 *Postconditions:* `_Has_data == true`.

- 21 `_Type == path_data_type::change_matrix`.

- 22 `_Data.matrix.m00 == value.matrix().m00()`.

- 23 `_Data.matrix.m01 == value.matrix().m01()`.

- 24 `_Data.matrix.m10 == value.matrix().m10()`.

- 25 `_Data.matrix.m11 == value.matrix().m11()`.

- 26 `_Data.matrix.m20 == value.matrix().m20()`.

- 27 `_Data.matrix.m21 == value.matrix().m21()`.

```
path_data_item(const path_data_item::change_origin& value) noexcept;
```



```

28 Effects: Constructs an object of type path_data_item.
29 Postconditions: _Has_data == true.
30 _Type == path_data_type::change_origin.
31 _Data.point.x == value.origin().x().
32 _Data.point.y == value.origin().y().

 path_data_item(const path_data_item::close_path& value) noexcept;

33 Effects: Constructs an object of type path_data_item.
34 Postconditions: _Has_data == true.
35 _Type == path_data_type::close_path.

 path_data_item(const path_data_item::curve_to& value) noexcept;

36 Effects: Constructs an object of type path_data_item.
37 Postconditions: _Has_data == true.
38 _Type == path_data_type::curve_to.
39 _Data.curve.cpt1x == value.control_point_1().x().
40 _Data.curve.cpt1y == value.control_point_1().y().
41 _Data.curve.cpt2x == value.control_point_2().x().
42 _Data.curve.cpt2y == value.control_point_2().y().
43 _Data.curve.eptx == value.end_point().x().
44 _Data.curve.epty == value.end_point().y().

 path_data_item(const path_data_item::rel_curve_to& value) noexcept;

45 Effects: Constructs an object of type path_data_item.
46 Postconditions: _Has_data == true.
47 _Type == path_data_type::rel_curve_to.
48 _Data.curve.cpt1x == value.control_point_1().x().
49 _Data.curve.cpt1y == value.control_point_1().y().
50 _Data.curve.cpt2x == value.control_point_2().x().
51 _Data.curve.cpt2y == value.control_point_2().y().
52 _Data.curve.eptx == value.end_point().x().
53 _Data.curve.epty == value.end_point().y().

 path_data_item(const path_data_item::new_sub_path& value) noexcept;

54 Effects: Constructs an object of type path_data_item.
55 Postconditions: _Has_data == true.
56 _Type == path_data_type::new_sub_path.

 path_data_item(const path_data_item::line_to& value) noexcept;

```

```

57 Effects: Constructs an object of type path_data_item.
58 Postconditions: _Has_data == true.
59 _Type == path_data_type::line_to.
60 _Data.point.x == value.to().x().
61 _Data.point.y == value.to().y().

 path_data_item(const path_data_item::move_to& value) noexcept;

62 Effects: Constructs an object of type path_data_item.
63 Postconditions: _Has_data == true.
64 _Type == path_data_type::move_to.
65 _Data.point.x == value.to().x().
66 _Data.point.y == value.to().y().

 path_data_item(const path_data_item::rel_line_to& value) noexcept;

67 Effects: Constructs an object of type path_data_item.
68 Postconditions: _Has_data == true.
69 _Type == path_data_type::rel_line_to.
70 _Data.point.x == value.to().x().
71 _Data.point.y == value.to().y().

 path_data_item(const path_data_item::rel_move_to& value) noexcept;

72 Effects: Constructs an object of type path_data_item.
73 Postconditions: _Has_data == true.
74 _Type == path_data_type::rel_move_to.
75 _Data.point.x == value.to().x().
76 _Data.point.y == value.to().y().

```

### 10.3.4 `path_data_item` modifiers

[pathdataitem.modifiers]

```

void assign(const path_data_item::arc& value) noexcept;

1 Postconditions: _Has_data == true.
2 _Type == path_data_type::arc.
3 _Data.arc.centerX == value.center().x().
4 _Data.arc.centerY == value.center().y().
5 _Data.arc.radius == value.radius().
6 _Data.arc.angle1 == value.angle_1().
7 _Data.arc.angle2 == value.angle_2().

void assign(const path_data_item::arc_negative& value) noexcept;

```

```

8 Postconditions: _Has_data == true.
9 _Type == path_data_type::arc_negative.
10 _Data.arc.centerX == value.center().x().
11 _Data.arc.centerY == value.center().y().
12 _Data.arc.radius == value.radius().
13 _Data.arc.angle1 == value.angle_1().
14 _Data.arc.angle2 == value.angle_2().

void assign(const path_data_item::change_matrix& value) noexcept;

15 Postconditions: _Has_data == true.
16 _Type == path_data_type::change_matrix.
17 _Data.matrix.m00 == value.matrix().m00().
18 _Data.matrix.m01 == value.matrix().m01().
19 _Data.matrix.m10 == value.matrix().m10().
20 _Data.matrix.m11 == value.matrix().m11().
21 _Data.matrix.m20 == value.matrix().m20().
22 _Data.matrix.m21 == value.matrix().m21().

void assign(const path_data_item::change_origin& value) noexcept;

23 Postconditions: _Has_data == true.
24 _Type == path_data_type::change_origin.
25 _Data.point.x == value.origin().x().
26 _Data.point.y == value.origin().y().

void assign(const path_data_item::close_path& value) noexcept;

27 Postconditions: _Has_data == true.
28 _Type == path_data_type::close_path.

void assign(const path_data_item::curve_to& value) noexcept;

29 Postconditions: _Has_data == true.
30 _Type == path_data_type::curve_to.
31 _Data.curve.cpt1x == value.control_point_1().x().
32 _Data.curve.cpt1y == value.control_point_1().y().
33 _Data.curve.cpt2x == value.control_point_2().x().
34 _Data.curve.cpt2y == value.control_point_2().y().
35 _Data.curve.eptx == value.end_point().x().
36 _Data.curve.epty == value.end_point().y().

void assign(const path_data_item::rel_curve_to& value) noexcept;

```

```

37 Postconditions: _Has_data == true.
38 _Type == path_data_type::rel_curve_to.
39 _Data.curve.cpt1x == value.control_point_1().x().
40 _Data.curve.cpt1y == value.control_point_1().y().
41 _Data.curve.cpt2x == value.control_point_2().x().
42 _Data.curve.cpt2y == value.control_point_2().y().
43 _Data.curve.eptx == value.end_point().x().
44 _Data.curve.epty == value.end_point().y().

 void assign(const path_data_item::new_sub_path& value) noexcept;

45 Postconditions: _Has_data == true.
46 _Type == path_data_type::new_sub_path.

 void assign(const path_data_item::line_to& value) noexcept;

47 Postconditions: _Has_data == true.
48 _Type == path_data_type::line_to.
49 _Data.point.x == value.to().x().
50 _Data.point.y == value.to().y().

 void assign(const path_data_item::move_to& value) noexcept;

51 Postconditions: _Has_data == true.
52 _Type == path_data_type::move_to.
53 _Data.point.x == value.to().x().
54 _Data.point.y == value.to().y().

 void assign(const path_data_item::rel_line_to& value) noexcept;

55 Postconditions: _Has_data == true.
56 _Type == path_data_type::rel_line_to.
57 _Data.point.x == value.to().x().
58 _Data.point.y == value.to().y().

 void assign(const path_data_item::rel_move_to& value) noexcept;

59 Postconditions: _Has_data == true.
60 _Type == path_data_type::rel_move_to.
61 _Data.point.x == value.to().x().
62 _Data.point.y == value.to().y().

```

**10.3.5 path\_data\_item observers****[pathdataitem.observers]**

```
bool has_data() const noexcept;
```

1 *Returns:* `_Has_data`.

```
path_data_type type() const;
```

```
path_data_type type(error_code& ec) const noexcept;
```

2 *Returns:* `_Type`.

3 *Throws:* As specified in Error reporting (3).

4 *Error conditions:* `errc::operation_not_permitted` if `!this->has_data()`.

**10.3.6 path\_data\_item member function templates [pathdataitem.functiontemplates]**

1 The specializations of the `path_data_item::get` member function template are specified at 10.3.20.

**10.3.7 Class path\_data\_item::path\_data****[pathdataitem.pathdata]**

1 The class `path_data_item::path_data` serves as an abstract base class for classes that describe operations performed on path geometries.

**10.3.7.1 path\_data\_item::path\_data synopsis****[pathdataitem.pathdata.synopsis]**

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::path_data {
 public:
 // 10.3.7.2, construct/copy/move/destroy:
 path_data() noexcept;
 path_data(const path_data& other) noexcept;
 path_data& operator=(const path_data& other) noexcept;
 path_data(path_data&& other) noexcept;
 path_data& operator=(path_data&& other) noexcept;
 virtual ~path_data() noexcept;

 // 10.3.7.3, observers:
 virtual path_data_type type() const noexcept = 0;
 };
} } } }
```

**10.3.7.2 path\_data\_item::path\_data constructors and assignment operators****[pathdataitem.pathdata.cons]**

```
virtual ~path_data() noexcept;
```

1 *Effects:* Destroys an object of class `path_data_item::path_data`.

**10.3.7.3 path\_data\_item::path\_data observers****[pathdataitem.pathdata.observers]**

```
virtual path_data_type type() const noexcept = 0;
```

1 *Returns:* The `path_data_type` of the `path_data`-derived object.

2 *Note:* This is used for casting to the correct type when iterating through a collection of `path_data` objects.

### 10.3.8 Class `path_data_item::arc` [pathdataitem.arc]

- 1 The class `path_data_item::arc` describes an operation on a path geometry collection.
- 2 This operation creates a circular arc with clockwise rotation.
- 3 The unit for the values passed to and returned by `path_data_item::arc::angle_1` and `path_data_item::arc::angle_2` is the radian.
- 4 The arc's *start point* is `vector_2d{ *this.radius() * cos(*this.angle_1()), -( *this.radius() * -sin(*this.angle_1())) } + *this.center()`.
- 5 Its *end point* is `vector_2d{ *this.radius() * cos(*this.angle_2()), -( *this.radius() * -sin(*this.angle_2())) } + *this.center()`.
- 6 If the current path geometry has a current point, a line is created from the current point to the start point before this arc operation is processed. Otherwise the start point is set as the current point and last-move-to point of the current path geometry.
- 7 The arc rotates around the point returned by `*this.center()`.
- 8 The arc begins at its start point and proceeds clockwise until it reaches its end point.
- 9 The current point is set be to the arc's end point at the end of this operation.
- 10 For purposes of determining whether a point is on the arc, if the value returned by `*this.angle_2()` is less than the value returned by `*this.angle_1()` then the value returned by `*this.angle_2()` shall be continuously incremented by  $2\pi$  until it is greater than the value returned by `*this.angle_1()`.

#### 10.3.8.1 `path_data_item::arc` synopsis [pathdataitem.arc.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::arc : public path_data_item::path_data {
 public:
 // 10.3.8.2, construct/copy/move/destroy:
 arc() noexcept;
 arc(const arc&) noexcept;
 path_data_item::arc& operator=(const arc&) noexcept;
 arc(arc&&) noexcept;
 path_data_item::arc& operator=(arc&&) noexcept;
 arc(const vector_2d& ctr, double rad, double angle1, double angle2) noexcept;

 // 10.3.8.3, modifiers:
 void center(const vector_2d& value) noexcept;
 void radius(double value) noexcept;
 void angle_1(double radians) noexcept;
 void angle_2(double radians) noexcept;

 // 10.3.8.4, observers:
 vector_2d center() const noexcept;
 double radius() const noexcept;
 double angle_1() const noexcept;
 double angle_2() const noexcept;
 virtual path_data_type type() const noexcept override;

 private:
 vector_2d _Center; // exposition only
 double _Radius; // exposition only
 double _Angle_1; // exposition only
 double _Angle_2; // exposition only
 };
};
```

```
} } } }
```

### 10.3.8.2 path\_data\_item::arc constructors and assignment operators [pathdataitem.arc.cons]

```
arc() noexcept;
```

1 *Effects:* Constructs an object of type path\_data\_item::arc.

2 *Postconditions:* \_Center == vector\_2d(0.0, 0.0).

```
_Radius == 0.0.
```

```
_Angle_1 == 0.0.
```

```
_Angle_2 == 0.0.
```

```
arc(const vector_2d& ctr, double rad, double angle1, double angle2) noexcept;
```

3 *Effects:* Constructs an object of type path\_data\_item::arc.

4 *Postconditions:* \_Center == ctr.

```
_Radius == rad.
```

```
_Angle_1 == angle1.
```

```
_Angle_2 == angle2.
```

### 10.3.8.3 path\_data\_item::arc modifiers

[pathdataitem.arc.modifiers]

```
void center(const vector_2d& value) noexcept;
```

1 *Postconditions:* \_Center == value.

```
void radius(double value) noexcept;
```

2 *Postconditions:* \_Radius == value.

```
void angle_1(double value) noexcept;
```

3 *Postconditions:* \_Angle\_1 == value.

```
void angle_2(double value) noexcept;
```

4 *Postconditions:* \_Angle\_2 == value.

### 10.3.8.4 path\_data\_item::arc observers

[pathdataitem.arc.observers]

```
vector_2d center() const noexcept;
```

1 *Returns:* \_Center.

```
double radius() const noexcept;
```

2 *Returns:* \_Radius.

```
double angle_1() const noexcept;
```

3 *Returns:* \_Angle\_1.

```
double angle_2() const noexcept;
```

4 *Returns:* \_Angle\_2.

```
virtual path_data_type type() const noexcept override;
```

5 *Returns:* path\_data\_type::arc.

### 10.3.9 Class `path_data_item::arc_negative` [pathdataitem.arcnegative]

- 1 The class `path_data_item::arc_negative` describes an operation on a path geometry collection.
- 2 This operation creates a circular arc with counterclockwise rotation.
- 3 The unit for the values passed to and returned by `path_data_item::arc_negative::angle_1` and `path_data_item::arc_negative::angle_2` is the radian.
- 4 The arc's *start point* is `vector_2d{ *this.radius() * cos(*this.angle_1()), *this.radius() * -sin(*this.angle_1()) } + *this.center()`.
- 5 Its *end point* is `vector_2d{ *this.radius() * cos(*this.angle_2()), *this.radius() * -sin(*this.angle_2()) } + *this.center()`.
- 6 If the current path geometry has a current point, a line is created from the current point to the start point before this arc operation is processed. Otherwise the start point is set as the current point and last-move-to point of the current path geometry.
- 7 The arc rotates around the point returned by `*this.center()`.
- 8 The arc begins at its start point and proceeds counterclockwise until it reaches its end point.
- 9 The current point is set be to the arc's end point at the end of this operation.
- 10 For purposes of determining whether a point is on the arc, if the value returned by `*this.angle_2()` is greater than the value returned by `*this.angle_1()` then the value returned by `*this.angle_2()` shall be continuously decremented by  $2\pi$  until it is less than the value returned by `*this.angle_1()`.

#### 10.3.9.1 `path_data_item::arc_negative` synopsis [pathdataitem.arcnegative.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::arc_negative : public path_data_item::path_data {
 public:
 // 10.3.9.2, construct/copy/move/destroy:
 arc_negative() noexcept;
 arc_negative(const arc_negative&) noexcept;
 path_data_item::arc_negative& operator=(const arc_negative&) noexcept;
 arc_negative(arc_negative&&) noexcept;
 path_data_item::arc_negative& operator=(arc_negative&&) noexcept;
 arc_negative(const vector_2d& ctr, double rad, double angle1,
 double angle2) noexcept;

 // 10.3.9.3, modifiers:
 void center(const vector_2d& value) noexcept;
 void radius(double value) noexcept;
 void angle_1(double radians) noexcept;
 void angle_2(double radians) noexcept;

 // 10.3.9.4, observers:
 vector_2d center() const noexcept;
 double radius() const noexcept;
 double angle_1() const noexcept;
 double angle_2() const noexcept;
 virtual path_data_type type() const noexcept override;

 private:
 vector_2d _Center; // exposition only
 double _Radius; // exposition only
 double _Angle_1; // exposition only
 double _Angle_2; // exposition only
 };
};
```



```
};
} } } }
```

### 10.3.9.2 `path_data_item::arc_negative` constructors and assignment operators [pathdataitem.arcnegative.cons]

```
arc_negative() noexcept;
```

```
1 Effects: Constructs an object of type path_data_item::arc_negative.
2 Postconditions: _Center == vector_2d(0.0, 0.0).
 _Radius == 0.0.
 _Angle_1 == 0.0.
 _Angle_2 == 0.0.
```

```
arc_negative(const vector_2d& ctr, double rad, double angle1,
 double angle2) noexcept;
```

```
3 Effects: Constructs an object of type path_data_item::arc_negative.
4 Postconditions: _Center == ctr.
 _Radius == rad.
 _Angle_1 == angle1.
 _Angle_2 == angle2.
```

### 10.3.9.3 `path_data_item::arc_negative` modifiers [pathdataitem.arcnegative.modifiers]

```
void center(const vector_2d& value) noexcept;
```

```
1 Postconditions: _Center == value.
```

```
void radius(double value) noexcept;
```

```
2 Postconditions: _Radius == value.
```

```
void angle_1(double value) noexcept;
```

```
3 Postconditions: _Angle_1 == value.
```

```
void angle_2(double value) noexcept;
```

```
4 Postconditions: _Angle_2 == value.
```

### 10.3.9.4 `path_data_item::arc_negative` observers [pathdataitem.arcnegative.observers]

```
vector_2d center() const noexcept;
```

```
1 Returns: _Center.
```

```
double radius() const noexcept;
```

```
2 Returns: _Radius.
```

```
double angle_1() const noexcept;
```

```
3 Returns: _Angle_1.
```

```
double angle_2() const noexcept;
```

```
4 Returns: _Angle_2.
```

```
virtual path_data_type type() const noexcept override;
```

```
5 Returns: path_data_type::arc_negative.
```

### 10.3.10 Class `path_data_item::close_path` [pathdataitem.closepath]

- <sup>1</sup> The class `path_data_item::close_path` describes an operation on a path geometry collection.
- <sup>2</sup> If the current path geometry has a current point, this operation creates a line from the current point to the last-move-to point. It then starts a new path geometry and sets its current point and last-move-to point to the value of the previous path geometry's last-move-to point.
- <sup>3</sup> If there is no current point, then this operation does nothing. [ *Note*: Because this operation does nothing if there is no current point, there is no need to track whether or not a path geometry has a valid last-move-to point. This operation is the only operation that uses the last-move-to point and all operations that establish a current point for a path geometry also establish a valid last-move-to point for that path geometry. — *end note* ]

#### 10.3.10.1 `path_data_item::close_path` synopsis [pathdataitem.closepath.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::close_path : public path_data_item::path_data {
 public:
 // construct/copy/move/destroy:
 close_path() noexcept;
 close_path(const close_path&) noexcept;
 path_data_item::close_path& operator=(const close_path&) noexcept;
 close_path(close_path&&) noexcept;
 path_data_item::close_path& operator=(close_path&&) noexcept;

 // 10.3.10.2, observers:
 virtual path_data_type type() const noexcept override;
 };
} } } }
```

#### 10.3.10.2 `path_data_item::close_path` observers [pathdataitem.closepath.observers]

```
virtual path_data_type type() const noexcept override;
```

- <sup>1</sup> *Returns*: `path_data_type::close_path`.

### 10.3.11 Class `path_data_item::change_matrix` [pathdataitem.changematrix]

#### 10.3.11.1 `path_data_item::change_matrix` synopsis [pathdataitem.changematrix.synopsis]

- <sup>1</sup> The class `path_data_item::change_matrix` describes an operation on a path geometry collection.
- <sup>2</sup> This operation changes the transformation matrix for a path geometry collection to be the value returned by `*this.matrix()`. As shown in 10.1.2, the new transformation matrix does not affect any operations that came before this operation. It is only used in processing operations that come after it. It continues to be used until another `path_data_item::change_matrix` object is encountered or the end of the path geometry collection is reached.

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::change_matrix : public path_data_item::path_data {
 public:
 // 10.3.11.2, construct/copy/move/destroy:
 change_matrix() noexcept;
 change_matrix(const change_matrix&) noexcept;
 path_data_item::change_matrix& operator=(const change_matrix&) noexcept;
 change_matrix(change_matrix&&) noexcept;
 path_data_item::change_matrix& operator=(change_matrix&&) noexcept;
 explicit change_matrix(const matrix_2d& m) noexcept;
```

```

// 10.3.11.3, modifiers:
void matrix(const matrix_2d& value) noexcept;

// 10.3.11.4, observers:
matrix_2d matrix() const noexcept;
virtual path_data_type type() const noexcept override;

private:
 matrix_2d _Matrix; // exposition only
};
} } } }

```

### 10.3.11.2 `path_data_item::change_matrix` constructors and assignment operators [pathdataitem.changematrix.cons]

```

change_matrix() noexcept;
1 Effects: Constructs an object of type path_data_item::change_matrix.
2 Postconditions: _Matrix == matrix_2d{}.

explicit change_matrix(const matrix_2d& m) noexcept;
3 Effects: Constructs an object of type path_data_item::change_matrix.
4 Postconditions: _Matrix == m.

```

### 10.3.11.3 `path_data_item::change_matrix` modifiers [pathdataitem.changematrix.modifiers]

```

void matrix(const matrix_2d& value) noexcept;
1 Postconditions: _Matrix == value.

```

### 10.3.11.4 `path_data_item::change_matrix` observers [pathdataitem.changematrix.observers]

```

matrix_2d matrix() const noexcept;
1 Returns: _Matrix.

virtual path_data_type type() const noexcept override;
2 Returns: path_data_type::change_matrix.

```

## 10.3.12 Class `path_data_item::change_origin` [pathdataitem.changeorigin]

- 1 The class `path_data_item::change_origin` describes an operation on a path geometry collection.
- 2 This operation changes the origin point for a path geometry collection to be the value returned by `*this.origin()`. As shown in 10.1.2, the new origin point does not affect any operations that came before this operation. It is only used in processing operations that come after it. It continues to be used until another `path_data_item::change_origin` object is encountered or the end of the path geometry collection is reached.

### 10.3.12.1 `path_data_item::change_origin` synopsis [pathdataitem.changeorigin.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::change_origin : public path_data_item::path_data {
 public:
 // 10.3.12.2, construct/copy/move/destroy:
 change_origin() noexcept;
 change_origin(const change_origin&) noexcept;
 path_data_item::change_origin& operator=(const change_origin&) noexcept;
 }
}
}
}

```

```

 change_origin(change_origin&&) noexcept;
 path_data_item::change_origin& operator=(change_origin&&) noexcept;
 explicit change_origin(const vector_2d& pt) noexcept;

 // 10.3.12.3, modifiers:
 void origin(const vector_2d& value) noexcept;

 // 10.3.12.4, observers:
 vector_2d origin() const noexcept;
 virtual path_data_type type() const noexcept override;

private:
 vector_2d _Data; // exposition only
};
} } } }

```

### 10.3.12.2 `path_data_item::change_origin` constructors and assignment operators [pathdataitem.changeorigin.cons]

```

 change_origin() noexcept;
1 Effects: Constructs an object of type path_data_item::change_origin.
2 Postconditions: _Data == vector_2d(0.0, 0.0).

 explicit change_origin(const vector_2d& pt) noexcept;
3 Effects: Constructs an object of type path_data_item::change_origin.
4 Postconditions: _Data == pt.

```

### 10.3.12.3 `path_data_item::change_origin` modifiers [pathdataitem.changeorigin.modifiers]

```

 void origin(const vector_2d& value) noexcept;
1 Postconditions: _Data == value.

```

### 10.3.12.4 `change_origin` observers [pathdataitem.changeorigin.observers]

```

 vector_2d origin() const noexcept;
1 Returns: _Data.

 virtual path_data_type type() const noexcept override;
2 Returns: path_data_type::change_origin.

```

## 10.3.13 Class `path_data_item::curve_to` [pathdataitem.curveto]

- 1 The class `path_data_item::curve_to` describes an operation on a path geometry collection.
- 2 If the current path geometry has no current point, then this operation behaves exactly as if this object was preceded by a `path_data_item::move_to` object constructed with the value returned by `*this.control_point_1()` as its argument.
- 3 This operation creates a cubic Bézier curve from the current point to the point returned by `*this.end_point()`, with the first control point being the point returned by `*this.control_point_1()` and the second control point being the point returned by `*this.control_point_2()`. It then sets the current point to be the point returned by `*this.end_point()`.

### 10.3.13.1 `path_data_item::curve_to` synopsis [pathdataitem.curveto.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::curve_to : public path_data_item::path_data {
 public:
 // 10.3.13.2, construct/copy/move/destroy:
 curve_to() noexcept;
 curve_to(const curve_to&) noexcept;
 path_data_item::curve_to& operator=(const curve_to&) noexcept;
 curve_to(curve_to&&) noexcept;
 path_data_item::curve_to& operator=(curve_to&&) noexcept;
 curve_to(const vector_2d& controlPoint1, const vector_2d& controlPoint2,
 const vector_2d& endPoint) noexcept;

 // 10.3.13.3, modifiers:
 void control_point_1(const vector_2d& value) noexcept;
 void control_point_2(const vector_2d& value) noexcept;
 void end_point(const vector_2d& value) noexcept;

 // 10.3.13.4, observers:
 vector_2d control_point_1() const noexcept;
 vector_2d control_point_2() const noexcept;
 vector_2d end_point() const noexcept;
 virtual path_data_type type() const noexcept override;

 private:
 vector_2d _Control_pt1; // exposition only
 vector_2d _Control_pt2; // exposition only
 vector_2d _End_pt; // exposition only
 };
} } } }

```

### 10.3.13.2 `path_data_item::curve_to` constructors and assignment operators [pathdataitem.curveto.cons]

```

curve_to() noexcept;

```

1     *Effects:* Constructs an object of type `path_data_item::curve_to`.

2     *Postconditions:* `_Control_pt1 == vector_2d(0.0, 0.0).`  
       `_Control_pt2 == vector_2d(0.0, 0.0).`  
       `_End_pt == vector_2d(0.0, 0.0).`

```

curve_to(const vector_2d& controlPoint1, const vector_2d& controlPoint2,
 const vector_2d& endPoint) noexcept;

```

3     *Effects:* Constructs an object of type `path_data_item::curve_to`.

4     *Postconditions:* `_Control_pt1 == controlPoint1.`  
       `_Control_pt2 == controlPoint2.`  
       `_End_pt == endPoint.`

### 10.3.13.3 `path_data_item::curve_to` modifiers [pathdataitem.curveto.modifiers]

```

void control_point_1(const vector_2d& value) noexcept;

```

1     *Postconditions:* `_Control_pt_1 == value.`

```
void control_point_2(const vector_2d& value) noexcept;
```

```
2 Postconditions: _Control_pt_2 == value.
```

```
void end_point(const vector_2d& value) noexcept;
```

```
3 Postconditions: _End_pt == value.
```

#### 10.3.13.4 path\_data\_item::curve\_to observers [pathdataitem.curveto.observers]

```
vector_2d control_point_1() const noexcept;
```

```
1 Returns: _Control_pt_1.
```

```
vector_2d control_point_2() const noexcept;
```

```
2 Returns: _Control_pt_2.
```

```
vector_2d end_point() const noexcept;
```

```
3 Returns: _End_pt.
```

```
virtual path_data_type type() const noexcept override;
```

```
4 Returns: path_data_type::curve_to.
```

#### 10.3.14 Class path\_data\_item::line\_to [pathdataitem.lineto]

1 The class `path_data_item::line_to` describes an operation on a path geometry collection.

2 If the current path geometry has a current point then this operation creates a line from the current point to the point returned by `*this.to()` and then sets current point to be the point returned by `*this.to()`. Otherwise, this operation behaves exactly as if this object was a `path_data_item::move_to` object constructed which was constructed with the value returned by `*this.to()` as its argument.

##### 10.3.14.1 path\_data\_item::line\_to synopsis [pathdataitem.lineto.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::line_to : public path_data_item::path_data {
 public:
 // 10.3.14.2, construct/copy/move/destroy:
 line_to() noexcept;
 line_to(const line_to&) noexcept;
 path_data_item::line_to& operator=(const line_to&) noexcept;
 line_to(line_to&&) noexcept;
 path_data_item::line_to& operator=(line_to&&) noexcept;
 explicit line_to(const vector_2d& pt) noexcept;

 // 10.3.14.3, modifiers:
 void to(const vector_2d& pt) noexcept;

 // 10.3.14.4, observers:
 vector_2d to() const noexcept;
 virtual path_data_type type() const noexcept override;

 private:
 vector_2d _Data; // exposition only
 };
} } } }
```

### 10.3.14.2 `path_data_item::line_to` constructors and assignment operators [pathdataitem.lineto.cons]

```
line_to() noexcept;
```

1 *Effects:* Constructs an object of type `path_data_item::line_to`.

2 *Postconditions:* `_Data == vector_2d(0.0, 0.0)`.

```
explicit line_to(const vector_2d& pt) noexcept;
```

3 *Effects:* Constructs an object of type `path_data_item::line_to`.

4 *Postconditions:* `_Data == pt`.

### 10.3.14.3 `path_data_item::line_to` modifiers [pathdataitem.lineto.modifiers]

```
void to(const vector_2d& pt) noexcept;
```

1 *Postconditions:* `_Data == pt`.

### 10.3.14.4 `path_data_item::line_to` observers [pathdataitem.lineto.observers]

```
vector_2d to() const noexcept;
```

1 *Returns:* `_Data`.

```
virtual path_data_type type() const noexcept override;
```

2 *Returns:* `path_data_type::line_to`.

## 10.3.15 Class `path_data_item::move_to` [pathdataitem.moveto]

1 The class `path_data_item::move_to` describes an operation on a path geometry collection.

2 This operation starts a new path geometry and sets its current point and last-move-to point to the value of `*this.to()`.

### 10.3.15.1 `path_data_item::move_to` synopsis [pathdataitem.moveto.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::move_to : public path_data_item::path_data {
 public:
 // 10.3.15.2, construct/copy/move/destroy:
 move_to() noexcept;
 move_to(const move_to&) noexcept;
 path_data_item::move_to& operator=(const move_to&) noexcept;
 move_to(move_to&&) noexcept;
 path_data_item::move_to& operator=(move_to&&) noexcept;
 explicit move_to(const vector_2d& pt) noexcept;

 // 10.3.15.3, modifiers:
 void to(const vector_2d& pt) noexcept;

 // 10.3.15.4, observers:
 vector_2d to() const noexcept;
 virtual path_data_type type() const noexcept override;

 private:
 vector_2d _Data; // exposition only
 };
} } } }
```

### 10.3.15.2 `path_data_item::move_to` constructors and assignment operators [pathdataitem.moveto.cons]

```
move_to() noexcept;
```

1 *Effects:* Constructs an object of type `path_data_item::move_to`.

2 *Postconditions:* `_Data == vector_2d(0.0, 0.0)`.

```
explicit move_to(const vector_2d& pt) noexcept;
```

3 *Effects:* Constructs an object of type `path_data_item::move_to`.

4 *Postconditions:* `_Data == pt`.

### 10.3.15.3 `path_data_item::move_to` modifiers [pathdataitem.moveto.modifiers]

```
void to(const vector_2d& pt) noexcept;
```

1 *Postconditions:* `_Data == pt`.

### 10.3.15.4 `path_data_item::move_to` observers [pathdataitem.moveto.observers]

```
vector_2d to() const noexcept;
```

1 *Returns:* `_Data`.

```
virtual path_data_type type() const noexcept override;
```

2 *Returns:* `path_data_type::move_to`.

### 10.3.16 Class `path_data_item::new_sub_path` [pathdataitem.newsubpath]

1 The class `path_data_item::new_sub_path` describes an operation on a path geometry collection.

2 This operation starts a new path geometry. The new path geometry has no current point.

#### 10.3.16.1 `path_data_item::new_sub_path` synopsis [pathdataitem.newsubpath.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::new_sub_path : public path_data_item::path_data {
 public:
 // construct/copy/move/destroy:
 new_sub_path() noexcept;
 new_sub_path(const new_sub_path&) noexcept;
 path_data_item::new_sub_path& operator=(const new_sub_path&) noexcept;
 new_sub_path(new_sub_path&&) noexcept;
 path_data_item::new_sub_path& operator=(new_sub_path&&) noexcept;

 // 10.3.16.2, observers:
 virtual path_data_type type() const noexcept override;
 };
} } } }
```

#### 10.3.16.2 `path_data_item::new_sub_path` observers [pathdataitem.newsubpath.observers]

```
virtual path_data_type type() const noexcept override;
```

1 *Returns:* `path_data_type::new_sub_path`.



### 10.3.17 Class `path_data_item::rel_curve_to` [pathdataitem.relcurveto]

- <sup>1</sup> The class `path_data_item::rel_curve_to` describes an operation on a path geometry collection.
- <sup>2</sup> This operation creates a cubic Bézier curve from the current point to the point that is the sum of the current point and the point returned by `*this.end_point()`, with the first control point being the point that is the sum of the current point and the point returned by `*this.control_point_1()` and the second control point being the point that is the sum of the current point and the point returned by `*this.control_point_2()`. It then sets the current point to be the point that is the sum of the current point and the point returned by `*this.end_point()`.
- <sup>3</sup> If the current path geometry does not have a current point when this operation is requested the path geometry collection is malformed.

#### 10.3.17.1 `path_data_item::rel_curve_to` synopsis [pathdataitem.relcurveto.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::rel_curve_to : public path_data_item::path_data {
 public:
 // 10.3.17.2, construct/copy/move/destroy:
 rel_curve_to() noexcept;
 rel_curve_to(const rel_curve_to&) noexcept;
 path_data_item::rel_curve_to& operator=(const rel_curve_to&) noexcept;
 rel_curve_to(rel_curve_to&&) noexcept;
 path_data_item::rel_curve_to& operator=(rel_curve_to&&) noexcept;
 rel_curve_to(const vector_2d& controlPoint1, const vector_2d& controlPoint2,
 const vector_2d& endPoint) noexcept;

 // 10.3.17.3, modifiers:
 void control_point_1(const vector_2d& value) noexcept;
 void control_point_2(const vector_2d& value) noexcept;
 void end_point(const vector_2d& value) noexcept;

 // 10.3.17.4, observers:
 vector_2d control_point_1() const noexcept;
 vector_2d control_point_2() const noexcept;
 vector_2d end_point() const noexcept;
 virtual path_data_type type() const noexcept override;

 private:
 vector_2d _Control_pt1; // exposition only
 vector_2d _Control_pt2; // exposition only
 vector_2d _End_pt; // exposition only
 };
} } }
```

#### 10.3.17.2 `path_data_item::rel_curve_to` constructors and assignment operators [pathdataitem.relcurveto.cons]

```
rel_curve_to() noexcept;
```

- <sup>1</sup> *Effects:* Constructs an object of type `path_data_item::rel_curve_to`.
- <sup>2</sup> *Postconditions:* `_Control_pt1 == vector_2d(0.0, 0.0).`  
`_Control_pt2 == vector_2d(0.0, 0.0).`  
`_End_pt == vector_2d(0.0, 0.0).`

```
rel_curve_to(const vector_2d& controlPoint1, const vector_2d& controlPoint2,
 const vector_2d& endPoint) noexcept;
```

3     *Effects:* Constructs an object of type `path_data_item::rel_curve_to`.

4     *Postconditions:* `_Control_pt1 == controlPoint1`.

`_Control_pt2 == controlPoint2`.

`_End_pt == endPoint`.

### 10.3.17.3 `path_data_item::rel_curve_to` modifiers [pathdataitem.relcurveto.modifiers]

```
void control_point_1(const vector_2d& value) noexcept;
```

1     *Postconditions:* `_Control_pt_1 == value`.

```
void control_point_2(const vector_2d& value) noexcept;
```

2     *Postconditions:* `_Control_pt_2 == value`.

```
void end_point(const vector_2d& value) noexcept;
```

3     *Postconditions:* `_End_pt == value`.

### 10.3.17.4 `path_data_item::rel_curve_to` observers [pathdataitem.relcurveto.observers]

```
vector_2d control_point_1() const noexcept;
```

1     *Returns:* `_Control_pt_1`.

```
vector_2d control_point_2() const noexcept;
```

2     *Returns:* `_Control_pt_2`.

```
vector_2d end_point() const noexcept;
```

3     *Returns:* `_End_pt`.

```
virtual path_data_type type() const noexcept override;
```

4     *Returns:* `path_data_type::rel_curve_to`.

## 10.3.18 Class `path_data_item::rel_line_to` [pathdataitem.rellineto]

1     The class `path_data_item::rel_line_to` describes an operation on a path geometry collection.

2     This operation creates a line from the current point to the point that is the sum of the current point and the point returned by `*this.to()`. It then sets current point to be the sum of the current point and the point returned by `*this.to()`.

3     If the current path geometry does not have a current point when this operation is requested the path geometry collection is malformed.

### 10.3.18.1 `path_data_item::rel_line_to` synopsis [pathdataitem.rellineto.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::rel_line_to : public path_data_item::path_data {
 public:
 // 10.3.18.2, construct/copy/move/destroy:
 rel_line_to() noexcept;
 rel_line_to(const line_to&) noexcept;
 path_data_item::rel_line_to& operator=(const line_to&) noexcept;
 rel_line_to(line_to&&) noexcept;
```

```

 path_data_item::rel_line_to& operator=(line_to&&) noexcept;
 explicit rel_line_to(const vector_2d& pt) noexcept;

 // 10.3.18.3, modifiers:
 void to(const vector_2d& pt) noexcept;

 // 10.3.18.4, observers:
 vector_2d to() const noexcept;
 virtual path_data_type type() const noexcept override;

private:
 vector_2d _Data; // exposition only
};
} } } }

```

### 10.3.18.2 `path_data_item::rel_line_to` constructors and assignment operators [pathdataitem.rellineto.cons]

```

 rel_line_to() noexcept;

1 Effects: Constructs an object of type path_data_item::rel_line_to.
2 Postconditions: _Data == vector_2d(0.0, 0.0).

 explicit rel_line_to(const vector_2d& pt) noexcept;

3 Effects: Constructs an object of type path_data_item::rel_line_to.
4 Postconditions: _Data == pt.

```

### 10.3.18.3 `path_data_item::rel_line_to` modifiers [pathdataitem.rellineto.modifiers]

```

 void to(const vector_2d& pt) noexcept;

1 Postconditions: _Data == pt.

```

### 10.3.18.4 `path_data_item::rel_line_to` observers [pathdataitem.rellineto.observers]

```

 vector_2d to() const noexcept;

1 Returns: _Data.

 virtual path_data_type type() const noexcept override;

2 Returns: path_data_type::rel_line_to.

```

## 10.3.19 Class `path_data_item::rel_move_to` [pathdataitem.relmoveto]

- 1 The class `path_data_item::rel_move_to` describes an operation on a path geometry collection.
- 2 This operation starts a new path geometry and sets its current point and last-move-to point to the point that is the sum of the previous path geometry's current point and the point returned by `*this.to()`.
- 3 If the existing path geometry does not have a current point when this operation is requested the path geometry collection is malformed.

### 10.3.19.1 `path_data_item::rel_move_to` synopsis [pathdataitem.relmoveto.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_data_item::rel_move_to : public path_data_item::path_data {
 public:
 // 10.3.19.2, construct/copy/move/destroy:

```

```

 rel_move_to() noexcept;
 rel_move_to(const rel_move_to&) noexcept;
 path_data_item::rel_move_to& operator=(const rel_move_to&) noexcept;
 rel_move_to(rel_move_to&&) noexcept;
 path_data_item::rel_move_to& operator=(rel_move_to&&) noexcept;
 explicit rel_move_to(const vector_2d& pt) noexcept;

 // 10.3.19.3, modifiers:
 void to(const vector_2d& pt) noexcept;

 // 10.3.19.4, observers:
 vector_2d to() const noexcept;
 virtual path_data_type type() const noexcept override;

private:
 vector_2d _Data; // exposition only
};
} } } }

```

### 10.3.19.2 `path_data_item::rel_move_to` constructors and assignment operators [pathdataitem.relmoveto.cons]

```

 rel_move_to() noexcept;
1 Effects: Constructs an object of type path_data_item::rel_move_to.
2 Postconditions: _Data == vector_2d(0.0, 0.0).

 explicit rel_move_to(const vector_2d& pt) noexcept;
3 Effects: Constructs an object of type path_data_item::rel_move_to.
4 Postconditions: _Data == pt.

```

### 10.3.19.3 `path_data_item::rel_move_to` modifiers [pathdataitem.relmoveto.modifiers]

```

 void to(const vector_2d& pt) noexcept;
1 Postconditions: _Data == pt.

```

### 10.3.19.4 `path_data_item::rel_move_to` observers [pathdataitem.relmoveto.observers]

```

 vector_2d to() const noexcept;
1 Returns: _Data.

 virtual path_data_type type() const noexcept override;
2 Returns: path_data_type::rel_move_to.

```

## 10.3.20 Class `path_data_item::get` member function template specializations [pathdataitem.get]

### 10.3.20.1 `path_data_item::get` synopsis [pathdataitem.get.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 template <>
 path_data_item::arc path_data_item::get() const;
 template <>
 path_data_item::arc path_data_item::get(error_code& ec) const noexcept;

```

```

template <>
path_data_item::arc_negative path_data_item::get() const;
template <>
path_data_item::arc_negative path_data_item::get(error_code& ec) const
 noexcept;

template <>
inline path_data_item::change_matrix path_data_item::get() const;
template <>
path_data_item::change_matrix path_data_item::get(error_code& ec) const
 noexcept;

template <>
path_data_item::change_origin path_data_item::get() const;
template <>
path_data_item::change_origin path_data_item::get(error_code& ec) const
 noexcept;

template <>
path_data_item::close_path path_data_item::get() const;
template <>
path_data_item::close_path path_data_item::get(error_code& ec) const noexcept;

template <>
path_data_item::curve_to path_data_item::get() const;
template <>
path_data_item::curve_to path_data_item::get(error_code& ec) const noexcept;

template <>
path_data_item::rel_curve_to path_data_item::get() const;
template <>
path_data_item::rel_curve_to path_data_item::get(error_code& ec) const
 noexcept;

template <>
path_data_item::new_sub_path path_data_item::get() const;
template <>
path_data_item::new_sub_path path_data_item::get(error_code& ec) const
 noexcept;

template <>
path_data_item::line_to path_data_item::get() const;
template <>
path_data_item::line_to path_data_item::get(error_code& ec) const noexcept;

template <>
path_data_item::move_to path_data_item::get() const;
template <>
path_data_item::move_to path_data_item::get(error_code& ec) const noexcept;

template <>
path_data_item::rel_line_to path_data_item::get() const;
template <>
path_data_item::rel_line_to path_data_item::get(error_code& ec) const
 noexcept;

```

```

template <>
path_data_item::rel_move_to path_data_item::get() const;
template <>
path_data_item::rel_move_to path_data_item::get(error_code& ec) const
 noexcept;
} } } }

```

### 10.3.20.2 path\_data\_item::get specializations

[pathdataitem.get.specializations]

```

template <>
path_data_item::arc path_data_item::get() const;
template <>
path_data_item::arc path_data_item::get(error_code& ec) const noexcept;

```

- 1 *Returns:* A copy of the stored path\_data\_item::arc object.
- 2 If an error occurs and the function was called with an error\_code& argument, returns path\_data\_item::arc{ }.
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Error conditions:* errc::operation\_not\_permitted if !this->has\_data().
- 5 errc::invalid\_argument if this->type() != path\_data\_type::arc.

```

template <>
path_data_item::arc_negative path_data_item::get() const;
template <>
path_data_item::arc_negative path_data_item::get(error_code& ec) const noexcept;

```

- 6 *Returns:* A copy of the stored path\_data\_item::arc\_negative object.
- 7 If an error occurs and the function was called with an error\_code& argument, returns path\_data\_item::arc\_negative{ }.
- 8 *Throws:* As specified in Error reporting (3).
- 9 *Error conditions:* errc::operation\_not\_permitted if !this->has\_data().
- 10 errc::invalid\_argument if this->type() != path\_data\_type::arc\_negative.

```

template <>
inline path_data_item::change_matrix path_data_item::get() const;
template <>
path_data_item::change_matrix path_data_item::get(error_code& ec) const
 noexcept;

```

- 11 *Returns:* A copy of the stored path\_data\_item::change\_matrix object.
- 12 If an error occurs and the function was called with an error\_code& argument, returns path\_data\_item::change\_matrix{ }.
- 13 *Throws:* As specified in Error reporting (3).
- 14 *Error conditions:* errc::operation\_not\_permitted if !this->has\_data().
- 15 errc::invalid\_argument if this->type() != path\_data\_type::change\_matrix.

```

template <>
path_data_item::change_origin path_data_item::get() const;
template <>
path_data_item::change_origin path_data_item::get(error_code& ec) const
 noexcept;

```

16     *Returns:* A copy of the stored `path_data_item::change_origin` object.

17     If an error occurs and the function was called with an `error_code&` argument, returns `path_data_item::change_origin{ }`.

18     *Throws:* As specified in Error reporting (3).

19     *Error conditions:* `errc::operation_not_permitted` if `!this->has_data()`.

20     `errc::invalid_argument` if `this->type() != path_data_type::change_origin`.

```
template <>
path_data_item::close_path path_data_item::get() const;
template <>
path_data_item::close_path path_data_item::get(error_code& ec) const noexcept;
```

21     *Returns:* A copy of the stored `path_data_item::close_path` object.

22     If an error occurs and the function was called with an `error_code&` argument, returns `path_data_item::close_path{ }`.

23     *Throws:* As specified in Error reporting (3).

24     *Error conditions:* `errc::operation_not_permitted` if `!this->has_data()`.

25     `errc::invalid_argument` if `this->type() != path_data_type::close_path`.

```
template <>
path_data_item::rel_curve_to path_data_item::get() const;
template <>
path_data_item::rel_curve_to path_data_item::get(error_code& ec) const noexcept;
```

26     *Returns:* A copy of the stored `path_data_item::rel_curve_to` object.

27     If an error occurs and the function was called with an `error_code&` argument, returns `path_data_item::rel_curve_to{ }`.

28     *Throws:* As specified in Error reporting (3).

29     *Error conditions:* `errc::operation_not_permitted` if `!this->has_data()`.

30     `errc::invalid_argument` if `this->type() != path_data_type::rel_curve_to`.

```
template <>
path_data_item::new_sub_path path_data_item::get() const;
template <>
path_data_item::new_sub_path path_data_item::get(error_code& ec) const noexcept;
```

31     *Returns:* A copy of the stored `path_data_item::new_sub_path` object.

32     If an error occurs and the function was called with an `error_code&` argument, returns `path_data_item::new_sub_path{ }`.

33     *Throws:* As specified in Error reporting (3).

34     *Error conditions:* `errc::operation_not_permitted` if `!this->has_data()`.

35     `errc::invalid_argument` if `this->type() != path_data_type::new_sub_path`.

```
template <>
path_data_item::line_to path_data_item::get() const;
template <>
path_data_item::line_to path_data_item::get(error_code& ec) const noexcept;
```

```

36 Returns: A copy of the stored path_data_item::line_to object.
37 If an error occurs and the function was called with an error_code& argument, returns path_data_
 item::line_to{ }.
38 Throws: As specified in Error reporting (3).
39 Error conditions: errc::operation_not_permitted if !this->has_data().
40 errc::invalid_argument if this->type() != path_data_type::line_to.

 template <>
 path_data_item::move_to path_data_item::get() const;
 template <>
 path_data_item::move_to path_data_item::get(error_code& ec) const noexcept;
41 Returns: A copy of the stored path_data_item::move_to object.
42 If an error occurs and the function was called with an error_code& argument, returns path_data_
 item::move_to{ }.
43 Throws: As specified in Error reporting (3).
44 Error conditions: errc::operation_not_permitted if !this->has_data().
45 errc::invalid_argument if this->type() != path_data_type::move_to.

 template <>
 path_data_item::rel_line_to path_data_item::get() const;
 template <>
 path_data_item::rel_line_to path_data_item::get(error_code& ec) const noexcept;
46 Returns: A copy of the stored path_data_item::rel_line_to object.
47 If an error occurs and the function was called with an error_code& argument, returns path_data_
 item::rel_line_to{ }.
48 Throws: As specified in Error reporting (3).
49 Error conditions: errc::operation_not_permitted if !this->has_data().
50 errc::invalid_argument if this->type() != path_data_type::rel_line_to.

 template <>
 path_data_item::rel_move_to path_data_item::get() const;
 template <>
 path_data_item::rel_move_to path_data_item::get(error_code& ec) const noexcept;
51 Returns: A copy of the stored path_data_item::rel_move_to object.
52 If an error occurs and the function was called with an error_code& argument, returns path_data_
 item::rel_move_to{ }.
53 Throws: As specified in Error reporting (3).
54 Error conditions: errc::operation_not_permitted if !this->has_data().
55 errc::invalid_argument if this->type() != path_data_type::rel_move_to.

```

## 10.4 Class path

[path]

- 1 The `path` class represents an immutable resource wrapper containing a path geometry graphics resource.
- 2 When a `path` object is set on a `surface` object using `surface::path`, the geometric paths represented by it can be stroked or filled.
- 3 A `path` object shall be usable with any `surface` or `surface-derived` object.



### 10.4.1 path synopsis

[path.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path {
 public:
 // 10.4.2, construct/copy/destroy:
 path() = delete;
 explicit path(const path_factory& pb);
 path(const path_factory& pb, error_code& ec) noexcept;
 explicit path(const vector<path_data_item>& p);
 path(const vector<path_data_item>& p, error_code& ec) noexcept;
 path(const path&) noexcept;
 path& operator=(const path&) noexcept;
 path(path&&) noexcept;
 path& operator=(path&&) noexcept;
 };
} } } }

```

### 10.4.2 path constructors and assignment operators

[path.cons]

```

explicit path(const path_factory& pb);
path(const path_factory& pb, error_code& ec) noexcept;
explicit path(const vector<path_data_item>& p);
path(const vector<path_data_item>& p, error_code& ec) noexcept;

```

- 1 *Effects:* Constructs an object of class `path`. Implementations shall create a path geometry graphics resource from the path geometries contained in `p` or `pb.data_ref()` as if they followed the procedure set forth in 10.1.2.
- 2 *Throws:* As specified in Error reporting (3).
- 3 *Remarks:* It is unspecified whether a `path` object shall require further processing when it is passed as an argument to a `surface` or `surface`-derived object.
- 4 Implementations should avoid or minimize the need for further processing of a `path` object after it has been constructed.
- 5 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.
- 6 `io2d_error::no_current_point` if, when processing the path geometries, an operation was encountered which required a current point and the current path geometry had no current point.
- 7 `io2d_error::invalid_matrix` if, when processing the path geometries, an operation was encountered which required the current transformation matrix to be invertible and the matrix was not invertible.

## 10.5 Class path\_factory

[pathfactory]

- 1 The `path_factory` class is a factory class used to create path geometry collection data from which `path` objects are created.

### 10.5.1 path\_factory synopsis

[pathfactory.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
 class path_factory {
 public:
 // 10.5.2, construct/copy/destroy:
 path_factory() noexcept;
 path_factory(const path_factory&);
 path_factory& operator=(const path_factory&);
 path_factory(path_factory&&) noexcept;
 };
} } } }

```

```

path_factory& operator=(path_factory&&) noexcept;

// 10.5.3, modifiers:
void append(const path_factory& p);
void append(const path_factory& p, error_code& ec) noexcept;
void append(const vector<path_data_item>& p);
void append(const vector<path_data_item>& p, error_code& ec) noexcept;
void new_sub_path();
void new_sub_path(error_code& ec) noexcept;
void close_path();
void close_path(error_code& ec) noexcept;
void arc(const vector_2d& center, double radius, double angle1,
 double angle2);
void arc(const vector_2d& center, double radius, double angle1,
 double angle2, error_code& ec) noexcept;
void arc_negative(const vector_2d& center, double radius, double angle1,
 double angle2);
void arc_negative(const vector_2d& center, double radius, double angle1,
 double angle2, error_code& ec) noexcept;
void curve_to(const vector_2d& pt0, const vector_2d& pt1,
 const vector_2d& pt2);
void curve_to(const vector_2d& pt0, const vector_2d& pt1,
 const vector_2d& pt2, error_code& ec) noexcept;
void line_to(const vector_2d& pt);
void line_to(const vector_2d& pt, error_code& ec) noexcept;
void move_to(const vector_2d& pt);
void move_to(const vector_2d& pt, error_code& ec) noexcept;
void rectangle(const experimental::io2d::rectangle& r);
void rectangle(const experimental::io2d::rectangle& r,
 error_code& ec) noexcept;
void rel_curve_to(const vector_2d& dpt0, const vector_2d& dpt1,
 const vector_2d& dpt2);
void rel_curve_to(const vector_2d& dpt0, const vector_2d& dpt1,
 const vector_2d& dpt2, error_code& ec) noexcept;
void rel_line_to(const vector_2d& dpt);
void rel_line_to(const vector_2d& dpt, error_code& ec) noexcept;
void rel_move_to(const vector_2d& dpt);
void rel_move_to(const vector_2d& dpt, error_code& ec) noexcept;
void transform_matrix(const matrix_2d& m);
void transform_matrix(const matrix_2d& m, error_code& ec) noexcept;
void origin(const vector_2d& pt);
void origin(const vector_2d& pt, error_code& ec) noexcept;
void clear() noexcept;

// 10.5.4, observers:
experimental::io2d::rectangle path_extents() const;
experimental::io2d::rectangle path_extents(error_code& ec) const noexcept;
bool has_current_point() const noexcept;
vector_2d current_point() const;
vector_2d current_point(error_code& ec) const noexcept;
matrix_2d transform_matrix() const noexcept;
vector_2d origin() const noexcept;
vector<path_data_item> data() const;
vector<path_data_item> data(error_code& ec) const noexcept;
path_data_item data_item(unsigned int index) const;

```

```

 path_data_item data_item(unsigned int index, error_code& ec) const noexcept;
 const vector<path_data_item>& data_ref() const noexcept;

private:
 vector<path_data_item> _Data; // exposition only
 bool _Has_current_point; // exposition only
 vector_2d _Current_point; // exposition only
 vector_2d _Last_move_to_point; // exposition only
 matrix_2d _Transform_matrix; // exposition only
 vector_2d _Origin; // exposition only
};
} } } }

```

### 10.5.2 path\_factory constructors and assignment operators

[pathfactory.cons]

```
path_factory();
```

- 1 *Effects:* Constructs an object of type `path_factory`.
- 2 *Postconditions:* `_Data.empty() == true`.
- 3 `_Has_current_point == false`.
- 4 `_Transform_matrix == matrix_2d::init_identity()`.
- 5 `_Origin == vector_2d`.

### 10.5.3 path\_factory modifiers

[pathfactory.modifiers]

```

void append(const path_factory& p);
void append(const path_factory& p, error_code& ec) noexcept;

```

- 1 *Effects:* Appends the result of calling `p.data_ref()` to `_Data` as if each `path_data_item` object was added to `*this` by calling the member function that adds a `path_data_item` of the same type to `*this`.
  - 2 *Throws:* As specified in Error reporting (3).
  - 3 *Remarks:* In the event of an error, the object shall not be modified.
  - 4 *Error conditions:* `errc::not_enough_memory` if the attempt to append the `p.data_ref()` failed. The object shall not be modified.
  - 5 `io2d_error::no_current_point` if the first item to be appended required a current point and `_Has_current_point == false`.
- ```

void append(const vector<path_data_item>& p);
void append(const vector<path_data_item>& p, error_code& ec) noexcept;

```
- 6 *Effects:* Appends the result of calling `p.data_ref()` to `_Data` as if each `path_data_item` object was added to `*this` by calling the member function that adds a `path_data_item` of the same type to `*this`.
 - 7 *Throws:* As specified in Error reporting (3).
 - 8 *Remarks:* In the event of an error, the object shall not be modified.
 - 9 *Error conditions:* `errc::not_enough_memory` if the attempt to append the `p.data_ref()` failed. The object shall not be modified.
 - 10 `io2d_error::no_current_point` if the first item to be appended required a current point and `_Has_current_point == false`.
 - 11 `io2d_error::invalid_path_data` if `p` contains one or more invalid path geometries.

```

void new_sub_path();
void new_sub_path(error_code& ec) noexcept;
12   Effects: _Data.emplace_back(path_data_item::new_sub_path()).
13   _Has_current_point = false.
14   Throws: As specified in Error reporting (3).
15   Remarks: In the event of an error, the object shall not be modified.
16   Error conditions: errc::not_enough_memory if the attempt to add the path_data_item failed.

void close_path();
void close_path(error_code& ec) noexcept;
17   Effects: If _Has_current_point == true:
(17.1)   — _Data.emplace_back(path_data_item::close_path()).
(17.2)   — _Current_point = _Last_move_to_point.
18   Throws: As specified in Error reporting (3).
19   Remarks: In the event of an error, the object shall not be modified.
20   Error conditions: errc::not_enough_memory if the attempt to add the path_data_item failed.

void arc(const vector_2d& center, double radius, double angle1,
         double angle2);
void arc(const vector_2d& center, double radius, double angle1,
         double angle2, error_code& ec) noexcept;
21   Effects: _Data.emplace_back(path_data_item::arc(center, radius, angle1, angle2)).
22   _Current_point == vector_2{ radius * cos(angle2), -(radius * -sin(angle2)) } + center.
23   If _Has_current_point == false:
(23.1)   — _Last_move_to_point == vector_2{ radius * cos(angle1), -(radius * -sin(angle1)) } + center.
(23.2)   — _Has_current_point == true.
24   Throws: As specified in Error reporting (3).
25   Remarks: In the event of an error, the object shall not be modified.
26   Error conditions: errc::not_enough_memory if the attempt to add the path_data_item failed.

void arc_negative(const vector_2d& center, double radius, double angle1,
                 double angle2);
void arc_negative(const vector_2d& center, double radius, double angle1,
                 double angle2, error_code& ec) noexcept;
27   Effects: _Data.emplace_back(path_data_item::arc_negative(center, radius, angle1, angle2)).
28   _Current_point = vector_2{ radius * cos(angle1), radius * -sin(angle1) } + center.
29   If _Has_current_point == false:
(29.1)   — _Last_move_to_point = vector_2{ radius * cos(angle2), radius * -sin(angle2) } + center.
(29.2)   — _Has_current_point = true.

```

30 *Throws:* As specified in Error reporting (3).
 31 *Remarks:* In the event of an error, the object shall not be modified.
 32 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

```
void curve_to(const vector_2d& pt0, const vector_2d& pt1,
              const vector_2d& pt2);
void curve_to(const vector_2d& pt0, const vector_2d& pt1,
              const vector_2d& pt2, error_code& ec) noexcept;
```

33 *Effects:* If `_Has_current_point == false`:

(33.1) — `_Data.reserve(_Data.size() + 2U)`.

(33.2) — `*this.move_to(pt0)`.

34 `_Data.emplace_back(path_data_item::curve_to(pt0, pt1, pt2))`.

35 *Throws:* As specified in Error reporting (3).

36 *Remarks:* In the event of an error, the object shall not be modified.

37 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

```
void line_to(const vector_2d& pt);
void line_to(const vector_2d& pt, error_code& ec) noexcept;
```

38 *Effects:* `_Data.emplace_back(path_data_item::line_to(pt))`.

39 If `_Has_current_point == false`:

(39.1) — `_Last_move_to_point = pt`.

(39.2) — `_Has_current_point = true`.

40 `_Current_point = pt`.

41 *Throws:* As specified in Error reporting (3).

42 *Remarks:* In the event of an error, the object shall not be modified.

43 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

```
void move_to(const vector_2d& pt);
void move_to(const vector_2d& pt, error_code& ec) noexcept;
```

44 *Effects:* `_Data.emplace_back(path_data_item::move_to(pt))`.

45 `_Has_current_point = true`.

46 `_Current_point = pt`.

47 *Throws:* As specified in Error reporting (3).

48 *Remarks:* In the event of an error, the object shall not be modified.

49 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

```
void rectangle(const experimental::io2d::rectangle& r);
void rectangle(const experimental::io2d::rectangle& r,
              error_code& ec) noexcept;
```

50 *Effects:*

1. `_Data.reserve(_Data.size() + 5U)`.

2. `*this.move_to({ r.x(), r.y() })`.

```

3. *this.rel_line_to({ r.width(), 0.0 }).
4. *this.rel_line_to({ 0.0, r.height() }).
5. *this.rel_line_to({ -r.width(), 0.0 }).
6. *this.close_path().

```

51 *Throws:* As specified in Error reporting (3).

52 *Remarks:* In the event of an error, the object shall not be modified.

53 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

```

void rel_curve_to(const vector_2d& dpt0, const vector_2d& dpt1,
    const vector_2d& dpt2);
void rel_curve_to(const vector_2d& dpt0, const vector_2d& dpt1,
    const vector_2d& dpt2, error_code& ec) noexcept;

```

54 *Effects:* `_Data.emplace_back(path_data_item::rel_curve_to(dpt0, dpt1, dpt2))`.

55 `_Current_point = dpt2 + _Current_point`.

56 *Throws:* As specified in Error reporting (3).

57 *Remarks:* In the event of an error, the object shall not be modified.

58 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

59 `io2d_error::no_current_point` if `_Has_current_point == false`.

```

void rel_line_to(const vector_2d& dpt);
void rel_line_to(const vector_2d& dpt, error_code& ec) noexcept;

```

60 *Effects:* `_Data.emplace_back(path_data_item::rel_line_to(pt))`.

61 `_Current_point = dpt + _Current_point`.

62 *Throws:* As specified in Error reporting (3).

63 *Remarks:* In the event of an error, the object shall not be modified.

64 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

65 `io2d_error::no_current_point` if `_Has_current_point == false`.

```

void rel_move_to(const vector_2d& dpt);
void rel_move_to(const vector_2d& dpt, error_code& ec) noexcept;

```

66 *Effects:* `_Data.emplace_back(path_data_item::rel_move_to(dpt))`.

67 `_Current_point = dpt + _Current_point`.

68 *Throws:* As specified in Error reporting (3).

69 *Remarks:* In the event of an error, the object shall not be modified.

70 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

71 `io2d_error::no_current_point` if `_Has_current_point == false`.

```

void transform_matrix(const matrix_2d& m);
void transform_matrix(const matrix_2d& m, error_code& ec) noexcept;

```

72 *Effects:* `_Data.emplace_back(path_data_item::change_matrix(m))`.

73 `_Transform_matrix = m`.

74 *Throws:* As specified in Error reporting (3).

75 *Remarks:* In the event of an error, the object shall not be modified.

76 *Error conditions:* `errc::not_enough_memory` if the attempt to add the `path_data_item` failed.

```

void origin(const vector_2d& pt);
void origin(const vector_2d& pt, error_code& ec) noexcept;
77   Effects: _Data.emplace_back(path_data_item::change_origin(pt)).
78   _Origin = pt.
79   Postconditions: _Origin == pt.
80   Throws: As specified in Error reporting (3).
81   Remarks: In the event of an error, the object shall not be modified.
82   Error conditions: errc::not_enough_memory if the attempt to add the path_data_item failed.

void clear() noexcept;
83   Postconditions: _Data.empty() == true.
84   _Has_current_point == false.
85   _Transform_matrix == matrix_2d::init_identity{ }.
86   _Origin == vector_2d{ }.

```

10.5.4 path_factory observers

[pathfactory.observers]

```

experimental::io2d::rectangle path_extents() const;
experimental::io2d::rectangle path_extents(error_code& ec) const noexcept;
1   Returns: A rectangle object which contains the extents of the path segments, including degenerate
    path segments, in _Data when it is processed as described in 10.1.2. [ Note: By using path segments,
    this description intentionally excludes points established by move_to and rel_move_to operations from
    the extents value except where those points are subsequently used in defining a path segment. — end
    note ]
2   Throws: As specified in Error reporting (3).
3   Error conditions: io2d_error::invalid_matrix if _Data includes a change_matrix operation which
    establishes a non-invertible matrix_2d as the transformation matrix and that matrix must subsequently
    be inverted in order to process the path geometries.

bool has_current_point() const noexcept;
4   Returns: _Has_current_point.

vector_2d current_point() const;
vector_2d current_point(error_code& ec) const noexcept;
5   Returns: _Current_point.
6   Throws: As specified in Error reporting (3).
7   Error conditions: io2d_error::no_current_point if _Has_current_point == false.

matrix_2d transform_matrix() const noexcept;
8   Returns: _Transform_matrix.

vector_2d origin() const noexcept;
9   Returns: _Origin.

vector<path_data_item> data() const;
vector<path_data_item> data(error_code& ec) const noexcept;

```

- 10 *Returns:* A copy of `_Data`.
- 11 *Throws:* As specified in Error reporting (3).
- 12 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.

```
path_data_item data_item(unsigned int index) const;  
path_data_item data_item(unsigned int index, error_code& ec) const noexcept;
```

- 13 *Returns:* `_Data.at(index)`.
- 14 *Throws:* As specified in Error reporting (3).
- 15 *Error conditions:* `io2d_error::invalid_index` if `_Data.size() <= index`.

```
const vector<path_data_item>& data_ref() const noexcept;
```

- 16 *Returns:* `_Data`.

11 Fonts

[fonts]

- ¹ Fonts describe how text should be rendered and composed. This Technical Specification leaves much of how the rendering and composing happens up to the implementors.
- ² Fonts exist to describe how text is rendered. The only requirements that exist in this Technical Specification are:
 - (2.1) — The text to be rendered is in the UTF-8 character encoding.
 - (2.2) — The default font face of the implementation shall correctly render all of the characters of the Unicode Basic Multilingual Plane C0 Controls and Basic Latin.
- ³ This section is forthcoming in a future revision.

11.1 Enum class `font_slant`

[fontslant]

11.1.1 `font_slant` Summary

[fontslant.summary]

- ¹ The `font_slant` enum class specifies the slant requested for rendering text.
- ² These values have different meanings for different scripts. For some scripts they may have no meaning at all. Further, not all typefaces will support every value.
- ³ As such, these values are requests which implementations should honor if possible.
- ⁴ See Table 4 for the meaning of each `font_slant` enumerator.

11.1.2 `font_slant` Synopsis

[fontslant.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class font_slant {
        normal,
        italic,
        oblique
    };
} } } } // namespaces std::experimental::io2d::v1
```

11.1.3 `font_slant` Enumerators

[fontslant.enumerators]

Table 4 — `font_slant` enumerator meanings

Enumerator	Meaning
<code>normal</code>	The text shall be rendered in whatever is a normal type for the font. If a font has both an italic type and an oblique type but does not have another type, then the italic type shall be the normal type for the font unless the font includes data that specifies otherwise. [<i>Note</i> : If a font only has an italic type or only an oblique type then that is the normal type for the font. — <i>end note</i>]

Table 4 — `font_slant` enumerator meanings (continued)

Enumerator	Meaning
<code>italic</code>	The text should be rendered in whatever is an italic type for the font. If a font does not have an italic type but does have an oblique type, the oblique type shall be used if <code>italic</code> is requested. If a font has neither an italic type nor an oblique type, the normal type shall be used.
<code>oblique</code>	The text should be rendered in whatever is an oblique type for the font. If a font does not have an oblique type but does have an italic type, the italic type shall be used if <code>oblique</code> is requested. If a font has neither an italic type nor an oblique type, the normal type shall be used.

11.2 Enum class `font_weight` [font.weight]

11.2.1 `font_weight` Synopsis [font.weight.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class font_weight {
        normal,
        bold
    };
} } } }
```

11.2.2 `font_weight` Summary [font.weight.summary]

¹ The `font_weight` enum class specifies the font weight for rendering text.

² See Table 5 for the meaning of each enumerator.

11.2.3 `font_weight` Enumerators [font.weight.enumerators]

Table 5 — `font_weight` enumerator meanings

Enumerator	Meaning
<code>normal</code>	The text shall be rendered in whatever is a normal weight for the script.
<code>bold</code>	The text shall be rendered in whatever is a bold weight for the script.

11.3 Enum class `subpixel_order` [subpixel.order]

11.3.1 `subpixel_order` Summary [subpixel.order.summary]

¹ The `subpixel_order` enum class is used to request a specific order of color channels for each pixel of an output device. When a `surface` object's `font_options` object has its `antialias` value set to `antialias::subpixel` and its `subpixel_order` value set to one of these values, an implementation should use the specified `subpixel_order` to render text. See Table 6 for the meaning of each `subpixel_order` enumerator.

11.3.2 `subpixel_order` Synopsis [subpixel.order.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class subpixel_order {
        default_subpixel_order,
    };
} } } }
```

```

    horizontal_rgb,
    horizontal_bgr,
    vertical_rgb,
    vertical_bgr
};
} } } } // namespaces std::experimental::io2d::v1

```

11.3.3 subpixel_order Enumerators

[subpixel.order.enumerators]

Table 6 — subpixel_order enumerator meanings

Enumerator	Meaning
default_subpixel_order	The implementation should use the target surface object's default subpixel order.
horizontal_rgb	The color channels should be arranged horizontally starting with red on the left, followed by green, then blue.
horizontal_bgr	The color channels should be arranged horizontally starting with blue on the left, followed by green, then red.
vertical_rgb	The color channels should be arranged vertically starting with red on the top, followed by green, then blue.
vertical_bgr	The color channels should be arranged vertically starting with blue on the top, followed by green, then red.

11.4 Class font_options

[fontoptions]

11.4.1 font_options synopsis

[fontoptions.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class font_options {
    public:
        // 11.4.3, construct/copy/destroy:
        font_options() noexcept;
        font_options(const font_options& other) noexcept;
        font_options& operator=(const font_options& other) noexcept;
        font_options(font_options&& other) noexcept;
        font_options& operator=(font_options&& other) noexcept;
        font_options(std::experimental::io2d::antialias a,
                     std::experimental::io2d::subpixel_order so) noexcept;

        // 11.4.4, modifiers:
        void antialias(std::experimental::io2d::antialias value) noexcept;
        void subpixel_order(std::experimental::io2d::subpixel_order value) noexcept;

        // 11.4.5, observers:
        std::experimental::io2d::antialias antialias() const noexcept;
        std::experimental::io2d::subpixel_order subpixel_order() const noexcept;

    private:
        std::experimental::io2d::antialias _Antialias; // exposition only
        std::experimental::io2d::subpixel_order _Subpixel_order; // exposition only
    };
} } } }

```

11.4.2 font_options Description [fontoptions.intro]

- 1 The `font_options` class describes an object that holds values which specify certain aspects of how text should be rendered.

11.4.3 font_options constructors and assignment operators [fontoptions.cons]

- ```
font_options() noexcept;
```
- 1 *Effects:* Constructs an object of type `font_options`.
- 2 *Postconditions:* `_Antialias == std::experimental::io2d::antialias::default_antialias.`  
`_Subpixel_order == std::experimental::io2d::subpixel_order::default_subpixel_order.`
- ```
font_options(std::experimental::io2d::antialias a,
             std::experimental::io2d::subpixel_order so) noexcept;
```
- 3 *Effects:* Constructs an object of type `font_options`.
- 4 *Postconditions:* `_Antialias == a.`
`_Subpixel_order == so.`

11.4.4 font_options modifiers [fontoptions.modifiers]

- ```
void antialias(std::experimental::io2d::antialias value) noexcept;
```
- 1 *Postconditions:* `_Antialias == value.`
- ```
void subpixel_order(std::experimental::io2d::subpixel_order value) noexcept;
```
- 2 *Postconditions:* `_Subpixel_order == value.`

11.4.5 font_options observers [fontoptions.observers]

- ```
std::experimental::io2d::antialias antialias() const noexcept;
```
- 1 *Returns:* `_Antialias.`
- ```
std::experimental::io2d::subpixel_order subpixel_order() const noexcept;
```
- 2 *Returns:* `_Subpixel_order.`

11.5 Class font_face [fontface]

11.5.1 font_face synopsis [fontface.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class font_face {
    public:
        // See 2.3
        typedef implementation-defined native_handle_type;
        native_handle_type native_handle() const noexcept;

        // 11.5.3, construct/copy/destroy:
        font_face() = delete;
        font_face(const font_face&) noexcept;
        font_face& operator=(const font_face&) noexcept;
        font_face(font_face&& other) noexcept;
        font_face& operator=(font_face&& other) noexcept;
        virtual ~font_face();
    };
} } } }
```

11.5.2 font_face Description [fontface.intro]

- ¹ The `font_face` class is an opaque resource wrapper. It represents a font that is used to render text. It shall not be directly instantiable.

11.5.3 font_face constructors and assignment operators [fontface.cons]

```
virtual ~font_face();
```

- ¹ *Effects:* Destroys an object of class `font_face`.

11.6 Class simple_font_face [simplefontface]

11.6.1 simple_font_face synopsis [simplefontface.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class simple_font_face : public font_face {
    public:
        // See 2.3
        typedef implementation-defined native_handle_type; // exposition only
        native_handle_type native_handle() const noexcept; // exposition only

        // 11.6.3, construct/copy/destroy:
        simple_font_face() = delete;
        simple_font_face(const simple_font_face&) noexcept;
        simple_font_face& operator=(const simple_font_face&) noexcept;
        simple_font_face(simple_font_face&& other) noexcept;
        simple_font_face& operator=(simple_font_face&& other) noexcept;
        simple_font_face(const string& typeface,
            std::experimental::io2d::font_slant fs,
            std::experimental::io2d::font_weight fw);
        simple_font_face(const string& typeface,
            std::experimental::io2d::font_slant fs,
            std::experimental::io2d::font_weight fw, error_code& ec) noexcept;

        // 11.6.4, observers:
        string typeface() const;
        void typeface(string& str, error_code& ec) const noexcept;
        ::std::experimental::io2d::font_slant font_slant() const noexcept;
        ::std::experimental::io2d::font_weight font_weight() const noexcept;

    private:
        string _Typeface; // exposition only
        std::experimental::io2d::font_slant _Font_slant; // exposition only
        std::experimental::io2d::font_weight _Font_weight; // exposition only
    };
} } } }
```

11.6.2 simple_font_face Description [simplefontface.intro]

- ¹ The `simple_font_face` class represents a font that is used to render text. It only guarantees rudimentary text rendering capabilities.
- ² It shall correctly render text written horizontally in left-to-right scripts where there is a one-to-one mapping between characters and glyphs.
- ³ It may correctly render:
- (3.1) — right-to-left scripts;

- (3.2) — bi-directional text;
- (3.3) — vertically-oriented text;
- (3.4) — combining character sequences.

4 It may correctly handle some or all aspects of complex text rendering, e.g.:

- (4.1) — bi-directional text;
- (4.2) — context sensitive shaping;
- (4.3) — ligatures;
- (4.4) — cursive scripts;
- (4.5) — text where the proper *display order* (the order text is rendered) differs from the *logical order* (the order text is typed on a keyboard).

11.6.3 `simple_font_face` constructors and assignment operators [`simplefontface.cons`]

```
simple_font_face(const string& typeface,
    std::experimental::io2d::font_slant fs,
    std::experimental::io2d::font_weight fw);
simple_font_face(const string& typeface,
    std::experimental::io2d::font_slant fs,
    std::experimental::io2d::font_weight fw, error_code& ec) noexcept;
```

1 *Effects:* Constructs an object of class `simple_font_face`. A font that can be used to render text to a `surface` object is associated with the object.

2 Implementations should perform all possible actions that are required to use the font to render text rather than delay such actions until the font is actually used.

3 *Postconditions:* `_Typeface == typeface` if the implementation located a usable typeface by that name, otherwise `_Typeface` will be set to the name of the implementation-defined nearest match typeface which the implementation selected instead. [*Note:* Implementations have been given control over deciding what the "nearest match" is because typeface names are arbitrary and any individual with the proper tools can make a new typeface.

Ideally an implementation which could not find an exact match would search for nearest matches in the following order. First it would run a search for typefaces with names that closely matched the requested typeface string based on common misspellings and typos. If no match was found it would then search some form of lookup table which stored distinguishing characteristics of a large number of typefaces (e.g. those which are provided by operating systems and applications which have an install base in excess of one million copies). If it found a listing in the table for the requested typeface or, barring that, a typeface which was a near match due to misspellings and typos, it would then use the characteristics provided by the table to make the nearest possible match. Optimally the table would include first preference matches where one font is known to have been developed to match another font as closely as legally permissible. The implementation would then use the results of the lookup table to find the closest available matching typeface. Lastly, only if the previous methods had failed would the implementation provide a font of its choosing.

Nonetheless, implementations may, with equal validity, return as being the nearest match the same typeface regardless of any information that could be gleaned from the unmatched request and without any consideration of spelling mistakes or typos. — *end note*]

4 `_Font_slant` shall equal the value determined according to the requirements set forth in 11.1 after the value of `_Typeface` has been determined. If that `font_slant` value is different than `fs`, `_Font_slant` shall be set to the actual value used. [*Example: If `font_slant::italic` is requested but an italic type does not exist for the chosen typeface, then if an oblique type exists, it is chosen and `_Font_slant == font_slant::oblique`, otherwise the normal type is chosen and `_Font_slant == font_slant::normal`. — end example*]

5 `_Font_weight == fw`.

6 *Throws:* As specified in Error reporting (3).

7 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.

11.6.4 `simple_font_face` observers [simplefontface.observers]

```
string typeface() const;
1     Returns: _Typeface.
2     Throws: As specified in Error reporting (3).
3     Error conditions: errc::not_enough_memory if there was a failure to allocate memory.

void typeface(string& str, error_code& ec) const noexcept;
4     Effects: str = _Typeface.
5     Throws: As specified in Error reporting (3).
6     Error conditions: errc::not_enough_memory if there was a failure to allocate memory.

std::experimental::io2d::font_slant font_slant() const noexcept;
7     Returns: _Font_slant.

std::experimental::io2d::font_weight font_weight() const noexcept;
8     Returns: _Font_weight.
```

11.7 Class `font_extents` [fontextents]

11.7.1 `font_extents` synopsis [fontextents.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
  class font_extents {
  public:
    // 11.7.3, construct/copy/move/destroy:
    font_extents() noexcept;
    font_extents(const font_extents& other) noexcept;
    font_extents& operator=(const font_extents& other) noexcept;
    font_extents(font_extents&& other) noexcept;
    font_extents& operator=(font_extents&& other) noexcept;
    font_extents(double ascent, double descent, double height) noexcept;

    // 11.7.4, modifiers:
    void ascent(double value) noexcept;
    void descent(double value) noexcept;
    void height(double value) noexcept;

    // 11.7.5, observers:
    double ascent() const noexcept;
    double descent() const noexcept;
```

```

    double height() const noexcept;

private:
    double _Asc;    // exposition only
    double _Desc;   // exposition only
    double _Height; // exposition only
};
} } } }

```

11.7.2 font_extents Description

[fontextents.intro]

- 1 The class `font_extents` describes metric information for a font.
- 2 It is used by a `surface` object to report certain metrics of its currently selected font in the `surface` object's untransformed coordinate space units.
- 3 These metrics cover all glyphs in a font and thus may be noticeably larger than the values obtained by getting the `text_extents` for a particular string.
- 4 [*Note:* This object's observable values can be manipulated by library users for their convenience. But since the `font_extents` object returned by `surface::font_extents()` is not a reference or a pointer, the changes do not reflect back to the surface or its current font. — *end note*]

11.7.3 font_extents constructors and assignment operators

[fontextents.cons]

```

font_extents() noexcept;

1   Effects: Constructs an object of type font_extents.
2   Postconditions: _Asc == 0.0.
   _Desc == 0.0.
   _Height == 0.0.

font_extents(double ascent, double descent, double height) noexcept;

3   Effects: Constructs an object of type font_extents.
4   Postconditions: _Asc == ascent.
   _Desc == descent.
   _Height == height.

```

11.7.4 font_extents modifiers

[fontextents.modifiers]

```

void ascent(double value) noexcept;

1   Postconditions: _Asc == value.

void descent(double value) noexcept;

2   Postconditions: _Desc == value.

void height(double value) noexcept;

3   Postconditions: _Height == value.

```


11.7.5 font_extents observers**[fontextents.observers]**

```
double ascent() const noexcept;
```

1 *Returns:* `_Asc`.

2 *Remarks:* This value is the distance in untransformed coordinate space units from the top of the font's bounding box to the font's baseline.

3 Some glyphs may extend slightly above the top of the font's bounding box due to hinting or for aesthetic reasons.

```
double descent() const noexcept;
```

4 *Returns:* `_Desc`.

5 *Remarks:* This value is the distance in untransformed coordinate space units from the bottom of the font's bounding box to the font's baseline.

6 Some glyphs may extend slightly below the bottom of the font's bounding box due to hinting or for aesthetic reasons.

7 [*Note:* Some font rendering technologies express this value as a negative value. Because it is defined here as a distance from the baseline, the value should typically be positive or zero. It would only be negative if the font's baseline was set below the bottom of its bounding box, which, while highly unlikely, is not impossible. — *end note*]

```
double height() const noexcept;
```

8 *Returns:* `_Height`.

9 *Remarks:* This value is the font designer's suggested distance, in untransformed coordinate space units, from the baseline of one line of text to the baseline of a consecutive line of text.

10 This value is may be greater than the sum of `ascent()` and `descent()`. This occurs when a font includes a value known as a line gap or as external leading, which is additional whitespace added for aesthetic reasons.

11 Fonts whose `height()` is equal to their `ascent() + descent()` likely include line gap in their ascent or descent rather than specifying it separately.

11.8 Class text_extents**[textextents]****11.8.1 text_extents synopsis****[textextents.synopsis]**

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
  class text_extents {
  public:
    // 11.8.3, construct/copy/move/destroy:
    text_extents() noexcept;
    text_extents(const text_extents& other) noexcept;
    text_extents& operator=(const text_extents& other) noexcept;
    text_extents(font_extents&& other) noexcept;
    text_extents& operator=(font_extents&& other) noexcept;
    text_extents(double xBearing, double yBearing, double width,
      double height, double xAdvance, double yAdvance) noexcept;

    // 11.8.4, modifiers:
    void x_bearing(double value) noexcept;
    void y_bearing(double value) noexcept;
    void width(double value) noexcept;
```

```

void height(double value) noexcept;
void x_advance(double value) noexcept;
void y_advance(double value) noexcept;

// 11.8.5, observers:
double x_bearing() const noexcept;
double y_bearing() const noexcept;
double width() const noexcept;
double height() const noexcept;
double x_advance() const noexcept;
double y_advance() const noexcept;

private:
double _X_bear; // exposition only
double _Y_bear; // exposition only
double _Width; // exposition only
double _Height; // exposition only
double _X_adv; // exposition only
double _Y_adv; // exposition only
};
} } } }

```

11.8.2 `text_extents` Description

[textextents.intro]

- 1 The class `text_extents` describes extents for a string.
- 2 It is used by a `surface` object to report the extents of a string in the `surface` object's untransformed coordinate space units if the string were rendered with the currently selected font.
- 3 [*Note:* This object's observable values can be manipulated by library users for their convenience. But since the `text_extents` object returned by `surface::text_extents()` is not a reference or a pointer, the changes do not reflect back to the surface or its current font. — *end note*]

11.8.3 `text_extents` constructors and assignment operators

[textextents.cons]

- ```

text_extents() noexcept;

```
- 1 *Effects:* Constructs an object of type `text_extents`.
  - 2 *Postconditions:* `_X_bear == 0.0`.  
`_Y_bear == 0.0`.  
`_Width == 0.0`.  
`_Height == 0.0`.  
`_X_adv == 0.0`.  
`_Y_adv == 0.0`.
- ```

text_extents(double xBearing, double yBearing, double width,
             double height, double xAdvance, double yAdvance) noexcept;

```
- 3 *Effects:* Constructs an object of type `text_extents`.
 - 4 *Postconditions:* `_X_bear == xBearing`.
`_Y_bear == yBearing`.
`_Width == width`.
`_Height == height`.

```

_X_adv == xAdvance.
_Y_adv == yAdvance.

```

11.8.4 text_extents modifiers

[textextents.modifiers]

```

void x_bearing(double value) noexcept;
1   Postconditions: _X_bear == value.

void y_bearing(double value) noexcept;
2   Postconditions: _Y_bear == value.

void width(double value) noexcept;
3   Postconditions: _Width == value.

void height(double value) noexcept;
4   Postconditions: _Height == value.

void x_advance(double value) noexcept;
5   Postconditions: _X_adv == value.

void y_advance(double value) noexcept;
6   Postconditions: _Y_adv == value.

```

11.8.5 text_extents observers

[textextents.observers]

```

double x_bearing() const noexcept;
1   Returns: _X_bear.
2   Remarks: This value is the x axis offset of the leftmost visible part of the text as rendered from the x
    coordinate of the specified position at which to draw the text.
3   Leading and trailing spaces can affect this value due to the fact that spaces change the position of
    other text and thus can change the position of the first visible text that is rendered.
4   [Note: Because this value is an offset from the specified position and is given in untransformed units,
    it remains the same regardless of the position at which the text will be drawn.
5   This value will typically be negative, zero, or slightly positive depending on the font used and the text
    being rendered (e.g. scripts that are written right-to-left will normally have a negative x_bearing()
    value). — end note]

double y_bearing() const noexcept;
6   Returns: _Y_bear.
7   Remarks: This value is the y axis offset of the topmost visible part of the text as rendered from the y
    coordinate of the specified position at which to draw the text.
8   [Note: This value may range from negative to positive depending on the font origin chosen by the font
    designer. Usually this value is negative. — end note]

double width() const noexcept;

```

Returns: `_Width`.

Remarks: This value is the width of the text as rendered from its leftmost visible part to its rightmost visible part.

This value may include a de minimus amount of whitespace, e.g. 1 to 2 pixels when pixels are the coordinate space unit. This allowance is meant to cover discrepancies between expected rendering results and actual results which can arise due to techniques such as font hinting, antialiasing, and subpixel rendering.

`double height() const noexcept;`

Returns: `_Height`.

Remarks: This value is the height of the text as rendered from its topmost visible part to its bottommost visible part.

This value may include a de minimus amount of whitespace, e.g. 1 to 2 pixels when pixels are the coordinate space unit. This allowance is meant to cover discrepancies between expected rendering results and actual results which can arise due to techniques such as font hinting, antialiasing, and subpixel rendering.

`double x_advance() const noexcept;`

Returns: `_X_adv`.

Remarks: This value is amount to add to the x coordinate of the original specified position in order to properly draw text that will immediately follow this text on the same line.

In vertically oriented text, this value will typically be zero.

`double y_advance() const noexcept;`

Returns: `_Y_adv`.

Remarks: This value is amount to add to the y coordinate of the original specified position in order to properly draw text that will immediately follow this text on the same line.

In horizontally oriented text, this value will typically be zero.

12 Brushes

[brushes]

- 1 Brushes serve as sources of visual data for composing operations.
- 2 A brush has its own mutable coordinate space (by default the standard coordinate space). It also possesses a mutable **extend** value and a mutable **filter** value.
- 3 A brush is created using one of four factory classes:
 - (3.1) — **solid_color_brush_factory**,
 - (3.2) — **linear_brush_factory**,
 - (3.3) — **radial_brush_factory**, and
 - (3.4) — **surface_brush_factory**.

- 4 Excluding brushes created using a **solid_color_brush_factory** object, brushes produce very different composing operation results depending on the values of their mutable state.
- 5 Brushes created using either a **linear_brush_factory** object or a **radial_brush_factory** object are considered to be gradient brushes. They share similarities with each other that are not shared by brushes created using the other brush factories. This is discussed in more detail below (12.1).

12.1 Gradient brushes

[gradients]

12.1.1 Color stops

[gradients.colorstops]

- 1 A gradient has as part of its observable state a collection of color stops.
- 2 The collection of color stops has an implementation-defined maximum size.
- 3 Associated with the collection of color stops is a type called the *size_type*, which is an implementation-defined unsigned integer type that is large enough that it can serve as an index to address each color stop in any collection of color stops as if the collection of color stops was accessed as an array of color stops.
- 4 The *size* of a collection of color stops is the number of color stops it currently contains; its type is *size_type*.
- 5 When a color stop is added to a collection of color stops it is assigned an *index value* that is equal to the size of the collection of color stops before the color stop is added.
- 6 A *valid index value* for a collection of color stops is an unsigned integer value in the range $(0, size]$.
- 7 Color stops in a collection of color stops can be replaced.
- 8 A color stop which replaces another color stop in a collection of color stops is assigned the index value of the color stop it replaced. [*Example*: If 3 color stops are added to a collection of color stops and then the color stop at index 1 is replaced, the next color stop which is added will be assigned the index value 3. — *end example*]
- 9 Color stops in a collection of color stops can be removed.
- 10 When a color stop with the valid index value *i* is removed from a collection of color stops, the index value of every color stop in the collection of color stops which had an index value greater than *i* has its index value decremented by 1. [*Note*: The size of the collection of color stops is reduced by 1 as the result of the removal of one of its color stops. — *end note*]

12.1.2 Linear gradients

[gradients.linear]

- 1 A linear gradient is a type of gradient.
- 2 In addition to the observable state of a gradient, a linear gradient also has a *begin point* and an *end point* as parts of its observable state, both of which are objects of type `vector_2d`.
- 3 A linear gradient for which the distance between its begin point and its end point is not greater than `numeric_limits<double>::epsilon()` is a *degenerate linear gradient*.
- 4 All attempts to sample from a `brush` object created using a degenerate linear gradient shall return the color `rgba_color::transparent_black()`. The remainder of 12.1.2 is inapplicable to degenerate linear gradients.
- 5 The begin point and end point of a linear gradient define a line segment, with a color stop offset value of 0.0 corresponding to the begin point and a color stop offset value of 1.0 corresponding to the end point.
- 6 Color stop offset values in the range (0,1) linearly correspond to points on the line segment.
- 7 [Example: Given a linear gradient with a begin point of `vector_2d(0.0, 0.0)` and an end point of `vector_2d(10.0, 5.0)`, a color stop offset value of 0.6 would correspond to the point `vector_2d(6.0, 3.0)`. — end example]
- 8 To determine the offset value of a point p for a linear gradient, perform the following steps:
 - a) Create a line at the begin point of the linear gradient, the *begin line*, and another line at the end point of the linear gradient, the *end line*, with each line being perpendicular to the *gradient line segment*, which is the line segment delineated by the begin point and the end point.
 - b) Using the begin line, p , and the end line, create a line, the *p line*, which is parallel to the gradient line segment.
 - c) Defining dp as the distance between p and the point where the p line intersects the begin line and dt as the distance between the point where the p line intersects the begin line and the point where the p line intersects the end line, the offset value of p is $dp \div dt$.
 - d) The offset value shall be negative if
 - (8.1) — p is not on the line segment delineated by the point where the p line intersects the begin line and the point where the p line intersects the end line; and,
 - (8.2) — the distance between p and the point where the p line intersects the begin line is less than the distance between p and the point where the p line intersects the end line.

12.1.3 Radial gradients

[gradients.radial]

- 1 A radial gradient is a type of gradient.
- 2 In addition to the observable state of a gradient, a radial gradient also has a *start circle* and an *end circle* as part of its observable state, each of which is defined by a `vector_2d` object that denotes its center and a `double` value that denotes its radius.
- 3 A radial gradient is a *degenerate radial gradient* if:
 - (3.1) — its start circle has a negative radius; or,
 - (3.2) — its end circle has a negative radius; or,
 - (3.3) — the distance between the center point of its start circle and the center point of its end circle is not greater than `numeric_limits<double>::epsilon()` and the difference between the radius of its start circle and the radius of its end circle is not greater than `numeric_limits<double>::epsilon()`; or,

- (3.4) — its start circle has a radius of 0.0 and its end circle has a radius of 0.0.
- 4 All attempts to sample from a **brush** object created using a degenerate radial gradient shall return the color `rgba_color::transparent_black()`. The remainder of 12.1.3 is inapplicable to degenerate radial gradients.
- 5 A color stop offset of 0.0 corresponds to all points along the diameter of the start circle or to its center point if it has a radius value of 0.0.
- 6 A color stop offset of 1.0 corresponds to all points along the diameter of the end circle or to its center point if it has a radius value of 0.0.
- 7 A radial gradient shall be rendered as a continuous series of interpolated circles defined by the following equations:

- a) $x(o) = x_{start} + o \times (x_{end} - x_{start})$
- b) $y(o) = y_{start} + o \times (y_{end} - y_{start})$
- c) $radius(o) = radius_{start} + o \times (radius_{end} - radius_{start})$

where o is a color stop offset value.

- 8 The range of potential values for o shall be determined by the **extend** value of the **brush** object created using the radial gradient:
- (8.1) — For **extend::none**, the range of potential values for o is $[0, 1]$.
- (8.2) — For all other **extend** values, the range of potential values for o is $[numeric_limits<double>::lowest(), numeric_limits<double>::max()]$.
- 9 The interpolated circles shall be rendered starting from the smallest potential value of o .
- 10 An interpolated circle shall not be rendered if its value for o results in $radius(o)$ evaluating to a negative value.

12.1.4 Sampling from gradients

[gradients.sampling]

- 1 For any offset value o , its color value shall be determined according to the following rules:
- a) If there are less than two color stops or if all color stops have the same offset value, then the color value of every offset value shall be `rgba_color::transparent_black()` and the remainder of 12.1.4 is inapplicable.
- b) If exactly one color stop has an offset value equal to o , o 's color value shall be the color value of that color stop and the remainder of 12.1.4 is inapplicable.
- c) If two or more color stops have an offset value equal to o , o 's color value shall be the color value of the color stop which has the lowest index value among the set of color stops that have an offset value equal to o and the remainder of 12.1.4 is inapplicable.
- d) When no color stop has the offset value of 0.0, then, defining n to be the offset value that is nearest to 0.0 among the offset values in the set of all color stops, if o is in the offset range $[0, n)$, o 's color value shall be `rgba_color::transparent_black()` and the remainder of 12.1.4 is inapplicable. [*Note:* Since the range described does not include n , it does not matter how many color stops have n as their offset value for purposes of this rule. — end note]

- e) When no color stop has the offset value of 1.0, then, defining n to be the offset value that is nearest to 1.0 among the offset values in the set of all color stops, if o is in the offset range $(n, 1]$, o 's color value shall be `rgba_color::transparent_black()` and the remainder of 12.1.4 is inapplicable. [*Note*: Since the range described does not include n , it does not matter how many color stops have n as their offset value for purposes of this rule. — *end note*]
- f) Each color stop has, at most, two adjacent color stops: one to its left and one to its right.
- g) Adjacency of color stops is initially determined by offset values. If two or more color stops have the same offset value then index values are used to determine adjacency as described below.
- h) For each color stop a , the *set of color stops to its left* are those color stops which have an offset value which is closer to 0.0 than a 's offset value. [*Note*: This includes any color stops with an offset value of 0.0 provided that a 's offset value is not 0.0. — *end note*]
- i) For each color stop b , the *set of color stops to its right* are those color stops which have an offset value which is closer to 1.0 than b 's offset value. [*Note*: This includes any color stops with an offset value of 1.0 provided that b 's offset value is not 1.0. — *end note*]
- j) A color stop which has an offset value of 0.0 does not have an adjacent color stop to its left.
- k) A color stop which has an offset value of 1.0 does not have an adjacent color stop to its right.
- l) If a color stop a 's set of color stops to its left consists of exactly one color stop, that color stop is the color stop that is adjacent to a on its left.
- m) If a color stop b 's set of color stops to its right consists of exactly one color stop, that color stop is the color stop that is adjacent to b on its right.
- n) If two or more color stops have the same offset value then the color stop with the lowest index value is the only color stop from that set of color stops which can have a color stop that is adjacent to it on its left and the color stop with the highest index value is the only color stop from that set of color stops which can have a color stop that is adjacent to it on its right. This rule takes precedence over all of the remaining rules.
- o) If a color stop can have an adjacent color stop to its left, then the color stop which is adjacent to it to its left is the color stop from the set of color stops to its left which has an offset value which is closest to its offset value. If two or more color stops meet that criteria, then the color stop which is adjacent to it to its left is the color stop which has the highest index value from the set of color stops to its left which are tied for being closest to its offset value.
- p) If a color stop can have an adjacent color stop to its right, then the color stop which is adjacent to it to its right is the color stop from the set of color stops to its right which has an offset value which is closest to its offset value. If two or more color stops meet that criteria, then the color stop which is adjacent to it to its right is the color stop which has the lowest index value from the set of color stops to its right which are tied for being closest to its offset value.
- q) Where the value of o is in the range $[0, 1]$, its color value shall be determined by interpolating between the color stop, r , which is the color stop whose offset value is closest to o without being less than o and which can have an adjacent color stop to its left, and the color stop that is adjacent to r on r 's left. The acceptable forms of interpolating between color values is set forth later in this section.
- r) Where the value of o is outside the range $[0, 1]$, its color value depends on the `extend` value of the brush which is created using the gradient:

- (1.1) — If the **extend** value is **extend::none**, the color value of *o* shall be **rgba_color::transparent_black()**.
 - (1.2) — If the **extend** value is **extend::pad**, if *o* is negative then the color value of *o* shall be the same as if the value of *o* was 0.0, otherwise the color value of *o* shall be the same as if the value of *o* was 1.0.
 - (1.3) — If the **extend** value is **extend::repeat**, then 1.0 shall be added to or subtracted from *o* until *o* is in the range [0, 1], at which point its color value is the color value for the modified value of *o* as determined by these rules. [*Example:* Given *o* == 2.1, after application of this rule *o* == 0.1 and the color value of *o* shall be the same value as if the initial value of *o* was 0.1.
Given *o* == -0.3, after application of this rule *o* == 0.7 and the color value of *o* shall be the same as if the initial value of *o* was 0.7. — end example]
 - (1.4) — If the **extend** value is **extend::reflect**, *o* shall be set to the absolute value of *o*, then 2.0 shall be subtracted from *o* until *o* is in the range [0, 2], then if *o* is in the range (1, 2] then *o* shall be set to 1.0 - (*o* - 1.0), at which point its color value is the color value for the modified value of *o* as determined by these rules. [*Example:* Given *o* == 2.8, after application of this rule *o* == 0.8 and the color value of *o* shall be the same value as if the initial value of *o* was 0.8.
Given *o* == 3.6, after application of this rule *o* == 0.4 and the color value of *o* shall be the same value as if the initial value of *o* was 0.4.
Given *o* == -0.3, after application of this rule *o* == 0.3 and the color value of *o* shall be the same as if the initial value of *o* was 0.3.
Given *o* == -5.8, after application of this rule *o* == 0.2 and the color value of *o* shall be the same as if the initial value of *o* was 0.2. — end example]
- 2 It is unspecified whether the interpolation between the color values of two adjacent color stops is performed linearly on each color channel, is performed within an RGB color space (with or without gamma correction), or is performed by a linear color interpolation algorithm implemented in hardware (typically in a graphics processing unit).
 - 3 Implementations shall interpolate between alpha channel values of adjacent color stops linearly except as provided in the following paragraph.
 - 4 A conforming implementation may use the alpha channel interpolation results from a linear color interpolation algorithm implemented in hardware even if those results differ from the results required by the previous paragraph.

12.2 Enum class **extend**

[**extend**]

12.2.1 **extend** Summary

[**extend.summary**]

- 1 The **extend** enum class describes how a point's visual data shall be determined if it is outside the bounds of the Source Brush (13.11.5) when sampling.
- 2 Depending on the Source Brush's **filter** value, the visual data of several points may be required to determine the appropriate visual data value for the point that is being sampled. In this case, each point shall be sampled according to the Source Brush's **extend** value with two exceptions:
 - a) If the point to be sampled is within the bounds of the Source Brush and the Source Brush's **extend** value is **extend::none**, then if the Source Brush's **filter** value requires that one or more points which are outside of the bounds of the Source Brush shall be sampled, each of those points shall be sampled as if the Source Brush's **extend** value is **extend::pad** rather than **extend::none**.
 - b) If the point to be sampled is within the bounds of the Source Brush and the Source Brush's **extend** value is **extend::none**, ce Brush and the Source Brush's **extend** value is **extend::none**, then if the

Source Brush's **filter** value requires that one or more points which are inside of the bounds of the Source Brush shall be sampled, each of those points shall be sampled such that the visual data that is returned shall be the equivalent of `rgba_color::transparent_black()`.

- ³ If a point to be sampled does not have a defined visual data element and the search for the nearest point with defined visual data produces two or more points with defined visual data that are equidistant from the point to be sampled, the returned visual data shall be an unspecified value which is the visual data of one of those equidistant points. Where possible, implementations should choose the among the equidistant points that have an *x* axisvalue and a *y* axisvalue that is nearest to 0.0.
- ⁴ See Table 7 for the meaning of each **extend** enumerator.

12.2.2 **extend Synopsis**

[extend.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class extend {
        none,
        repeat,
        reflect,
        pad
    };
} } } }
```

12.2.3 **extend Enumerators**

[extend.enumerators]

Table 7 — **extend** enumerator meanings

Enumerator	Meaning
none	If the point to be sampled is outside of the bounds of the Source Brush, the visual data that is returned shall be the equivalent of <code>rgba_color::transparent_black()</code> .
repeat	If the point to be sampled is outside of the bounds of the Source Brush, the visual data that is returned shall be the visual data that would have been returned if the Source Brush was infinitely large and repeated itself in a left-to-right-left-to-right and top-to-bottom-top-to-bottom fashion.
reflect	If the point to be sampled is outside of the bounds of the Source Brush, the visual data that is returned shall be the visual data that would have been returned if the Source Brush was infinitely large and repeated itself in a left-to-right-to-left-to-right and top-to-bottom-to-top-to-bottom fashion.
pad	If the point to be sampled is outside of the bounds of the Source Brush, the visual data that is returned shall be the visual data that would have been returned for the nearest defined point that is in bounds.

12.3 **Enum class filter**

[filter]

12.3.1 **filter Summary**

[filter.summary]

- ¹ The **filter** enum class specifies the type of filter to use when sampling from a pixmap.

- ² Three of the `filter` enumerators, `filter::fast`, `filter::good`, and `filter::best`, specify desired characteristics of the filter, leaving the choice of a specific filter to the implementation.

The other two, `filter::nearest` and `filter::bilinear`, each specify a particular filter that shall be used.

- ³ The result of sampling from a `brush` object `b` constructed from a `solid_color_brush_factory` is the same regardless of what filter is used and, as such, in these circumstances implementations should disregard the filter specified by the result of calling `b.filter()` when sampling from `b` and instead use an unspecified filter, even if that filter does not correspond to a filter specified by one of the `filter` enumerators.
- ⁴ See Table 8 for the meaning of each `filter` enumerator.

12.3.2 `filter` Synopsis

[filter.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class filter {
        fast,
        good,
        best,
        nearest,
        bilinear
    };
} } } }
```

12.3.3 `filter` Enumerators

[filter.enumerators]

Table 8 — `filter` enumerator meanings

Enumerator	Meaning
<code>fast</code>	The filter that corresponds to this value is implementation-defined. The implementation shall ensure that the time complexity of the chosen filter is not greater than the time complexity of the filter that corresponds to <code>filter::good</code> . [Note: By choosing this value, the user is hinting that performance is more important than quality. — end note]
<code>good</code>	The filter that corresponds to this value is implementation-defined. The implementation shall ensure that the time complexity of the chosen formula is not greater than the time complexity of the formula for <code>filter::best</code> . [Note: By choosing this value, the user is hinting that quality and performance are equally important. — end note]
<code>best</code>	The filter that corresponds to this value is implementation-defined. [Note: By choosing this value, the user is hinting that quality is more important than performance. — end note]
<code>nearest</code>	Nearest-neighbor interpolation filtering shall be used.
<code>bilinear</code>	Bilinear interpolation filtering shall be used.

12.4 Enum class `brush_type`

[brushtype]

12.4.1 `brush_type` Summary

[brushtype.summary]

- ¹ The `brush_type` enum class denotes which brush factory was used to form a `brush` object.

² See Table 9 for the meaning of each `brush_type` enumerator.

12.4.2 `brush_type` Synopsis

[`brushtype.synopsis`]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class brush_type {
        solid_color,
        surface,
        linear,
        radial
    };
} } } }
```

12.4.3 `brush_type` Enumerators

[`brushtype.enumerators`]

Table 9 — `brush_type` enumerator meanings

Enumerator	Meaning
<code>solid_color</code>	The brush object was created from a <code>solid_color_brush_factory</code> object.
<code>surface</code>	The brush object was created from a <code>surface_brush_factory</code> object.
<code>linear</code>	The brush object was created from a <code>linear_brush_factory</code> object.
<code>radial</code>	The brush object was created from a <code>radial_brush_factory</code> object.

12.5 Class `brush`

[`brush`]

12.5.1 `brush` synopsis

[`brush.synopsis`]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class brush {
    public:
        // 12.5.4, construct/copy/move/destroy:
        brush() = delete;
        brush(const brush&) noexcept;
        brush& operator=(const brush&) noexcept;
        brush(brush&& other) noexcept;
        brush& operator=(brush&& other) noexcept;
        brush(const solid_color_brush_factory& f);
        brush(const solid_color_brush_factory& f, error_code& ec) noexcept;
        brush(const linear_brush_factory& f);
        brush(const linear_brush_factory& f, error_code& ec) noexcept;
        brush(const radial_brush_factory& f);
        brush(const radial_brush_factory& f, error_code& ec) noexcept;
        brush(surface_brush_factory& f);
        brush(surface_brush_factory& f, error_code& ec) noexcept;

        // 12.5.5, modifiers:
        void extend(experimental::io2d::extend e) noexcept;
        void filter(experimental::io2d::filter f) noexcept;
        void matrix(const matrix_2d& m) noexcept;

        // 12.5.6, observers:
```

```

    experimental::io2d::extend extend() const noexcept;
    experimental::io2d::filter filter() const noexcept;
    matrix_2d matrix() const noexcept;
    brush_type type() const noexcept;

private:
    experimental::io2d::extend _Extend; // exposition only
    experimental::io2d::filter _Filter; // exposition only
    matrix_2d _Matrix;                  // exposition only
    brush_type _Brush_type;              // exposition only
};
} } } }

```

12.5.2 brush Description

[brush.intro]

- ¹ The class **brush** describes an opaque wrapper for a graphics data graphics resource.
- ² A **brush** object shall be usable with any **surface** or **surface**-derived object.
- ³ A **brush** object's graphics data is immutable. It is observable only by the effect that it produces when the brush is used as a Source Brush (13.11.5) or as a Mask Brush (13.11.4 and 13.11.10).
- ⁴ A **brush** object also has mutable **extend**, **filter**, and **matrix_2d** observable state data values.
- ⁵ A **brush** object has an immutable **brush_type** observable state data value which indicates the type of brush factory that was used in creating the **brush** object (Table 9).
- ⁶ The **brush** object's graphics data may have less precision than the graphics data of the brush factory object from which it was created.
- ⁷ [*Example: Several graphics and rendering technologies that are currently widely used store individual color and alpha channel data as 8-bit unsigned normalized integer values while the **double** type that is used by the **rgba_color** class for individual color and alpha is often a 64-bit value. It is precisely these situations which the preceding paragraph is intended to address. — end example*]

12.5.3 Sampling from a brush object

[brush.sampling]

- ¹ When sampling from a **brush** object **b**, the **brush_type** returned by calling **b.type()** shall determine how the results of sampling shall be determined:
 1. If the result of **b.type()** is **brush_type::solid_color** then **b** is a *solid color brush*.
 2. If the result of **b.type()** is **brush_type::surface** then **b** is a *surface brush*.
 3. If the result of **b.type()** is **brush_type::linear** then **b** is a *linear gradient brush*.
 4. If the result of **b.type()** is **brush_type::radial** then **b** is a *radial gradient brush*.

12.5.3.1 Sampling from a solid color brush

[brush.sampling.solidcolor]

- ¹ When **b** is a solid color brush, then when sampling from **b**, the visual data returned shall always be the visual data equivalent to the result of calling **solid_color_brush_factory::color** on the **solid_color_brush_factory** from which **b** was created, regardless of the point which is to be sampled and regardless of the return values of **b.extend()**, **b.filter()**, and **b.matrix()**.

12.5.3.2 Sampling from a linear gradient brush

[brush.sampling.linear]

- ¹ When **b** is a linear gradient brush, then when sampling from **b**, the visual data returned shall be from the point **pt** in the rendered linear gradient, where **pt** is the return value when passing the point to be sampled to **b.matrix().transform_coords** and the rendered linear gradient is created as specified by 12.1.2 and 12.1.4, taking into account the value of **b.extend()** and **b.filter()**.

12.5.3.3 Sampling from a radial gradient brush [brush.sampling.radial]

- ¹ When **b** is a radial gradient brush, then when sampling from **b**, the visual data returned shall be from the point **pt** in the rendered radial gradient, where **pt** is the return value when passing the point to be sampled to **b.matrix().transform_coords** and the rendered radial gradient is created as specified by 12.1.3 and 12.1.4, taking into account the value of **b.extend()** and **b.filter()**.

12.5.3.4 Sampling from a surface brush [brush.sampling.surface]

- ¹ When **b** is a surface brush, then when sampling from **b**, the visual data returned shall be from the point **pt** in the graphics data of the brush, where **pt** is the return value when passing the point to be sampled to **b.matrix().transform_coords**, taking into account the value of **b.extend()** and **b.filter()**.

12.5.4 brush constructors and assignment operators [brush.cons]

```
brush(const solid_color_brush_factory& f);
brush(const solid_color_brush_factory& f, error_code& ec) noexcept;
```

- ¹ *Effects:* Constructs an object of type brush.
- ² The graphics data of the brush shall be created from the return value of **f.color()**. The visual data format of the graphics data shall be as if it is that specified by **format::argb**.
- ³ *Postconditions:* **_Brush_type == brush_type::solid_color**.
_Extend == std::experimental::io2d::extend::none.
_Filter == std::experimental::io2d::filter::fast.
_Matrix == matrix_2d::init_identity().
- ⁴ *Throws:* As specified in Error reporting (3).
- ⁵ *Error conditions:* **errc::not_enough_memory** if there was a failure to allocate memory.
io2d_error::invalid_status if there was a failure to allocate a resource other than memory.

```
brush(const linear_brush_factory& f);
brush(const linear_brush_factory& f, error_code& ec) noexcept;
```

- ⁶ *Effects:* Constructs an object of type brush.
- ⁷ The graphics data of the brush is nominally as specified in 12.1.1 and 12.1.2. However because the graphics data is not directly observable, it is unspecified what data is stored and how it is stored, provided that the results of sampling from the brush are the same as if the brush's graphics data was stored as specified in 12.1.1 and 12.1.2.
- ⁸ *Postconditions:* **_Brush_type == brush_type::linear**.
_Extend == std::experimental::io2d::extend::none.
_Filter == std::experimental::io2d::extend::good.
_Extend == std::experimental::io2d::extend::none.
- ⁹ *Throws:* As specified in Error reporting (3).
- ¹⁰ *Error conditions:* **errc::not_enough_memory** if there was a failure to allocate memory.
io2d_error::invalid_status if there was a failure to allocate a resource other than memory.

```
brush(const radial_brush_factory& f);
brush(const radial_brush_factory& f, error_code& ec) noexcept;
```

- 11 *Effects:* Constructs an object of type `brush`.
- 12 The graphics data of the brush is nominally as specified in 12.1.1 and 12.1.3. However because the graphics data is not directly observable, it is unspecified what data is stored and how it is stored, provided that the results of sampling from the brush are the same as if the brush's graphics data was stored as specified in 12.1.1 and 12.1.3.
- 13 *Postconditions:* `_Brush_type == brush_type::radial`.
`_Extend == std::experimental::io2d::extend::none`.
`_Filter == std::experimental::io2d::extend::good`.
`_Extend == std::experimental::io2d::extend::none`.
- 14 *Throws:* As specified in Error reporting (3).
- 15 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.
`io2d_error::invalid_status` if there was a failure to allocate a resource other than memory.

```
brush(surface_brush_factory& f);
brush(surface_brush_factory& f, error_code& ec) noexcept;
```

- 16 *Effects:* Constructs an object of type `brush`.
- 17 The graphics data of the brush is as if it was a copy of the underlying raster graphics data graphics resource of the `image_surface` object returned by const reference by `f.surface()`.
- 18 *Postconditions:* `_Brush_type == brush_type::surface`.
`_Extend == std::experimental::io2d::extend::none`.
`_Filter == std::experimental::io2d::extend::good`.
`_Extend == std::experimental::io2d::extend::none`.
- 19 *Throws:* As specified in Error reporting (3).
- 20 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.
`io2d_error::invalid_status` if there was a failure to allocate a resource other than memory.

12.5.5 brush modifiers

[brush.modifiers]

```
void extend(experimental::io2d::extend e) noexcept;
```

- 1 *Effects:* Sets the `extend` value for the `brush` object.
- 2 *Postconditions:* `_Extend == e`.

```
void filter(experimental::io2d::filter f) noexcept;
```

- 3 *Effects:* Sets the `filter` value for the `brush` object.
- 4 *Postconditions:* `_Filter == f`.

```
void matrix(const matrix_2d& m) noexcept;
```

- 5 *Effects:* Sets the `matrix_2d` value for the `brush` object.
- 6 *Postconditions:* `_Matrix == m`.

12.5.6 brush observers**[brush.observers]**

```

experimental::io2d::extend extend() const noexcept;
1     Returns: _Extend.

experimental::io2d::filter filter() const noexcept;
2     Returns: _Filter.

matrix_2d matrix() const noexcept;
3     Returns: _Matrix.

brush_type type() const noexcept;
4     Returns: _Brush_type.

```

12.6 Class solid_color_brush_factory**[solidcolorbrushfact]****12.6.1 solid_color_brush_factory synopsis****[solidcolorbrushfact.synopsis]**

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class solid_color_brush_factory {
    public:
        // 12.6.3, construct/copy/move/destroy:
        solid_color_brush_factory() noexcept;
        solid_color_brush_factory(const solid_color_brush_factory&) noexcept;
        solid_color_brush_factory& operator=(
            const solid_color_brush_factory&) noexcept;
        solid_color_brush_factory(solid_color_brush_factory&& other) noexcept;
        solid_color_brush_factory& operator=(
            solid_color_brush_factory&& other) noexcept;
        solid_color_brush_factory(const rgba_color& color) noexcept;

        // 12.6.4, modifiers:
        void color(const rgba_color& value) noexcept;

        // 12.6.5, observers:
        rgba_color color() const noexcept;

    private:
        rgba_color _Color;      // exposition only
    };
} } } }

```

12.6.2 solid_color_brush_factory Description**[solidcolorbrushfact.intro]**

1 The class `solid_color_brush_factory` describes a mutable factory for creating brush objects with uniform color and alpha data.

12.6.3 solid_color_brush_factory constructors and assignment operators**[solidcolorbrushfact.cons]**

```

solid_color_brush_factory() noexcept;
1     Effects: Constructs an object of type solid_color_brush_factory.
2     Postcondition: _Color == rgba_color{}.

```



```
solid_color_brush_factory(const rgba_color& color) noexcept;
```

3 *Effects:* Constructs an object of type `solid_color_brush_factory`. A brush created using this object will have `color` as its color.

4 *Postcondition:* `_Color == color`.

12.6.4 `solid_color_brush_factory` modifiers [solidcolorbrushfact.modifiers]

```
void color(const rgba_color& value) noexcept;
```

1 *Effects:* A brush created using this object will have `value` as its color.

2 *Postcondition:* `_Color == value`.

12.6.5 `solid_color_brush_factory` observers [solidcolorbrushfact.observers]

```
rgba_color color() const noexcept;
```

1 *Returns:* `_Color`.

12.7 Class `linear_brush_factory` [linearbrushfact]

12.7.1 `linear_brush_factory` synopsis [linearbrushfact.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class linear_brush_factory {
    public:
        // types
        typedef implementation-defined          size_type; // See (12.7.2).

        // 12.7.4, construct/copy/move/destroy:
        linear_brush_factory() noexcept;
        linear_brush_factory(const linear_brush_factory&);
        linear_brush_factory& operator=(const linear_brush_factory&);
        linear_brush_factory(linear_brush_factory&& other) noexcept;
        linear_brush_factory& operator=(linear_brush_factory&& other) noexcept;
        linear_brush_factory(const vector_2d& begin, const vector_2d& end) noexcept;

        // 12.7.5, modifiers:
        void add_color_stop(double offset, const rgba_color& color);
        void add_color_stop(double offset, const rgba_color& color,
            error_code& ec) noexcept;
        void color_stop(size_type index, double offset,
            const rgba_color& color);
        void color_stop(size_type index, double offset,
            const rgba_color& color, error_code& ec) noexcept;
        void begin_point(const vector_2d& value) noexcept;
        void end_point(const vector_2d& value) noexcept;

        // 12.7.6, observers:
        size_type color_stop_count() const noexcept;
        tuple<double, rgba_color> color_stop(size_type index) const;
        tuple<double, rgba_color> color_stop(size_type index,
            error_code& ec) const noexcept;
        vector_2d begin_point() const noexcept;
        vector_2d end_point() const noexcept;

    private:
```

```

    vector_2d _Begin_point;           // exposition only
    vector_2d _End_point;             // exposition only
    vector<tuple<double, rgba_color>> _Color_stops; // exposition only
};
} } } }

```

12.7.2 `linear_brush_factory::size_type` [linearbrushfact.sizetype]

- 1 The type of `linear_brush_factory::size_type` shall comply with the restrictions specified for the `size_`-type of the collection of color stops that is part of the observable state of a gradient (12.1.1).

12.7.3 `linear_brush_factory` Description [linearbrushfact.intro]

- 1 The class `linear_brush_factory` describes a mutable factory for creating `brush` objects with a linear gradient describing its color and alpha data.
- 2 For more information about gradients, including linear gradients, see 12.1.

12.7.4 `linear_brush_factory` constructors and assignment operators [linearbrushfact.cons]

```
linear_brush_factory() noexcept;
```

- 1 *Effects:* Constructs an object of type `linear_brush_factory`.
- 2 *Postconditions:* `_Begin_point == vector_2d{ }`.
- 3 `_End_point == vector_2d{ }`.
- 4 `_Color_stops.empty() == true`.

```
linear_brush_factory(const vector_2d& begin, const vector_2d& end) noexcept;
```

- 5 *Effects:* Constructs an object of type `linear_brush_factory`.
- 6 *Postconditions:* `_Begin_point == begin`.
- 7 `_End_point == end`.
- 8 `_Color_stops.empty() == true`.

12.7.5 `linear_brush_factory` modifiers [linearbrushfact.modifiers]

```

void add_color_stop(double offset, const rgba_color& color);
void add_color_stop(double offset, const rgba_color& color,
    error_code& ec) noexcept;

```

- 1 *Effects:* Adds a color stop with an offset value of `offset` and a color value of `color`.
- 2 `_Color_stops.push_back(make_tuple(offset, color))`.
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Error conditions:* `errc::not_enough_memory` if the attempt to add the color stop fails.

```

void color_stop(size_type index, double offset,
    const rgba_color& color);
void color_stop(size_type index, double offset,
    const rgba_color& color, error_code& ec) noexcept;

```

- 5 *Effects:* Replaces the color stop at index `index` with a color stop with an offset of `offset` and a color of `color`.
- 6 *Postconditions:* `_Color_stops.at(index) == make_tuple(offset, color)`.

7 *Throws:* As specified in Error reporting (3).
 8 *Error conditions:* `io2d_error::invalid_index` if `_Color_stops.size() <= index`.

```
void begin_point(const vector_2d& value) noexcept;
```

9 *Effects:* Sets the begin point to value.

10 *Postconditions:* `_Begin_point == value`.

```
void end_point(const vector_2d& value) noexcept;
```

11 *Effects:* Sets the end point to value.

12 *Postconditions:* `_End_point == value`.

12.7.6 color_stop_count observers

[linearbrushfact.observers]

```
size_type color_stop_count() const noexcept;
```

1 *Returns:* `static_cast<size_type>(_Color_stops.size())`.

```
tuple<double, rgba_color> color_stop(unsigned int index) const;
tuple<double, rgba_color> color_stop(unsigned int index,
    error_code& ec) const noexcept;
```

2 *Returns:* `_Color_stops.at(index)`.

3 *Throws:* As specified in Error reporting (3).

4 *Error conditions:* `io2d_error::invalid_index` if `_Color_stops.size() <= index`.

```
vector_2d begin_point() const noexcept;
```

5 *Returns:* `_Begin_point`.

```
vector_2d end_point() const noexcept;
```

6 *Returns:* `_End_point`.

12.8 Class radial_brush_factory

[radialbrushfact]

12.8.1 radial_brush_factory synopsis

[radialbrushfact.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class radial_brush_factory {
    public:
```

```
        // types
```

```
        typedef implementation_defined size_type; // See (12.8.2).
```

```
        // 12.8.4, construct/copy/move/destroy:
```

```
        radial_brush_factory() noexcept;
        radial_brush_factory(const radial_brush_factory&);
        radial_brush_factory& operator=(const radial_brush_factory&);
        radial_brush_factory(radial_brush_factory&& other) noexcept;
        radial_brush_factory& operator=(radial_brush_factory&& other) noexcept;
        radial_brush_factory(const vector_2d& center0, double radius0,
            const vector_2d& center1, double radius1) noexcept;
```

```
        // 12.8.5, modifiers:
```

```
        void add_color_stop(double offset, const rgba_color& color);
        void add_color_stop(double offset, const rgba_color& color,
```

```

    error_code& ec) noexcept;
void color_stop(size_type index, double offset, const rgba_color& color);
void color_stop(size_type index, double offset, const rgba_color& color,
    error_code& ec) noexcept;
void radial_circles(const vector_2d& center0, double radius0,
    const vector_2d& center1, double radius1) noexcept;

// 12.8.6, observers:
size_type color_stop_count() const noexcept;
tuple<double, rgba_color> color_stop(size_type index) const;
tuple<double, rgba_color> color_stop(size_type index,
    error_code& ec) const noexcept;
tuple<vector_2d, double, vector_2d, double> radial_circles() const noexcept;

private:
    vector_2d _Center0; // exposition only
    double _Radius0; // exposition only
    vector_2d _Center1; // exposition only
    double _Radius1; // exposition only
    vector<tuple<double, rgba_color>> _Color_stops; // exposition only
};
} } } }

```

12.8.2 radial_brush_factory::size_type [radialbrushfact.sizetype]

- 1 The type of `radial_brush_factory::size_type` shall comply with the restrictions specified for the `size_type` of the collection of color stops that is part of the observable state of a gradient (12.1.1).

12.8.3 radial_brush_factory Description [radialbrushfact.intro]

- 1 The class `radial_brush_factory` describes a mutable factory for creating `brush` objects with a radial gradient describing its color and alpha data.
- 2 For more information about gradients, including radial gradients, see 12.1.

12.8.4 radial_brush_factory constructors and assignment operators [radialbrushfact.cons]

```
radial_brush_factory() noexcept;
```

- 1 *Effects:* Constructs an object of type `radial_brush_factory`.
- 2 *Postconditions:* `_Center0 == vector_2d{ }`.
- 3 `_Radius0 == 0.0`.
- 4 `_Center1 == vector_2d{ }`.
- 5 `_Radius1 == 0.0`.
- 6 `_Color_stops.empty() == true`.

```
radial_brush_factory(const vector_2d& center0, double radius0,
    const vector_2d& center1, double radius1) noexcept;
```

- 7 *Effects:* Constructs an object of type `radial_brush_factory`.
- 8 *Postconditions:* `_Center0 == center0`.
- 9 `_Radius0 == radius0`.
- 10 `_Center1 == center1`.

```

11     _Radius1 == radius1.
12     _Color_stops.empty() == true.

```

12.8.5 radial_brush_factory modifiers

[radialbrushfact.modifiers]

```

void add_color_stop(double offset, const rgba_color& color);
void add_color_stop(double offset, const rgba_color& color,
    error_code& ec) noexcept;

```

- 1 *Effects:* Adds a color stop with an offset value of `offset` and a color value of `color`.
- 2 *Postconditions:* `_Color_stops.push_back(make_tuple(offset, color))`.
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Error conditions:* `errc::not_enough_memory` if the attempt to add the color stop fails.

```

void color_stop(size_type index, double offset, const rgba_color& color);
void color_stop(size_type index, double offset, const rgba_color& color,
    error_code& ec) noexcept;

```

- 5 *Effects:* Replaces the color stop at index `index` with a color stop with an offset of `offset` and a color of `color`.
- 6 *Postconditions:* `_Color_stops.at(index) == make_tuple(offset, color)`.
- 7 *Throws:* As specified in Error reporting (3).
- 8 *Error conditions:* `io2d_error::invalid_index` if `_Color_stops.size() <= index`.

```

void radial_circles(const vector_2d& center0, double radius0,
    const vector_2d& center1, double radius1) noexcept;

```

- 9 *Postconditions:* `_Center0 == center0`.
- 10 `_Radius0 == radius0`.
- 11 `_Center1 == center1`.
- 12 `_Radius1 == radius1`.

12.8.6 radial_brush_factory observers

[radialbrushfact.observers]

```

size_type color_stop_count() const noexcept;

```

- 1 *Returns:* `static_cast<size_type>(_Color_stops.size())`.

```

tuple<double, rgba_color> color_stop(size_type index) const;
tuple<double, rgba_color> color_stop(size_type index,
    error_code& ec) const noexcept;

```

- 2 *Returns:* `_Color_stops.at(index)`.
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Error conditions:* `io2d_error::invalid_index` if `_Color_stops.size() <= index`.

```

tuple<vector_2d, double, vector_2d, double> radial_circles() const noexcept;

```

- 5 *Returns:* `make_tuple(_Center0, _Radius0, _Center1, _Radius1)`.

12.9 Class `surface_brush_factory` [surfacebrushfact]

12.9.1 `surface_brush_factory` synopsis [surfacebrushfact.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class surface_brush_factory {
    public:
        // 12.9.3, construct/copy/move/destroy:
        surface_brush_factory() noexcept;
        surface_brush_factory(const surface_brush_factory&) = delete;
        surface_brush_factory& operator=(const surface_brush_factory&) = delete;
        surface_brush_factory(surface_brush_factory&& other) noexcept;
        surface_brush_factory& operator=(surface_brush_factory&& other) noexcept;
        surface_brush_factory(experimental::io2d::surface& s);
        surface_brush_factory(experimental::io2d::surface& s, error_code& ec) noexcept;

        // 12.9.4, modifiers:
        image_surface surface(experimental::io2d::surface& s);
        void surface(experimental::io2d::surface& s, image_surface& oldSurface,
            error_code& ec) noexcept;
        void surface(experimental::io2d::surface& s, error_code& ec) noexcept;

        // 12.9.5, observers:
        bool has_surface() const noexcept;
        const image_surface& surface() const;

    private:
        unique_ptr<image_surface> _Surface; // exposition only
    };
} } }
```

12.9.2 `surface_brush_factory` Description [surfacebrushfact.intro]

- 1 The class `surface_brush_factory` describes a mutable factory for creating `brush` objects with a `surface` object describing its color and alpha data.

12.9.3 `surface_brush_factory` constructors and assignment operators [surfacebrushfact.cons]

```
surface_brush_factory(experimental::io2d::surface& s);
surface_brush_factory(experimental::io2d::surface& s, error_code& ec) noexcept;
```

- 1 *Effects:* Constructs an object of type `surface_brush_factory`. Calls `s.flush()` then creates a copy of `s` and stores it as the surface which will be painted by a brush formed using `*this`.
- 2 *Postconditions:* `_Surface` contains a valid pointer to an `image_surface` object with the same width, height, `format` as `s` and a copy of the visual data of `s` as raster graphics data.
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Remarks:* Excluding any effects of calling `s.flush()`, `s` shall not be modified.
- 5 *Error conditions:* Any error condition documented for `surface::flush`.
- 6 `errc::invalid_argument` if `!s.has_surface_resource()`.
- 7 `io2d_error::surface_finished` if `s.is_finished()`.
- 8 `errc::not_enough_memory` if an attempted memory allocation failed.

12.9.4 surface_brush_factory modifiers

[surfacebrushfact.modifiers]

```
void surface(experimental::io2d::surface& s);
void surface(experimental::io2d::surface& s, error_code& ec) noexcept;
```

- 1 *Effects:* Calls `s.flush()` then creates a copy of `s` and stores it as the surface which will be painted by a brush formed using `*this`. The stored surface shall be accessible as an object of type `image_surface`.
- 2 *Postconditions:* `_Surface` contains a valid pointer to an `image_surface` object with the same width, height, format as `s` and a copy of the visual data of `s` as raster graphics data.
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Remarks:* Excluding the effects of calling `s.flush()`, `s` shall not be modified.
- 5 *Error conditions:* Any error condition documented for `surface::flush` (13.11).
- 6 `errc::invalid_argument` if `!s.has_surface_resource()`.
- 7 `io2d_error::surface_finished` if `s.is_finished()`.
- 8 `errc::not_enough_memory` if an attempted memory allocation failed.

12.9.5 surface_brush_factory observers

[surfacebrushfact.observers]

```
bool has_surface() const noexcept;

const image_surface& surface() const;
```

- 1 *Returns:* `_Surface.get != nullptr`.
- 2 *Requires:* `has_surface() == true`.
- 3 *Returns:* `_Surface.get()`.

13 Surfaces

[surfaces]

- ¹ Surfaces are composed of visual data, stored in a graphics data graphics resource. [*Note*: All well-defined **surface**-derived types are currently raster graphics data graphics resources with defined bounds. To allow for easier additions of future surface-derived types which a not composed of raster graphics data or do not have fixed bounds, such as a vector graphics-based surface, the less constrained term graphics data graphics resource is used. — *end note*]
- ² The surface's visual data is manipulated by rendering and composing operations (13.11.4).
- ³ Surfaces are stateful objects.
- ⁴ The various **surface**-derived classes each provide specific, unique functionality that enables a broad variety of 2D graphics operations to be accomplished efficiently.

13.1 Enum class **antialias**

[antialias]

13.1.1 **antialias** Summary

[antialias.summary]

- ¹ The **antialias** enum class specifies the type of anti-aliasing that the rendering system shall use for rendering text. See Table 10 for the meaning of each **antialias** enumerator.

13.1.2 **antialias** Synopsis

[antialias.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class antialias {
        default_antialias,
        none,
        gray,
        subpixel,
        fast,
        good,
        best
    };
} } } }
```

13.1.3 **antialias** Enumerators

[antialias.enumerators]

Table 10 — **antialias** enumerator meanings

Enumerator	Meaning
default_antialias	The meaning of this value is implementation-defined.
none	No anti-aliasing.
gray	Monochromatic anti-aliasing. [<i>Note</i> : When rendering black text on a white background, this would produce gray-scale
subpixel	Anti-aliasing that breaks pixels into their constituent color channels and manipulates those color channels individually. The meaning of this value for any rendering operation other than surface::show_text , surface::show_glyphs , and surface::show_text_glyphs is implementation-defined.

Table 10 — **antialias** enumerator meanings (continued)

Enumerator	Meaning
fast	The meaning of this value is implementation-defined. Implementations shall enable some form of anti-aliasing when this option is selected. [<i>Note</i> : By choosing this value, the user is hinting that faster anti-aliasing is preferable to better anti-aliasing. — <i>end note</i>]
good	The meaning of this value is implementation-defined. Implementations shall enable some form of anti-aliasing when this option is selected. [<i>Note</i> : By choosing this value, the user is hinting that sacrificing some performance to obtain better anti-aliasing is acceptable but that performance is still a concern.
best	The meaning of this value is implementation-defined. Implementations shall enable some form of text anti-aliasing when this option is selected. [<i>Note</i> : By choosing this value, the user is hinting that better anti-aliasing is more important than performance.

13.2 Enum class **content**

[content]

13.2.1 **content** Summary

[content.summary]

- ¹ The **content** enum class describes the type of data that a **surface** object contains. See Table 11 for the meaning of each enumerator.

13.2.2 **content** Synopsis

[content.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class content {
        color,
        alpha,
        color_alpha
    };
} } } }
```

13.2.3 **content** Enumerators

[content.enumerators]

Table 11 — **content** value meanings

Enumerator	Meaning
color	The surface holds opaque color data only.
alpha	The surface holds alpha (translucency) data only.
color_alpha	The surface holds both color and alpha data.

13.3 Enum class **fill_rule**

[fillrule]

13.3.1 **fill_rule** Summary

[fillrule.summary]

- ¹ The **fill_rule** enum class determines how the Filling operation (13.11.8) is performed on a path geometry graphics resource.
- ² For each point, draw a ray from that point to infinity which does not pass through the start point or

end point of any non-degenerate path segment in the path geometry graphics resource, is not tangent to any non-degenerate path segment in the path geometry graphics resource, and is not coincident with any non-degenerate path segment in the path geometry graphics resource.

- ³ See Table 12 for the meaning of each `fill_rule` enumerator.

13.3.2 `fill_rule` Synopsis

[fillrule.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class fill_rule {
        winding,
        even_odd
    };
} } } }
```

13.3.3 `fill_rule` Enumerators

[fillrule.enumerators]

Table 12 — `fill_rule` enumerator meanings

Enumerator	Meaning
<code>winding</code>	If the Fill Type (Table 21) is <code>fill_type::winding</code> , then using the ray described above and beginning with a count of zero, add one to the count each time a non-degenerate path segment crosses the ray going left-to-right from its begin point to its end point, and subtract one each time a non-degenerate path segment crosses the ray going from right-to-left from its begin point to its end point. If the resulting count is zero after all non-degenerate path segments that cross the ray have been evaluated, the point shall not be filled; otherwise the point shall be filled.
<code>even_odd</code>	If the Fill Type is <code>fill_type::even_odd</code> , then using the ray described above and beginning with a count of zero, add one to the count each time a non-degenerate path segment crosses the ray. If the resulting count is an odd number after all non-degenerate path segments that cross the ray have been evaluated, the point shall be filled; otherwise the point shall not be filled. [<i>Note</i> : Mathematically, zero is an even number, not an odd number. — <i>end note</i>]

13.4 Enum class `line_cap`

[linecap]

13.4.1 `line_cap` Summary

[linecap.summary]

- ¹ The `line_cap` enum class specifies how the ends of lines should be rendered when a `path` object is stroked. See Table 13 for the meaning of each `line_cap` enumerator.

13.4.2 `line_cap` Synopsis

[linecap.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class line_cap {
        butt,
        round,
        square
    };
} } } }
```

```
};
} } } }
```

13.4.3 line_cap Enumerators

[linecap.enumerators]

Table 13 — line_cap enumerator meanings

Enumerator	Meaning
butt	The line has no cap. It terminates exactly at the end point.
round	The line has a circular cap, with the end point serving as the center of the circle and the line width serving as its diameter.
square	The line has a square cap, with the end point serving as the center of the square and the line width serving as the length of each side.

13.5 Enum class line_join

[linejoin]

13.5.1 line_join Summary

[linejoin.summary]

- ¹ The `line_join` enum class specifies how the junction of two line segments should be rendered when a path is stroked. See Table 14 for the meaning of each enumerator.

13.5.2 line_join Synopsis

[linejoin.synopsis]

```
namespace std { namespace experimental { namespace drawing { inline namespace
v1 {
    enum class line_join {
        miter,
        round,
        bevel
    };
} } } }
```

13.5.3 line_join Enumerators

[linejoin.enumerators]

Table 14 — line_join enumerator meanings

Enumerator	Meaning
miter	Joins will be mitered or beveled, depending on the current Miter Limit (13.11.3).
round	Joins will be rounded, with the center of the circle being the join point.
bevel	Joins will be beveled, with the join cut off at half the line width from the join point. Implementations may vary the cut off distance by an amount that is less than one pixel at each join for aesthetic or technical reasons.

13.6 Enum class `compositing_operator` [`compositing.operator`]

13.6.1 `compositing_operator` Summary [`compositing.operator.summary`]

- ¹ The `compositing_operator` enum class specifies composition algorithms. See Table 15, Table 16 and Table 17 for the meaning of each `compositing_operator` enumerator.

13.6.2 `compositing_operator` Synopsis [`compositing.operator.synopsis`]

```
namespace std { namespace experimental { namespace drawing { inline namespace
v1 {
    enum class compositing_operator {
        // basic
        over,
        clear,
        source,
        in,
        out,
        atop,
        dest,
        dest_over,
        dest_in,
        dest_out,
        dest_atop,
        xor_op,
        add,
        saturate,
        // blend
        multiply,
        screen,
        overlay,
        darken,
        lighten,
        color_dodge,
        color_burn,
        hard_light,
        soft_light,
        difference,
        exclusion,
        // hsl
        hsl_hue,
        hsl_saturation,
        hsl_color,
        hsl_luminosity
    };
} } } }
```

13.6.3 `compositing_operator` Enumerators [`compositing.operator.enumerators`]

- ¹ The tables below specifies the mathematical formula for each enumerator's composition algorithm. The formulas differentiate between three color channels (red, green, and blue) and an alpha channel (transparency). For all channels, valid channel values are in the range [0.0, 1.0].
- ² Where a visual data format for a visual data element has no alpha channel, the visual data format shall be treated as though it had an alpha channel with a value of 1.0 for purposes of evaluating the formulas.
- ³ Where a visual data format for a visual data element has no color channels, the visual data format shall be treated as though it had a value of 0.0 for all color channels for purposes of evaluating the formulas.

- 4 The following symbols and specifiers are used:
 The *R* symbol means the result color value
 The *S* symbol means the source color value
 The *D* symbol means the destination color value
 The *c* specifier means the color channels of the value it follows
 The *a* specifier means the alpha channel of the value it follows
- 5 The color symbols *R*, *S*, and *D* may appear with or without any specifiers.
- 6 If a color symbol appears alone, it designates the entire color as a tuple in the unsigned normalized form (red, green, blue, alpha).
- 7 The specifiers *c* and *a* may appear alone or together after any of the three color symbols.
- 8 The presence of the *c* specifier alone means the three color channels of the color as a tuple in the unsigned normalized form (red, green, blue).
- 9 The presence of the *a* specifier alone means the alpha channel of the color in unsigned normalized form.
- 10 The presence of the specifiers together in the form *ca* means the value of the color as a tuple in the unsigned normalized form (red, green, blue, alpha), where the value of each color channel is the product of each color channel and the alpha channel and the value of the alpha channel is the original value of the alpha channel. [*Example:* When it appears in a formula, *Sca* means $((Sc \times Sa), Sa)$, such that, given a source color $Sc = (1.0, 0.5, 0.0)$ and an source alpha $Sa = (0.5)$, the value of *Sca* when specified in one of the formulas would be $Sca = (1.0 \times 0.5, 0.5 \times 0.5, 0.0 \times 0.5, 0.5) = (0.5, 0.25, 0.0, 0.5)$. The same is true for *Dca* and *Rca*. — end example]
- 11 No space is left between a value and its channel specifiers. Channel specifiers will be preceded by exactly one value symbol.
- 12 When performing an operation that involves evaluating the color channels, each color channel should be evaluated individually to produce its own value.
- 13 The basic enumerators specify a value for Bound. This value may be 'Yes', 'No', or 'N/A'.
- 14 If the Bound value is 'Yes', then the source is treated as though it is also a mask. As such, only areas of the surface where the source would affect the surface are altered. The remaining areas of the surface have the same color value as before the compositing operation.
- 15 If the Bound value is 'No', then every area of the surface that is not affected by the source will become transparent black. In effect, it is as though the source was treated as being the same size as the destination surface with every part of the source that does not already have a color value assigned to it being treated as though it were transparent black. Application of the formula with this precondition results in those areas evaluating to transparent black such that evaluation can be bypassed due to the predetermined outcome.
- 16 If the Bound value is 'N/A', the operation would have the same effect regardless of whether it was treated as 'Yes' or 'No' such that those Bound values are not applicable to the operation. A 'N/A' formula when applied to an area where the source does not provide a value will evaluate to the original value of the destination even if the source is treated as having a value there of transparent black. As such the result is the same as if the source were treated as being a mask, i.e. 'Yes' and 'No' treatment each produce the same result in areas where the source does not have a value.
- 17 If a clip is set and the Bound value is 'Yes' or 'N/A', then only those areas of the surface that the are within the clip will be affected by the compositing operation.
- 18 If a clip is set and the Bound value is 'No', then only those areas of the surface that the are within the clip will be affected by the compositing operation. Even if no part of the source is within the clip, the operation will still set every area within the clip to transparent black. Areas outside the clip are not modified.

Table 15 — `compositing_operator` basic enumerator meanings

Enumerator	Bound	Color	Alpha
<code>clear</code>	Yes	$Rc = 0$	$Ra = 0$
<code>source</code>	Yes	$Rc = Sc$	$Ra = Sa$
<code>over</code>	N/A	$Rc = \frac{(Sca + Dca \times (1 - Sa))}{Ra}$	$Ra = Sa + Da \times (1 - Sa)$
<code>in</code>	No	$Rc = Sc$	$Ra = Sa \times Da$
<code>out</code>	No	$Rc = Sc$	$Ra = Sa \times (1 - Da)$
<code>atop</code>	N/A	$Rc = Sca + Dc \times (1 - Sa)$	$Ra = Da$
<code>dest</code>	N/A	$Rc = Dc$	$Ra = Da$
<code>dest_over</code>	N/A	$Rc = \frac{(Sca \times (1 - Da) + Dca)}{Ra}$	$Ra = (1 - Da) \times Sa + Da$
<code>dest_in</code>	No	$Rc = Dc$	$Ra = Sa \times Da$
<code>dest_out</code>	N/A	$Rc = Dc$	$Ra = (1 - Sa) \times Da$
<code>dest_atop</code>	No	$Rc = Sc \times (1 - Da) + Dca$	$Ra = Sa$
<code>xor_op</code>	N/A	$Rc = \frac{(Sca \times (1 - Da) + Dca \times (1 - Sa))}{Ra}$	$Ra = Sa + Da - 2 \times Sa \times Da$
<code>add</code>	N/A	$Rc = \frac{(Sca + Dca)}{Ra}$	$Ra = \min(1, Sa + Da)$
<code>saturate</code>	N/A	$Rc = \frac{(\min(Sa, 1 - Da) \times Sc + Dca)}{Ra}$	$Ra = \min(1, Sa + Da)$

- ¹⁹ The blend enumerators and hsl enumerators share a common formula for the result color's color channel, with only one part of it changing depending on the enumerator. The result color's color channel value formula is as follows: $Rc = \frac{1}{Ra} \times ((1 - Da) \times Sca + (1 - Sa) \times Dca + Sa \times Da \times f(Sc, Dc))$. The function $f(Sc, Dc)$ is the component of the formula that is enumerator dependent.
- ²⁰ For the blend enumerators, the color channels shall be treated as separable, meaning that the color formula shall be evaluated separately for each color channel: red, green, and blue.
- ²¹ The color formula divides 1 by the result color's alpha channel value. As a result, if the result color's alpha channel is zero then a division by zero would normally occur. Implementations shall not throw an exception nor otherwise produce any observable error condition if the result color's alpha channel is zero. Instead, implementations shall bypass the division by zero and produce the result color (0.0, 0.0, 0.0, 0.0), i.e. *transparent black*, if the result color alpha channel formula evaluates to zero. [Note: The simplest way to comply with this requirement is to bypass evaluation of the color channel formula in the event that the result alpha is zero. However, in order to allow implementations the greatest latitude possible, only the result is specified. — end note]
- ²² For the enumerators in Table 16 and Table 17 the result color's alpha channel value formula is as follows: $Ra = Sa + Da \times (1 - Sa)$. [Note: Since it is the same formula for all enumerators in those tables, the formula is not included in those tables. — end note]
- ²³ All of the blend enumerators and hsl enumerators have a Bound value of 'N/A'.

Table 16 — `compositing_operator` blend enumerator meanings

Enumerator	Color
<code>multiply</code>	$f(S_c, D_c) = S_c \times D_c$
<code>screen</code>	$f(S_c, D_c) = S_c + D_c - S_c \times D_c$
<code>overlay</code>	$\text{if}(D_c \leq 0.5) \{$ $f(S_c, D_c) = 2 \times S_c \times D_c$ $\}$ $\text{else} \{$ $f(S_c, D_c) =$ $1 - 2 \times (1 - S_c) \times$ $(1 - D_c)$ $\}$ [<i>Note:</i> The difference between this enumerator and <code>hard_light</code> is that this tests the destination color (D_c) whereas <code>hard_light</code> tests the source color (S_c). — <i>end note</i>]
<code>darken</code>	$f(S_c, D_c) = \min(S_c, D_c)$
<code>lighten</code>	$f(S_c, D_c) = \max(S_c, D_c)$
<code>color_dodge</code>	$\text{if}(D_c < 1) \{$ $f(S_c, D_c) = \min(1, \frac{D_c}{(1 - S_c)})$ $\}$ $\text{else} \{$ $f(S_c, D_c) = 1\}$
<code>color_burn</code>	$\text{if}(D_c > 0) \{$ $f(S_c, D_c) = 1 - \min(1, \frac{1 - D_c}{S_c})$ $\}$ $\text{else} \{$ $f(S_c, D_c) = 0$ $\}$
<code>hard_light</code>	$\text{if}(S_c \leq 0.5) \{$ $f(S_c, D_c) = 2 \times S_c \times D_c$ $\}$ $\text{else} \{$ $f(S_c, D_c) =$ $1 - 2 \times (1 - S_c) \times$ $(1 - D_c)$ $\}$ [<i>Note:</i> The difference between this enumerator and <code>overlay</code> is that this tests the source color (S_c) whereas <code>overlay</code> tests the destination color (D_c). — <i>end note</i>]

Table 16 — `compositing_operator` blend enumerator meanings
(continued)

Enumerator	Color
<code>soft_light</code>	$\begin{aligned} & \text{if } (Sc \leq 0.5) \{ \\ & \quad f(Sc, Dc) = \\ & \quad \quad Dc - (1 - 2 \times Sc) \times Dc \times \\ & \quad \quad (1 - Dc) \\ & \quad \} \\ & \text{else } \{ \\ & \quad f(Sc, Dc) = \\ & \quad \quad Dc + (2 \times Sc - 1) \times \\ & \quad \quad (g(Dc) - Sc) \\ & \quad \} \end{aligned}$ $g(Dc)$ is defined as follows: $\begin{aligned} & \text{if } (Dc \leq 0.25) \{ \\ & \quad g(Dc) = \\ & \quad \quad ((16 \times Dc - 12) \times Dc + \\ & \quad \quad 4) \times Dc \\ & \quad \} \\ & \text{else } \{ \\ & \quad g(Dc) = \sqrt{Dc} \\ & \quad \} \end{aligned}$
<code>difference</code>	$f(Sc, Dc) = \text{abs}(Dc - Sc)$
<code>exclusion</code>	$f(Sc, Dc) = Sc + Dc - 2 \times Sc \times Dc$

²⁴ For the `hsl` enumerators, the color channels shall be treated as nonseparable, meaning that the color formula shall be evaluated once, with the colors being passed in as tuples in the form (red, green, blue).

²⁵ The following additional functions are used to define the `hsl` enumerator formulas:

²⁶ $\text{min}(x, y, z) = \text{min}(x, \text{min}(y, z))$

²⁷ $\text{max}(x, y, z) = \text{max}(x, \text{max}(y, z))$

²⁸ $\text{sat}(C) = \text{max}(Cr, Cg, Cb) - \text{min}(Cr, Cg, Cb)$

²⁹ $\text{lum}(C) = Cr \times 0.3 + Cg \times 0.59 + Cb \times 0.11$

³⁰ $\text{clip_color}(C) = \{$
 $\quad L = \text{lum}(C)$
 $\quad N = \text{min}(Cr, Cg, Cb)$
 $\quad X = \text{max}(Cr, Cg, Cb)$
 $\quad \text{if } (N < 0.0) \{$
 $\quad \quad Cr = L + \frac{((Cr - L) \times L)}{(L - N)}$
 $\quad \quad Cg = L + \frac{((Cg - L) \times L)}{(L - N)}$
 $\quad \quad Cb = L + \frac{((Cb - L) \times L)}{(L - N)}$
 $\quad \quad \}$
 $\quad \text{if } (X > 1.0) \{$


```

    Cr = L +  $\frac{((Cr - L) \times (1 - L))}{(X - L)}$ 
    Cg = L +  $\frac{((Cg - L) \times (1 - L))}{(X - L)}$ 
    Cb = L +  $\frac{((Cb - L) \times (1 - L))}{(X - L)}$ 
  }
  return C
}
31 set_lum(C, L) = {
    D = L - lum(C)
    Cr = Cr + D
    Cg = Cg + D
    Cb = Cb + D
    return clip_color(C)
}
32 set_sat(C, S) = {
    R = C
    auto& max = (Rr > Rg) ? ((Rr > Rb) ? Rr : Rb) : ((Rg > Rb) ? Rg : Rb)
    auto& mid = (Rr > Rg) ? ((Rr > Rb) ? ((Rg > Rb) ? Rg : Rb) : Rr) : ((Rg > Rb) ? ((Rr > Rb) ? Rr :
Rb) : Rg)
    auto& min = (Rr > Rg) ? ((Rg > Rb) ? Rb : Rg) : ((Rr > Rb) ? Rb : Rr)
    if (max > min) {
        mid =  $\frac{((mid - min) \times S)}{max - min}$ 
        max = S
    }
    else {
        mid = 0.0
        max = 0.0
    }
    min = 0.0
    return R
} [ Note: In the formula, max, mid, and min are reference variables which are bound to the highest value, sec-
ond highest value, and lowest value color channels of the (red, blue, green) tuple R such that the subsequent
operations modify the values of R directly. — end note ]

```

Table 17 — compositing_operator hsl enumerator meanings

Enumerator	Color & Alpha
hsl_hue	$f(Sc, Dc) = set_lum(set_sat(Sc, sat(Dc)), lum(Dc))$
hsl_saturation	$(Sc, Dc) = set_lum(set_sat(Dc, sat(Sc)), lum(Dc))$
hsl_color	$f(Sc, Dc) = set_lum(Sc, lum(Dc))$
hsl_luminosity	$f(Sc, Dc) = set_lum(Dc, lum(Sc))$

13.7 Enum class format

[format]

13.7.1 format Summary

[format.summary]

- ¹ The **format** enum class indicates a visual data format. See Table 18 for the meaning of each **format** enumerator.

- ² Unless otherwise specified, a visual data format shall be an unsigned integral value of the specified bit size in native-endian format.
- ³ A channel value of 0x0 means that there is no contribution from that channel. As the channel value increases towards the maximum unsigned integral value representable by the number of bits of the channel, the contribution from that channel also increases, with the maximum value representing the maximum contribution from that channel. [*Example:* Given a 5-bit channel representing the color , a value of 0x0 means that the red channel does not contribute any value towards the final color of the pixel. A value of 0x1F means that the red channel makes its maximum contribution to the final color of the pixel.

A — *end example*]

13.7.2 format Synopsis

[format.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class format {
        invalid,
        argb32,
        rgb24,
        a8,
        rgb16_565,
        rgb30
    };
} } } }
```

13.7.3 format Enumerators

[format.enumerators]

Table 18 — format enumerator meanings

Enumerator	Meaning
invalid	A previously specified <code>format</code> is unsupported by the implementation.
argb32	A 32-bit RGB color model pixel format. The upper 8 bits are an alpha channel, followed by an 8-bit red color channel, then an 8-bit green color channel, and finally an 8-bit blue color channel. The value in each channel is an unsigned normalized integer. This is a premultiplied format.
rgb24	A 32-bit RGB color model pixel format. The upper 8 bits are unused, followed by an 8-bit red color channel, then an 8-bit green color channel, and finally an 8-bit blue color channel.
a8	An 8-bit transparency data pixel format. All 8 bits are an alpha channel.
rgb16_565	A 16-bit RGB color model pixel format. The upper 5 bits are a red color channel, followed by a 6-bit green color channel, and finally a 5-bit blue color channel.
rgb30	A 32-bit RGB color model pixel format. The upper 2 bits are unused, followed by a 10-bit red color channel, a 10-bit green color channel, and finally a 10-bit blue color channel. The value in each channel is an unsigned normalized integer.

13.8 Enum class scaling

[scaling]

13.8.1 scaling Summary

[scaling.summary]

- ¹ The scaling enum class specifies the type of scaling a `display_surface` will use when the size of its Display Buffer (13.13.2) differs from the size of its Back Buffer (13.13.2).
- ² See Table 19 for the meaning of each `scaling` enumerator.

13.8.2 scaling Synopsis

[scaling.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class scaling {
        letterbox,
        uniform,
        fill_uniform,
        fill_exact,
        none
    };
} } } }
```

13.8.3 scaling Enumerators

[scaling.enumerators]

- ¹ [*Note:* In the following table, examples will be given to help explain the meaning of each enumerator. The examples will all use a `display_surface` called `ds`.

The Back Buffer (13.13.2) of `ds` is 640x480 (i.e. it has a width of 640 pixels and a height of 480 pixels), giving it an aspect ratio of 1.3̄.

The Display Buffer (13.13.2) of `ds` is 1280x720, giving it an aspect ratio of 1.7̄.

When a rectangle is defined in an example, the coordinate (x_1, y_1) denotes the top left corner of the rectangle, inclusive, and the coordinate (x_2, y_2) denotes the bottom right corner of the rectangle, exclusive. As such, a rectangle with $(x_1, y_1) = (10, 10)$, $(x_2, y_2) = (20, 20)$ is 10 pixels wide and 10 pixels tall and includes the pixel $(x, y) = (19, 19)$ but does not include the pixels $(x, y) = (20, 19)$ or $(x, y) = (19, 20)$. — *end note*]

Table 19 — **scaling** enumerator meanings

Enumerator	Meaning
letterbox	Fill the Display Buffer with the Letterbox Brush (13.13.3) of the display_surface. Uniformly scale the Back Buffer so that one dimension of it is the same length as the same dimension of the Display Buffer and the second dimension of it is not longer than the second dimension of the Display Buffer and transfer the scaled Back Buffer to the Display Buffer using sampling such that it is centered in the Display Buffer. [<i>Example:</i> The Display Buffer of ds will be filled with the brush object returned by ds.letterbox_brush(); . The Back Buffer of ds will be scaled so that it is 960x720, thereby retaining its original aspect ratio. The scaled Back Buffer will be transferred to the Display Buffer using sampling such that it is in the rectangle $(x1, y1) = (\frac{1280}{2} - \frac{960}{2}, 0) = (160, 0),$ $(x2, y2) = (960 + (\frac{1280}{2} - \frac{960}{2}), 720) = (1120, 720).$ This fulfills all of the conditions. At least one dimension of the scaled Back Buffer is the same length as the same dimension of the Display Buffer (both have a height of 720 pixels). The second dimension of the scaled Back Buffer is not longer than the second dimension of the Display Buffer (the Back Buffer's scaled width is 960 pixels, which is not longer than the Display Buffer's width of 1280 pixels. Lastly, the scaled Back Buffer is centered in the Display Buffer (on the <i>x</i> axis there are 160 pixels between each vertical side of the scaled Back Buffer and the nearest vertical edge of the Display Buffer and on the <i>y</i> axis there are 0 pixels between each horizontal side of the scaled Back Buffer and the nearest horizontal edge of the Display Buffer). — <i>end example</i>]

Table 19 — **scaling** enumerator meanings (continued)

Enumerator	Meaning
uniform	Uniformly scale the Back Buffer so that one dimension of it is the same length as the same dimension of the Display Buffer and the second dimension of it is not longer than the second dimension of the Display Buffer and transfer the scaled Back Buffer to the Display Buffer using sampling such that it is centered in the Display Buffer. [<i>Example:</i> The Back Buffer of ds will be scaled so that it is 960x720, thereby retaining its original aspect ratio. The scaled Back Buffer will be transferred to the Display Buffer using sampling such that it is in the rectangle $(x1, y1) = (\frac{1280}{2} - \frac{960}{2}, 0) = (160, 0),$ $(x2, y2) = (960 + (\frac{1280}{2} - \frac{960}{2}), 720) = (1120, 720).$ This fulfills all of the conditions. At least one dimension of the scaled Back Buffer is the same length as the same dimension of the Display Buffer (both have a height of 720 pixels). The second dimension of the scaled Back Buffer is not longer than the second dimension of the Display Buffer (the Back Buffer's scaled width is 960 pixels, which is not longer than the Display Buffer's width of 1280 pixels. Lastly, the scaled Back Buffer is centered in the Display Buffer (on the x axis there are 160 pixels between each vertical side of the scaled Back Buffer and the nearest vertical edge of the Display Buffer and on the y axis there are 0 pixels between each horizontal side of the scaled Back Buffer and the nearest horizontal edge of the Display Buffer). — <i>end example</i>] [<i>Note:</i> The difference between uniform and letterbox is that uniform does not modify the contents of the Display Buffer that fall outside of the rectangle into which the scaled Back Buffer is drawn while letterbox fills those areas with the display_surface object's Letterbox Brush. — <i>end note</i>]

Table 19 — **scaling** enumerator meanings (continued)

Enumerator	Meaning
fill_uniform	Uniformly scale the Back Buffer so that one dimension of it is the same length as the same dimension of the Display Buffer and the second dimension of it is not shorter than the second dimension of the Display Buffer and transfer the scaled Back Buffer to the Display Buffer using sampling such that it is centered in the Display Buffer. [<i>Example:</i> The Back Buffer of ds will be drawn in the rectangle $(x1, y1) = (0, -120)$, $(x2, y2) = (1280, 840)$. This fulfills all of the conditions. At least one dimension of the scaled Back Buffer is the same length as the same dimension of the Display Buffer (both have a width of 1280 pixels). The second dimension of the scaled Back Buffer is not shorter than the second dimension of the Display Buffer (the Back Buffer's scaled height is 840 pixels, which is not shorter than the Display Buffer's height of 720 pixels). Lastly, the scaled Back Buffer is centered in the Display Buffer (on the x axis there are 0 pixels between each vertical side of the rectangle and the nearest vertical edge of the Display Buffer and on the y axis there are 120 pixels between each horizontal side of the rectangle and the nearest horizontal edge of the Display Buffer). — <i>end example</i>]
fill_exact	Scale the Back Buffer so that each dimension of it is the same length as the same dimension of the Display Buffer and transfer the scaled Back Buffer to the Display Buffer using sampling such that its origin is at the origin of the Display Buffer. [<i>Example:</i> The Back Buffer will be drawn in the rectangle $(x1, y1) = (0, 0)$, $(x2, y2) = (1280, 720)$. This fulfills all of the conditions. Each dimension of the scaled Back Buffer is the same length as the same dimension of the Display Buffer (both have a width of 1280 pixels and a height of 720 pixels) and the origin of the scaled Back Buffer is at the origin of the Display Buffer. — <i>end example</i>]
none	Do not perform any scaling. Transfer the Back Buffer to the Display Buffer using sampling such that its origin is at the origin of the Display Buffer. [<i>Example:</i> The Back Buffer of ds will be drawn in the rectangle $(x1, y1) = (0, 0)$, $(x2, y2) = (640, 480)$ such that no scaling occurs and the origin of the Back Buffer is at the origin of the Display Buffer. — <i>end example</i>]

13.9 Enum class refresh_rate**[refreshrate]****13.9.1 refresh_rate Summary****[refreshrate.summary]**

- ¹ The **refresh_rate** enum class describes when the Draw Callback (Table 24) of a **display_surface** object shall be called. See Table 20 for the meaning of each enumerator.

13.9.2 refresh_rate Synopsis

[refreshrate.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    enum class refresh_rate {
        as_needed,
        as_fast_as_possible,
        fixed
    };
} } } }
```

13.9.3 refresh_rate Enumerators

[refreshrate.enumerators]

Table 20 — refresh_rate value meanings

Enumerator	Meaning
as_needed	The Draw Callback shall be called when the implementation needs to do so. [<i>Note</i> : The intention of this enumerator is that implementations will call the Draw Callback as little as possible in order to minimize power usage. Users can call <code>display_surface::redraw_required</code> to make the implementation run the Draw Callback whenever the user requires. — <i>end note</i>]
as_fast_as_possible	The Draw Callback shall be called as frequently as possible, subject to any limits of the execution environment and the underlying rendering and presentation technologies.

Table 20 — `refresh_rate` value meanings (continued)

Enumerator	Meaning
<code>fixed</code>	The Draw Callback shall be called as frequently as needed to maintain the Desired Frame Rate (Table 24) as closely as possible. If more time has passed between two successive calls to the Draw Callback than is required, it shall be called <i>excess time</i> and it shall count towards the <i>required time</i>, which is the time that is required to pass after a call to the Draw Callback before the next successive call to the Draw Callback shall be made. If the excess time is greater than the required time, implementations shall call the Draw Callback and then repeatedly subtract the required time from the excess time until the excess time is less than the required time. If the implementation needs to call the Draw Callback for some other reason, it shall use that call as the new starting point for maintaining the Desired Frame Rate. [<i>Example:</i> Given a Desired Frame Rate of 20.0, then as per the above, the implementation would call the Draw Callback at 50 millisecond intervals or as close thereto as possible. If for some reason the excess time is 51 milliseconds, the implementation would call the Draw Callback, subtract 50 milliseconds from the excess time, and then would wait 49 milliseconds before calling the Draw Callback again. If only 15 milliseconds have passed since the Draw Callback was last called and the implementation needs to call the Draw Callback again, then the implementation shall call the Draw Callback immediately and proceed to wait 50 milliseconds before calling the Draw Callback again. — <i>end example</i>]

13.10 Class device

[device]

13.10.1 device synopsis

[device.synopsis]

```

namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class device {
    public:
        // See 2.3
        typedef implementation-defined native_handle_type; // exposition only
        native_handle_type native_handle() const noexcept; // exposition only

        device() = delete;
        device(const device&) = delete;
        device& operator=(const device&) = delete;
        device(device&& other);
        device& operator=(device&& other);

        // 13.10.3, modifiers:
        void flush() noexcept;
        void lock();
        void lock(error_code& ec) noexcept;
    };
};
};
};

```



```

        void unlock();
        void unlock(error_code& ec) noexcept;
    };
} } } }

```

13.10.2 device Description

[device.intro]

- 1 The **device** class provides access to the underlying rendering and presentation technologies, such graphics devices, graphics device contexts, and swap chains.
- 2 A **device** object is obtained from a **surface** or **surface-derived** object.

13.10.3 device modifiers

[device.modifiers]

```
void flush() noexcept;
```

- 1 *Effects:* The user shall be able to manipulate the underlying rendering and presentation technologies used by the implementation without introducing a race condition.
- 2 *Postconditions:* Any pending device operations shall be executed, batched, or otherwise committed to the underlying rendering and presentation technologies.
- 3 Saved device state, if any, shall be restored.
- 4 *Remarks:* This function exists primarily to allow the user to take control of the underlying rendering and presentation technologies using an implementation-provided native handle.
- 5 The implementation's responsibility is to ensure that the user can safely make changes to the underlying rendering and presentation technologies using a native handle after calling this function.
- 6 The implementation is not required to ensure that every last operation has fully completed so long as those operations which are not complete do not prevent safe use of the underlying rendering and presentation technologies.
- 7 If the underlying technologies internally batch operations in a way that allows them to receive and batch further commands without introducing race conditions, the implementation should return as soon as all pending operations have been submitted to the batch queue.
- 8 This function should not flush the surface to which the device is bound.
- 9 If the implementation does not provide a native handle to the underlying rendering and presentation technologies, this function shall have no observable behavior.
- 10 *Notes:* Users call this function because they wish to use a native handle to the underlying rendering and presentation technologies in order to do something not provided by this Technical Specification (e.g. render native UI controls). As such, the user needs to know that using the underlying rendering system outside of this library will not introduce any race conditions. This function, in combination with locking the device, exists to provide that surety.

```
void lock();
void lock(error_code& ec) noexcept;
```

- 11 *Effects:* Produces all effects of `m.lock()` from *BasicLockable*, 30.2.5.2 in C++ 2014. Implementations shall make this function capable of being recursively reentered from the same thread.
- 12 *Throws:* As described in Error reporting (3).
- 13 *Error conditions:* `errc::resource_unavailable_try_again` if a lock cannot be obtained. [*Note:* One reason this error may occur is if a system limit on the maximum number of times a lock could be recursively acquired would be exceeded. — end note]

```
void unlock();
void unlock(error_code& ec) noexcept;
```

- 14 *Requires:* Meets all requirements of `m.unlock()` from *BasicLockable*, 30.2.5.2 in C++ 2014.
- 15 *Effects:* Produces all effects of `m.unlock()` from *BasicLockable*, 30.2.5.2 in C++ 2014. The lock on `m` shall not be fully released until `m.unlock` has been called a number of times equal to the number of times `m.lock` was successfully called.
- 16 *Throws:* As described in Error reporting (3).
- 17 *Remarks:* This function shall not be called more times than `lock` has been called; no diagnostic is required.

13.11 Class `surface`

[surface]

13.11.1 `surface` synopsis

[surface.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class surface {
    public:
        // 13.11.12, constructors, assignment operators, destructors:
        surface() = delete;
        surface(const surface&) = delete;
        surface& operator=(const surface&) = delete;
        surface(surface&& other) noexcept;
        surface& operator=(surface&& other) noexcept;
        virtual ~surface();

        // 13.11.13, state modifiers:
        virtual void finish() noexcept;
        void flush();
        void flush(error_code& ec) noexcept;
        shared_ptr<experimental::io2d::device> device();
        shared_ptr<experimental::io2d::device> device(error_code& ec) noexcept;
        void mark_dirty();
        void mark_dirty(error_code& ec) noexcept;
        void mark_dirty(const rectangle& rect);
        void mark_dirty(const rectangle& rect, error_code& ec) noexcept;
        void map(const function<void(mapped_surface&)>& action);
        void map(const function<void(mapped_surface&, error_code&)>& action,
            error_code& ec);
        void map(const function<void(mapped_surface&)>& action,
            const rectangle& extents);
        void map(const function<void(mapped_surface&, error_code&)>& action,
            const rectangle& extents, error_code& ec);
        virtual void save();
        virtual void save(error_code& ec) noexcept;
        virtual void restore();
        virtual void restore(error_code& ec) noexcept;
        void brush(experimental::nullopt_t) noexcept;
        void brush(const experimental::io2d::brush& source);
        void brush(const experimental::io2d::brush& source, error_code& ec)
            noexcept;
        void antialias(experimental::io2d::antialias a) noexcept;
        void dashes(experimental::nullopt_t) noexcept;
        void dashes(const experimental::io2d::dashes& d);
        void dashes(const experimental::io2d::dashes& d, error_code& ec) noexcept;
        void fill_rule(experimental::io2d::fill_rule fr) noexcept;
        void line_cap(experimental::io2d::line_cap lc) noexcept;
        void line_join(experimental::io2d::line_join lj) noexcept;
```

```

void line_width(double width) noexcept;
void miter_limit(double limit) noexcept;
void compositing_operator(experimental::io2d::compositing_operator co)
    noexcept;
void clip(const experimental::io2d::path& p);
void clip(const experimental::io2d::path& p, error_code& ec) noexcept;
void clip_immediate();
void clip_immediate(error_code& ec) noexcept;
void path(experimental::nullopt_t) noexcept;
void path(const experimental::io2d::path& p);
void path(const experimental::io2d::path& p, error_code& ec) noexcept;

// 13.11.14, immediate path modifiers:
experimental::io2d::path_factory& immediate() noexcept;

// 13.11.15, render modifiers:
void fill();
void fill(error_code& ec) noexcept;
void fill(const rgba_color& c);
void fill(const rgba_color& c, error_code& ec) noexcept;
void fill(const experimental::io2d::brush& b);
void fill(const experimental::io2d::brush& b, error_code& ec) noexcept;
void fill(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void fill(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
void fill_immediate();
void fill_immediate(error_code& ec) noexcept;
void fill_immediate(const rgba_color& c);
void fill_immediate(const rgba_color& c, error_code& ec) noexcept;
void fill_immediate(const experimental::io2d::brush& b);
void fill_immediate(const experimental::io2d::brush& b, error_code& ec)
    noexcept;
void fill_immediate(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void fill_immediate(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
void paint();
void paint(error_code& ec) noexcept;
void paint(const rgba_color& c);
void paint(const rgba_color& c, error_code& ec) noexcept;
void paint(const experimental::io2d::brush& b);
void paint(const experimental::io2d::brush& b, error_code& ec) noexcept;
void paint(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void paint(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
void paint(double alpha);
void paint(double alpha, error_code& ec) noexcept;

```

```

void paint(const rgba_color& c, double alpha);
void paint(const rgba_color& c, double alpha, error_code& ec) noexcept;
void paint(const experimental::io2d::brush& b, double alpha);
void paint(const experimental::io2d::brush& b, double alpha,
    error_code& ec) noexcept;
void paint(const surface& s, double alpha,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void paint(const surface& s, double alpha, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good)
    noexcept;
void stroke();
void stroke(error_code& ec) noexcept;
void stroke(const rgba_color& c);
void stroke(const rgba_color& c, error_code& ec) noexcept;
void stroke(const experimental::io2d::brush& b);
void stroke(const experimental::io2d::brush& b, error_code& ec) noexcept;
void stroke(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void stroke(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
void stroke_immediate();
void stroke_immediate(error_code& ec) noexcept;
void stroke_immediate(const rgba_color& c);
void stroke_immediate(const rgba_color& c, error_code& ec) noexcept;
void stroke_immediate(const experimental::io2d::brush& b);
void stroke_immediate(const experimental::io2d::brush& b, error_code& ec)
    noexcept;
void stroke_immediate(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void stroke_immediate(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;

// 13.11.16, mask render modifiers:
void mask(const experimental::io2d::brush& mb);
void mask(const experimental::io2d::brush& mb, error_code& ec)
    noexcept;
void mask(const experimental::io2d::brush& mb, const rgba_color& c);
void mask(const experimental::io2d::brush& mb, const rgba_color& c,
    error_code& ec) noexcept;
void mask(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b);
void mask(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b, error_code& ec) noexcept;
void mask(const experimental::io2d::brush& mb, const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void mask(const experimental::io2d::brush& mb, const surface& s,
    error_code& ec, matrix m = matrix_2d::init_identity(),
    extend e = extend::none, filter f = filter::good) noexcept;

```

```

void mask(surface& ms,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask(surface& ms, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask(surface& ms, const rgba_color& c,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask(surface& ms, const rgba_color& c, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask(surface& ms, const experimental::io2d::brush& b,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask(surface& ms, const experimental::io2d::brush& b, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask(surface& ms, const surface& s,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good);
void mask(surface& ms, const surface& s, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good) noexcept;
void mask_immediate(const experimental::io2d::brush& mb);
void mask_immediate(const experimental::io2d::brush& mb, error_code& ec)
    noexcept;
void mask_immediate(const experimental::io2d::brush& mb,
    const rgba_color& c);
void mask_immediate(const experimental::io2d::brush& mb,
    const rgba_color& c, error_code& ec) noexcept;
void mask_immediate(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b);
void mask_immediate(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b, error_code& ec) noexcept;
void mask_immediate(const experimental::io2d::brush& mb, const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void mask_immediate(const experimental::io2d::brush& mb, const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), error_code& ec,
    extend e = extend::none, filter f = filter::good) noexcept;
void mask_immediate(surface& ms,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask_immediate(surface& ms, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask_immediate(surface& ms, const rgba_color& c,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask_immediate(surface& ms, const rgba_color& c, error_code& ec,

```

```

    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask_immediate(surface& ms, const experimental::io2d::brush& b,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask_immediate(surface& ms, const experimental::io2d::brush& b,
    error_code& ec, const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask_immediate(surface& ms, const surface& s,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good);
void mask_immediate(surface& ms, const surface& s, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good) noexcept;

// 13.11.17, text render modifiers:
vector_2d render_text(const string& utf8, const vector_2d& pos);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    error_code& ec) noexcept;
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const rgba_color& c);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const rgba_color& c, error_code& ec) noexcept;
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const experimental::io2d::brush& b);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const experimental::io2d::brush& b, error_code& ec) noexcept;
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const surface& s, const matrix_2d& m = matrix_2d::init_identity(),
    extend e = extend::none, filter f = filter::good);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const surface& s, const matrix_2d& m = matrix_2d::init_identity(), error_code& ec,
    extend e = extend::none, filter f = filter::good) noexcept;

// 13.11.18, transformation modifiers:
void matrix(const matrix_2d& matrix);
void matrix(const matrix_2d& matrix, error_code& ec) noexcept;

// 13.11.19, font modifiers:
void font_face(const string& typeface, font_slant sl, font_weight w);
void font_face(const string& typeface, font_slant sl, font_weight w,
    error_code& ec) noexcept;
void font_face(const experimental::io2d::font_face& f);
void font_face(const experimental::io2d::font_face& f, error_code& ec)
    noexcept;
void font_size(double s) noexcept;
void font_matrix(const matrix_2d& m);
void font_matrix(const matrix_2d& m, error_code& ec) noexcept;
void font_options(const font_options& fo) noexcept;

// 13.11.20, state observers:

```

```

bool is_finished() const noexcept;
experimental::io2d::content content() const noexcept;
experimental::io2d::brush brush() const noexcept;
experimental::io2d::antialias antialias() const noexcept;
experimental::io2d::dashes dashes() const;
experimental::io2d::dashes dashes(error_code& ec) const noexcept;
experimental::io2d::fill_rule fill_rule() const noexcept;
experimental::io2d::line_cap line_cap() const noexcept;
experimental::io2d::line_join line_join() const noexcept;
double line_width() const noexcept;
double miter_limit() const noexcept;
experimental::io2d::compositing_operator compositing_operator() const
    noexcept;
rectangle clip_extents() const noexcept;
bool in_clip(const vector_2d& pt) const noexcept;
vector<rectangle> clip_rectangles() const;
vector<rectangle> clip_rectangles(error_code& ec) const noexcept;

// 13.11.21, render observers:
rectangle fill_extents() const noexcept;
rectangle fill_extents_immediate() const;
rectangle fill_extents_immediate(error_code& ec) const noexcept;
bool in_fill(const vector_2d& pt) const noexcept;
bool in_fill_immediate(const vector_2d& pt) const;
bool in_fill_immediate(const vector_2d& pt, error_code& ec) const noexcept;
rectangle stroke_extents() const noexcept;
rectangle stroke_extents_immediate() const;
rectangle stroke_extents_immediate(error_code& ec) const noexcept;
bool in_stroke(const vector_2d& pt) const noexcept;
bool in_stroke_immediate(const vector_2d& pt) const;
bool in_stroke_immediate(const vector_2d& pt, error_code& ec) const
    noexcept;
experimental::io2d::font_extents font_extents() const noexcept;
experimental::io2d::text_extents text_extents(const string& utf8) const;
experimental::io2d::text_extents text_extents(const string& utf8,
    error_code& ec) const noexcept;

// 13.11.22, transformation observers:
matrix_2d matrix() const noexcept;
vector_2d user_to_surface(const vector_2d& pt) const noexcept;
vector_2d user_to_surface_distance(const vector_2d& dpt) const noexcept;
vector_2d surface_to_user(const vector_2d& pt) const noexcept;
vector_2d surface_to_user_distance(const vector_2d& dpt) const noexcept;

// 13.11.23, font observers:
matrix_2d font_matrix() const noexcept;
experimental::io2d::font_options font_options() const noexcept;
experimental::io2d::font_face font_face() const;
experimental::io2d::font_face font_face(error_code& ec) const noexcept;
};
} } } }

```

13.11.2 surface description

[surface.intro]

- ¹ The `surface` class provides an interface for managing a graphics data graphics resource and its observable state data (13.11.3).

- ² A **surface** object is a move-only object.
- ³ The **surface** class provides two ways to modify its graphics resource:
- (3.1) — Rendering and composing operations.
 - (3.2) — Mapping.
- ⁴ [*Note:* While a **surface** object manages a graphics data graphics resource, the **surface** class does not provide well-defined semantics for the graphics resource. The **surface** class is intended to serve only as a base class and as such is not directly instantiable. — *end note*]
- ⁵ Directly instantiable types which derive, directly or indirectly, from the **surface** class shall either provide well-defined semantics for the graphics data graphics resource or inherit well-defined semantics for the graphics data graphics resource from a base class.
- ⁶ [*Example:* The **image_surface** class and the **display_surface** class each specify that they manage a raster graphics data graphics resource and that the members they inherit from the **surface** class shall use that raster graphics data graphics resource as their graphics data graphics resource. Since, unlike graphics data, raster graphics data provides well-defined semantics, these classes meet the requirements for being directly instantiable. — *end example*]
- ⁷ The definitions of the rendering and composing operations in 13.11.4 shall only be applicable when the graphics data graphics resource on which the **surface** members operate is a raster graphics data graphics resource. In all other cases, any attempt to invoke the rendering and composing operations shall result in undefined behavior.

13.11.3 surface state [surface.state]

- ¹ Table 21 specifies the name, type, function, and default value for each item of a surface's observable state.

13.11.3.1 surface state default values [surface.state.default]

Table 21 — Surface observable state

Name	Type	Function	Default value
<i>Current Brush</i>	brush	This is the brush that shall be available for use when performing rendering and composing operations	<code>brush{ { rgba_color::black() } }</code>
<i>General Antialiasing</i>	antialias	This is the type of antialiasing that should be used when performing non-text rendering operations	<code>antialias::default_ antialias</code>
<i>Dash Pattern</i>	dashes	This specifies the pattern that shall be used when performing stroke rendering operations	<code>dashes{ vector<double>(), 0.0 }</code>
<i>Fill Rule</i>	fill_rule	This controls which areas of paths shall be eligible to be filled when performing fill rendering operations	<code>fill_rule::winding</code>

Table 21 — Surface observable state (continued)

Name	Type	Function	Default value
<i>Line Cap</i>	<code>line_cap</code>	This specifies how the end of a path segment that is not joined to another path segment shall be rendered when performing stroke rendering operations	<code>line_cap::butt</code>
<i>Line Join</i>	<code>line_join</code>	This specifies how the join point of two path segments that are joined to each other shall be rendered when performing stroke rendering operations	<code>line_join::miter</code>
<i>Line Width</i>	<code>double</code>	This is the width, in surface coordinate space units, that shall be used to determine the rendered width of path segments when performing a stroke rendering operation	2.0
<i>Miter Limit</i>	<code>double</code>	This is the value that shall be used to calculate whether a line join shall be mitered or beveled when the value of Line Join is <code>line_join::miter</code> .	10.0
<i>Composition Operator</i>	<code>compositing_operator</code>	This specifies the composition algorithm that shall be used when performing rendering operations	<code>compositing_operator::over</code>
<i>Clip Area</i>	<i>unspecified</i>	The area or areas of the underlying graphics data graphics resource which are the only areas in which composing operations can have any effect. It is in the Surface Coordinate Space (13.11.6)	An unbounded area
<i>Current Path</i>	<code>path</code>	This is the <code>path</code> object that shall be used when performing non-immediate rendering operations. It is in the User Coordinate Space (13.11.6)	<code>path{ path_factory{ } }</code>
<i>Immediate Path</i>	<code>path_factory</code>	This is the <code>path_factory</code> object that shall be used when performing immediate rendering operations. It is in the User Coordinate Space (13.11.6)	<code>path_factory{ }</code>

Table 21 — Surface observable state (continued)

Name	Type	Function	Default value
<i>Transformation Matrix</i>	<code>matrix_2d</code>	This is the <code>matrix_2d</code> object that shall be used to transform points to and from the Surface Coordinate Space (13.11.6)	<code>matrix_2d::init_identity{ }</code>
<i>Font Face</i>	<code>font_face</code>	This is the font that shall be used when performing text rendering operations	implementation-defined
<i>Font Matrix</i>	<code>matrix_2d</code>	This is the <code>matrix_2d</code> object that shall be used to transform coordinates from a font's local coordinate space to the surface's coordinate space when performing text rendering operations	<code>matrix_2d::init_scale{ { 16.0, 16.0 } }</code>
<i>Font Options</i>	<code>font_options</code>	This is the <code>font_options</code> object that shall be used to determine certain aspects of how a font should be rendered when performing text rendering operations	<code>font_option{ antialias::default_antialias, subpixel_order::default_subpixel_order }</code>

- ¹ [*Note:* The Clip Area may be modified by intersecting the current Clip Area with the areas of a `path` object that would be filled according to the current Fill Rule. The resulting Clip Area will only contain areas that were already in the Clip Area and does not need to be a valid `path` object since the Clip Area is only guaranteed to be observable by the effects it has on composing operations. For this reason, the Clip Area's type is unspecified. The Clip Area may also be modified by resetting it to its default value, which is the only way of enlarging the current Clip Area. — *end note*]

13.11.3.2 surface saved state

[`surface.state.save`]

- ¹ A surface object provides an interface to save its current state and subsequently restore it.
- ² Save and restore operations are performed using `surface::save` and `surface::restore`, respectively.
- ³ Save and restore operations may be nested.
- ⁴ Each call to `surface::restore` restores a surface object's state to its values at the time of the most recent call to `surface::save`.
- ⁵ The following list denotes each item of observable state that shall be saved by `surface::save` and restored by `surface::restore`, using the state item names listed in Table 21:
- (5.1) — Current Brush
 - (5.2) — General Antialiasing
 - (5.3) — Dash Pattern
 - (5.4) — Fill Rule
 - (5.5) — Line Cap

- (5.6) — Line Join
- (5.7) — Line Width
- (5.8) — Miter Limit
- (5.9) — Compositing Operator
- (5.10) — Tolerance
- (5.11) — Clip Area
- (5.12) — Current Path
- (5.13) — Immediate Path
- (5.14) — Transformation Matrix
- (5.15) — Font Face
- (5.16) — Font Matrix
- (5.17) — Font Options

13.11.4 surface rendering and composing operations [surface.rendering]

- ¹ The **surface** class provides five fundamental rendering and composing operations:

Table 22 — **surface** rendering and composing operations

Operation	Function(s)
Painting	<code>surface::paint</code>
Filling	<code>surface::fill</code> , <code>surface::fill_immediate</code>
Strokeing	<code>surface::stroke</code> , <code>surface::stroke_immediate</code>
Masking	<code>surface::mask</code> , <code>surface::mask_immediate</code>
Typesetting	<code>surface::draw_text</code>

- ² The filling, stroking, and masking operations each provide two functions, not including overloads, for invoking their functionality.
- ³ Certain rendering and composing operations require a *Source Path*, which is a path geometry graphics resource.
- ⁴ The rendering and composing operations invoked by the `surface::fill`, `surface::stroke`, and `surface::mask` functions shall use the **surface** object's Current Path as their Source Path.
- ⁵ The rendering and composing operations invoked by the `surface::fill_immediate`, `surface::stroke_immediate`, and `surface::mask_immediate` functions shall use as their Source Path a path geometry graphics resource formed in the manner specified by 10.1.2 from the (*const* reference to) `vector<path_data_item>` returned by calling the `path_factory::data_ref` member function on the surface's Immediate Path.
- ⁶ Provided that none of the function calls results in an error, given a **surface** object **s**, there shall be no observable difference in the results of:
- (6.1) — calling `surface::fill_immediate` on **s** versus calling `surface::fill` on **s** with the same arguments immediately after creating a **path** object from the result of calling `surface::immediate` on **s** and then

immediately setting that **path** object as the Current Path by calling **surface::path** on **s** using that **path** object as the argument to that function;

- (6.2) — calling **surface::stroke_immediate** on **s** versus calling **surface::stroke** on **s** with the same arguments immediately after creating a **path** object from the result of calling **surface::immediate** on **s** and then immediately setting that **path** object as the Current Path by calling **surface::path** on **s** using that **path** object as the argument to that function;
 - (6.3) — calling **surface::mask_immediate** on **s** versus calling **surface::mask** on **s** with the same arguments immediately after creating a **path** object from the result of calling **surface::immediate** on **s** and then immediately setting that **path** object as the Current Path by calling **surface::path** on **s** using that **path** object as the argument to that function.
- ⁷ The painting operation is a single-part operation consisting of a composing operation. Each of the other operations is a multi-part operation consisting of a rendering operation followed by a composing operation.
- ⁸ The surface's Clip Area shall always apply to all composing operations. This is true even if no mention is made of the Clip Area in the description of a composing operation or if the description of a composing operation uses phraseology that could otherwise be read as being unconstrained. [*Note: Because of this, mention of the applicability of the Clip Area will be omitted in the descriptions of the composing operation portions of the rendering and composing operations that follow. — end note*]
- ⁹ Each of the five fundamental rendering and composing operations has its own set of minimum arguments.
- 1. The Painting operation has no minimum arguments.
 - 2. The Filling operation has no minimum arguments.
 - 3. The Stroking operation has no minimum arguments.
 - 4. The Masking operation has a Mask Brush (13.11.10) which can be composed of one or four minimum arguments: if the first argument to the Masking operation is of type **brush**, it has one minimum argument (that **brush** object); otherwise the first argument is of type **surface** and other three arguments are the first **matrix_2d** argument, the first **extend** argument, and the first **filter** argument, resulting in four minimum arguments.
 - 5. The Typesetting operation has two minimum arguments: an object, used as the text that is rendered and composed to the surface (see 13.11.11), that is of type **string**; and, an object, used to specify the coordinates at which the origin of the first character of the text is positioned (see 13.11.11), of type **vector_2d**.
- ¹⁰ If a reference to an **error_code** object (see 3) is passed to a rendering and composing operation, it shall be considered part of the set of minimum arguments for that operation.
- ¹¹ For the Masking operation, where a **surface** object is used as the Mask Brush argument, it may be immediately followed by an argument of type **vector_2d** which shall be used as the origin of the **surface** object used as the Mask Brush (see 13.11.10) and it shall be considered part of the set of minimum arguments for that operation.

13.11.5 Source Brush for rendering and composing operations [surface.sourcebrush]

- ¹ A *Source Brush* is composed of a graphics data graphics resource, an **extend** value, a **filter** value, and a **matrix_2d** object.
- ² A **brush** object provides all of the components of a Source Brush. As such, **brush** objects will commonly be used as Source Brushes.

- 3 The rendering and composing operations require a Source Brush each time they are invoked. The following paragraphs specify how the Source Brush shall be determined.
- 4 A Source Brush can be provided as a **brush** object or as an identified set of arguments which, together, provide all of the components of a Source Brush.
- 5 When a rendering and composing operation is invoked with only those arguments that are part of its set of minimum arguments, the **surface** object's Current Brush shall be the Source Brush.
- 6 When a rendering and composing operation is invoked with *extra arguments*, which are additional arguments beyond those that are part of its set of minimum arguments, the Source Brush shall be as follows, with any errors reported as specified in Error reporting (3):
 - (6.1) — Where there is one extra argument and it is a **brush** object, the Source Brush shall be that **brush** object.
 - (6.2) — Where there is one extra argument and it is an **rgba_color** object, the Source Brush shall be the same as if it were a **brush** object constructed from a **solid_color_brush_factory** constructed from the extra argument and sampling from it shall be as specified by 12.5.3 and 12.5.3.1.
 - (6.3) — Where there is more than one extra argument, including default arguments, and the first extra argument is a **surface** object, the Source Brush shall be the same as if it were a **brush** object **b** constructed from a **surface_brush_factory** constructed from the **surface** object extra argument and sampling from it shall be as specified by 12.5.3 and 12.5.3.4, subject to the following adjustments:
 1. The **surface** object extra argument should be used directly; no copy of it should be made. This differs from the **surface_brush_factory** class, which requires that a copy of the surface be made and that only the copy be used.
 2. The Source Brush shall produce the same observable results that **b** would produce after **brush::extend** was called on **b** with the **extend** object extra argument passed to **brush::extend** as an argument.
 3. The Source Brush shall produce the same observable results that **b** would produce after **brush::filter** was called on **b** with the **filter** object extra argument passed to **brush::filter** as an argument.
 4. The Source Brush shall produce the same observable results that **b** would produce after **brush::matrix** was called on **b** with the **matrix** object extra argument passed to **brush::matrix** as an argument.
- 7 [*Note*: The rendering and composing operations each have the same combinations of extra arguments, all of which are covered by the above requirements that determine the Source Brush when extra arguments are present. — *end note*]
- 8 A **surface** object shall not be used as a Mask Brush or as a Source Brush for any rendering and composing operations invoked on that **surface** object; no diagnostic is required. [*Note*: When a **brush** object **b** is created from a **surface** object **s** using the **surface_brush_factory** class, **b** can be used as a Mask Brush or as a Source Brush for rendering and composing operations invoked on **s** because the **surface_brush_factory** class creates a copy of **s** (see 12.9), thereby avoiding the aliasing problems and potential race conditions that would otherwise exist. — *end note*]

13.11.6 Standard coordinate spaces

[**surface.coordinatespaces**]

- 1 There are four standard coordinate spaces relevant to the rendering and composing operations (13.11.4):
 - (1.1) — the Brush Coordinate Space;
 - (1.2) — the Mask Coordinate Space;
 - (1.3) — the User Coordinate Space; and

(1.4) — the Surface Coordinate Space.

- 2 The *Brush Coordinate Space* is the standard coordinate space of the Source Brush (13.11.5). Its transformation matrix is as if it was a copy of the Source Brush's `matrix_2d` object.
- 3 The *Mask Coordinate Space* is the standard coordinate space of the Mask Brush (13.11.10). Its transformation matrix is as if it was a copy of the Mask Brush's `matrix_2d` object.
- 4 The *User Coordinate Space* is the standard coordinate space of `surface` object. Its transformation matrix is the return value of `matrix_2d::init_identity`.
- 5 The *Surface Coordinate Space* is the standard coordinate space of the `surface` object's underlying graphics data graphics resource. Its transformation matrix is the return value of calling `s.matrix()` where `s` is the `surface` object.
- 6 Given a point `pt`, a Brush Coordinate Space transformation matrix `bcs`, a Mask Coordinate Space transformation matrix `mcs`, a User Coordinate Space transformation matrix `ucs`, and a Surface Coordinate Space transformation matrix `s`, the following table describes how to transform it from each of these standard coordinate spaces to the other standard coordinate spaces:

Table 23 — Point transformations

From	To	Transform
Brush Coordinate Space	Mask Coordinate Space	<code>mcs.transform_point(bcs.invert().transform_point(pt)).</code>
Brush Coordinate Space	User Coordinate Space	<code>bcs.invert().transform_point(pt).</code>
Brush Coordinate Space	Surface Coordinate Space	<code>s.transform_point(bcs.invert().transform_point(pt)).</code>
User Coordinate Space	Brush Coordinate Space	<code>bcs.transform_point(pt).</code>
User Coordinate Space	Mask Coordinate Space	<code>mcs.transform_point(pt).</code>
User Coordinate Space	Surface Coordinate Space	<code>s.transform_point(pt).</code>
Surface Coordinate Space	Brush Coordinate Space	<code>bcs.transform_point(s.invert().transform_point(pt)).</code>
Surface Coordinate Space	Mask Coordinate Space	<code>mcs.transform_point(s.invert().transform_point(pt)).</code>
Surface Coordinate Space	User Coordinate Space	<code>s.invert().transform_point(pt).</code>

13.11.7 surface painting

[surface.painting]

- 1 When a Painting operation is initiated on a surface, the implementation shall produce results as if the following steps were performed:
 1. For each integral point *sp* of the underlying graphics data graphics resource, determine if *sp* is within the Clip Area (Table 21); if so, proceed with the remaining steps.
 2. Transform *sp* from the Surface Coordinate Space (13.11.6) to the Brush Coordinate Space (Table 23), resulting in point *bp*.
 3. Sample from point *bp* of the Source Brush (13.11.5), combine the resulting visual data with the visual data at point *sp* in the underlying graphics data graphics resource in the manner specified by the surface's current Composition Operator (Table 21), and modify the visual data of the underlying graphics

data graphics resource at point *sp* to reflect the result produced by application of the Composition Operator.

13.11.8 surface filling

[surface.filling]

- ¹ When a Filling operation is initiated on a surface, the implementation shall produce results as if the following steps were performed:
 1. For each integral point *sp* of the underlying graphics data graphics resource, determine if *sp* is within the Clip Area (Table 21); if so, proceed with the remaining steps.
 2. Transform *sp* from the Surface Coordinate Space (13.11.6) to the User Coordinate Space (Table 23), resulting in point *up*.
 3. Using the Source Path (13.11.4) and the Fill Rule (Table 21), determine whether *up* shall be filled; if so, proceed with the remaining steps.
 4. Transform *up* from the User Coordinate Space to the Brush Coordinate Space (13.11.6 and Table 23), resulting in point *bp*.
 5. Sample from point *bp* of the Source Brush (13.11.5), combine the resulting visual data with the visual data at point *sp* in the underlying graphics data graphics resource in the manner specified by the surface's current Composition Operator (Table 21), and modify the visual data of the underlying graphics data graphics resource at point *sp* to reflect the result produced by application of the Composition Operator.

13.11.9 surface stroking

[surface.stroking]

- ¹ When a Stroking operation is initiated on a surface, the implementation shall carry out the Stroking operation for each path geometry in the Source Path (13.11.4).
- ² The following rules shall apply when a Stroking operation is carried out on a path geometry:
 1. No part of the underlying graphics data graphics resource that is outside of the Clip Area shall be modified.
 2. If the path geometry only contains a degenerate path segment, then if the Line Cap value is either `line_cap::round` or `line_cap::square`, the line caps shall be rendered, resulting in a circle or a square, respectively. The remaining rules shall not apply.
 3. If the path geometry is a closed path geometry, then the point where the end point of its final path segment meets the start point of the initial path segment shall be rendered as specified by the Line Join value; otherwise the start point of the initial path segment and end point of the final path segment shall each be rendered as specified by the Line Cap value. The remaining meetings between successive end points and start points shall be rendered as specified by the Line Join value.
 4. If the Dash Pattern has its default value or if its `vector<double>` member is empty, the path segments shall be rendered as a continuous path.
 5. If the Dash Pattern's `vector<double>` member contains only one value, that value shall be used to define a repeating pattern in which the path is shown then hidden. The ends of each shown portion of the path shall be rendered as specified by the Line Cap value.
 6. If the Dash Pattern's `vector<double>` member contains two or more values, the values shall be used to define a pattern in which the path is alternatively rendered then not rendered for the length specified by the value. The ends of each rendered portion of the path shall be rendered as specified by the Line Cap value. If the Dash Pattern's `double` member, which specifies an offset value, is not 0.0,

the meaning of its value is implementation-defined. If a rendered portion of the path overlaps a not rendered portion of the path, the rendered portion shall be rendered.

- ³ When a Stroking operation is carried out on a path geometry, the width of each rendered portion shall be the Line Width. Ideally this means that the diameter of the stroke at each rendered point should be equal to the Line Width. However, because there is an infinite number of points along each rendered portion, implementations may choose an unspecified method of determining minimum distances between points along each rendered portion and the diameter of the stroke between those points shall be the same. [*Note*: This concept is sometimes referred to as a tolerance. It allows for a balance between precision and performance, especially in situations where the end result is in a non-exact format such as raster graphics data. — *end note*]
- ⁴ After all path geometries have been rendered but before the rendered result is composed to the underlying graphics data graphics resource, the rendered result shall be transformed from the User Coordinate Space (13.11.6) to the Surface Coordinate Space (13.11.6). [*Example*: If an open path geometry consisting solely of a vertical line from `vector_2d(20.0, 20.0)` to `vector_2d(20.0, 60.0)` is to be composed to the underlying graphics data graphics resource, the Line Cap is `line_cap::butt`, the Line Width is 12.0, and the Transformation Matrix is `matrix_2d::init_scale(0.5, 1.0)`, then the line will end up being composed within the area `rectangle( { 7.0, 20.0 }, { 13.0, 60.0 } )` on the underlying graphics data graphics resource. The Transformation Matrix causes the center of the *x* axis of the line to move from 20.0 to 10.0 and then causes the horizontal width of the line to be reduced from 12.0 to 6.0. — *end example*]

13.11.10 surface masking

[surface.masking]

- ¹ The Masking operation uses a *Mask Brush* as described in 13.11.4.
- ² When a Masking operation is initiated on a surface, the implementation shall produce results as if the following steps were performed:
 1. For each integral point *sp* of the underlying graphics data graphics resource, determine if *sp* is within the Clip Area (Table 21); if so, proceed with the remaining steps.
 2. Transform *sp* from the Surface Coordinate Space (13.11.6) to the Mask Coordinate Space (Table 23), resulting in point *mp*.
 3. Sample the alpha channel from point *mp* of the Mask Brush and store the result in *mac*; if the visual data format of the Mask Brush does not have an alpha channel, the value of *mac* shall always be 1.0.
 4. Transform *sp* from the Surface Coordinate Space to the Brush Coordinate Space, resulting in point *bp*.
 5. Sample from point *bp* of the Source Brush (13.11.5), combine the resulting visual data with the visual data at point *sp* in the underlying graphics data graphics resource in the manner specified by the surface's current Composition Operator (Table 21), multiply each channel of the result produced by application of the Composition Operator by *map* if the visual data format of the underlying graphics data graphics resource is a premultiplied format and if not then just multiply the alpha channel of the result by *map*, and modify the visual data of the underlying graphics data graphics resource at point *sp* to reflect the multiplied result.

13.11.11 surface typesetting

[surface.typesetting]

- ¹ This section is forthcoming in a future revision.

13.11.12 surface constructors, assignment operators, and destructors [surface.cons]

```
virtual ~surface();
```

- ¹ *Effects*: Destroys an object of class `surface`.

13.11.13 surface state modifiers**[surface.modifiers.state]**

```
virtual void finish() noexcept;
```

1 *Effects:* Releases all resources managed by the **surface** object **s** if **!s.is_finished()**, otherwise does nothing.

2 *Postconditions:* **s.is_finished()** shall return **true**.

3 *Remarks:* Once this function has been called, the surface is *finished*. The only valid operations on a finished surface are destruction, calling **finish()**, and calling **is_finished()**. Except as otherwise noted, any other operation on a finished surface or any attempt to use a finished **surface** or **surface-**derived object as an argument to a function produces undefined behavior; no diagnostic is required.

```
void flush();
```

```
void flush(error_code& ec) noexcept;
```

4 *Effects:* If the implementation does not provide a native handle to the **surface** object's underlying graphics data graphics resource, this function shall do nothing.

5 If the implementation does provide a native handle to the **surface** object's underlying graphics data graphics resource, then:

(5.1) — Any pending rendering and composing operations (13.11.4) shall be performed on the **surface** object. [*Note:* As long as the observable effect is the same as if they were performed immediately, it does not matter whether they are executed, batched, or otherwise committed to the underlying rendering and presentation technologies. — *end note*]

(5.2) — The implementation may then modify the **surface** object's observable state (13.11.3).

(5.3) — The **surface** object's observable state shall then be undefined.

6 *Throws:* As specified in Error reporting (3).

7 *Remarks:* This function exists to allow the user to take control of the underlying surface using an implementation-provided native handle without introducing a race condition. The implementation's responsibility is to ensure that the user can safely use the underlying surface.

8 *Error conditions:* The potential errors are implementation-defined.

9 Implementations should avoid producing errors here.

10 If the implementation does not provide a native handle to the **surface** object's underlying graphics data graphics resource, this function shall not produce any errors.

11 *Notes:* Because the **surface** object's observable state can change as a result of calling this function, users will typically want to call **surface::save** before calling this function. When the user is done using the **surface** object's native handle, he or she must call **surface::mark_dirty** if changes were made to the underlying graphics data graphics resource using the native handle as per that function's semantics. Once that is done, or immediately after access using the native handle is finished if no changes were made, the user needs to then call **surface::restore** (assuming a previous, valid call to **surface::save**). Otherwise the user must set every item of observable state before using the **surface** object in any other way.

```
shared_ptr<experimental::io2d::device> device();
```

```
shared_ptr<experimental::io2d::device> device(error_code& ec) noexcept;
```

12 *Returns:* A shared pointer to the **device** object for this **surface**. If a **device** object does not already exist for this **surface**, a shared **device** object shall be allocated and returned.

13 *Throws:* As specified in Error reporting (3).

14 *Error conditions:* **errc::not_enough_memory** if a **device** object needs to be created and not enough memory exists to do so.

```

void mark_dirty();
void mark_dirty(error_code& ec) noexcept;
void mark_dirty(const rectangle& extents);
void mark_dirty(const rectangle& extents, error_code& ec) noexcept;

```

15 *Effects:* If the implementation does not provide a native handle to the **surface** object's underlying graphics data graphics resource, this function shall do nothing.

16 If the implementation does provide a native handle to the **surface** object's underlying graphics data graphics resource, then:

(16.1) — If called without a **rect** argument, informs the implementation that external changes using a native handle were potentially made to the entire underlying graphics data graphics resource.

(16.2) — If called with a **rect** argument, informs the implementation that external changes using a native handle were potentially made to the underlying graphics data graphics resource within the bounds specified by the *bounding rectangle* **rectangle**{ **round**(**extents.x()**), **round**(**extents.y()**), **round**(**extents.width()**), **round**(**extents.height()**)}. No part of the bounding rectangle shall be outside of the bounds of the underlying graphics data graphics resource; no diagnostic is required.

17 *Throws:* As specified in Error reporting (3).

18 *Remarks:* After external changes are made to this **surface** object's underlying graphics data graphics resource using a native pointer, this function shall be called before using this **surface** object; no diagnostic is required.

19 No call to this function shall be required solely as a result of changes made to a surface using the functionality provided by **surface::map**. [*Note:* The **mapped_surface** type, which is used by **surface::map**, provides its own functionality for managing any such changes. — *end note*]

20 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

21 If the implementation does not provide a native handle to the **surface** object's underlying graphics data graphics resource, this function shall not produce any errors.

```

void map(const function<void(mapped_surface&)>& action);
void map(const function<void(mapped_surface&, error_code&)>& action, error_code& ec);
void map(const function<void(mapped_surface&)>& action, const rectangle& extents);
void map(const function<void(mapped_surface&, error_code&)>& action,
    const rectangle& extents, error_code& ec);

```

22 *Effects:* Creates a **mapped_surface** object and calls **action** using it.

23 The **mapped_surface** object is created using ***this**, which allows direct manipulation of the underlying graphics data graphics resource.

24 If called with a **const rectangle& extents** argument, the **mapped_surface** object shall only allow manipulation of the portion of ***this** specified by the *bounding rectangle* **rectangle**{ **round**(**extents.x()**), **round**(**extents.y()**), **round**(**extents.width()**), **round**(**extents.height()**)}. If any part of the bounding rectangle is outside of the bounds of ***this**, the call shall result in undefined behavior; no diagnostic is required.

25 *Throws:* As specified in Error reporting (3).

26 *Remarks:* Whether changes are committed to the underlying graphics data graphics resource immediately or only when the **mapped_surface** object is destroyed is unspecified.

27 Calling this function on a **surface** object and then calling any function on the **surface** object or using the **surface** object before the call to this function has returned shall result in undefined behavior; no diagnostic is required.

28 *Error conditions:* The errors, if any, produced by this function are implementation-defined or are produced by the user-provided function passed via the **action** argument.

```
virtual void save();
virtual void save(error_code& ec) noexcept;
```

29 *Effects:* Saves the state specified in 13.11.3.2 as if to an internal stack of saved states.

30 *Throws:* As specified in Error reporting (3).

31 *Error conditions:* **errc::not_enough_memory** if the state cannot be saved.

```
virtual void restore();
virtual void restore(error_code& ec) noexcept;
```

32 *Effects:* Restores the state of ***this** to the values saved by the most recent call to **surface::save**.

33 *Throws:* As specified in Error reporting (3).

Remarks: Because this function is only restoring previously saved state, except where the conditions for **io2d_error::invalid_restore** are met, implementations should not generate errors.

35 *Error conditions:* **io2d_error::invalid_restore** if this function is called without a previous matching call to **surface::save**. Implementations shall not produce **io2d_error::invalid_restore** except under the conditions stated in this paragraph.

36 Excluding the previously specified error, any errors produced by calling this function are implementation-defined.

```
void brush(nullopt_t) noexcept;
```

37 *Effects:* Sets the Current Brush (Table 21) to be a **brush** object that is the same as if it was the brush that is the default value of Current Brush 13.11.3.1.

38 [*Note:* This function does not require that the Current Brush be set to the same **brush** object that was the original default value Current Brush **brush** object. That said, because this function is **noexcept**, implementers can (and likely should) keep a reference to the original, default value Current Brush **brush** object and use it when this function is called. Allocating a new **brush** object may result in errors due to memory constraints or other considerations. — *end note*]

```
void brush(const experimental::io2d::brush& source);
void brush(const experimental::io2d::brush& source,
    error_code& ec) noexcept;
```

39 *Effects:* Sets **source** as the Current Brush (Table 21), ensuring that it shall not be destroyed and shall be recreated, if necessary, by the underlying rendering and presentation technologies so that it shall be available for use at least until such time as it is no longer the Current Brush of any **surface** or **surface**-derived object.

40 *Throws:* As specified in Error reporting (3).

41 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void antialias(experimental::io2d::antialias a) noexcept;
```

42 *Effects:* Sets General Antialiasing (Table 21) to the value of **a**.

```
void dashes(nullopt_t) noexcept;
```

43 *Effects:* Sets the Dash Pattern (Table 21) to be the default value of Dash Pattern.

```
void dashes(const experimental::io2d::dashes& d);
void dashes(const experimental::io2d::dashes& d, error_code& ec) noexcept;
```

44 *Effects:* Sets the Dash Pattern (Table 21) to the value of `d`.

45 *Throws:* As specified in Error reporting (3).

46 *Error conditions:* `errc::not_enough_memory` if there is a problem caching the new Dash Pattern.

47 `io2d_error::invalid_dash` if the new Dash Pattern contains a negative value or if it has values and all of them are 0.0.

`void fill_rule(experimental::io2d::fill_rule fr) noexcept;`

48 *Effects:* Sets the Fill Rule (Table 21) to the value of `fr`.

`void line_cap(experimental::io2d::line_cap lc) noexcept;`

49 *Effects:* Sets the Line Cap (Table 21) to the value of `lc`.

`void line_join(experimental::io2d::line_join lj) noexcept;`

50 *Effects:* Sets the Line Join (Table 21) to the value of `lj`.

`void line_width(double width) noexcept;`

51 *Effects:* Sets the Line Width (Table 21) to `max(width, 0.0)`.

`void miter_limit(double limit) noexcept;`

52 *Effects:* Sets the Miter Limit (Table 21) to the value of `limit` clamped to be within the range defined by the implementation-defined minimum value and the implementation-defined maximum value, inclusive.

53 *Remarks:* The *implementation-defined* minimum value shall not be greater than 2.0 and the *implementation-defined* maximum value shall not be less than 10.0.

54 *Notes:* The Miter Limit only applies when the Line Join is set to `line_join::miter`. — *end note*]

55 *Remarks:* The Miter Limit works as follow. The length of the miter is divided by the width of the line. If the resulting value is greater than the Miter Limit, the join will be beveled, otherwise it will be mitered.

`void compositing_operator(experimental::io2d::compositing_operator co) noexcept;`

56 *Effects:* Sets the Composition Operator to the value of `co`.

`void clip(const experimental::io2d::path& p);`
 `void clip(const experimental::io2d::path& p, error_code& ec) noexcept;`

57 *Effects:* Sets the Clip Area (Table 21) to be the intersection of the Clip Area and `p` where `p`'s area is determined in the same way as if `p` were filled according to the Fill Rule.

58 *Notes:* The Clip Area never increases as a result of this function.

`void clip_immediate();`
 `void clip_immediate(error_code& ec) noexcept;`

59 *Effects:* Sets the Clip Area (Table 21) to be the intersection of the Clip Area and the Immediate Path (Table 21) where the Immediate Path's area is determined in the same way as if the Immediate Path were filled according to the Fill Rule.

60 *Notes:* The Clip Area never increases as a result of this function.

`void path(nullopt_t) noexcept;`

61 *Effects:* Set's the Current Path (Table 21).

62 *Notes:* The empty `path` object is not supplied by the user. The implementation is expected to provide an empty `path` object. Since a `path` object is immutable, an implementation should create an empty `path` object in advance for maximum efficiency.

```
void path(const experimental::io2d::path& p);
void path(const experimental::io2d::path& p, error_code& ec) noexcept;
```

63 *Effects:* Set's Current Path (Table 21) to `p`.

64 *Remarks:* Processing the path data so that it is properly transformed can be done at the time it is first set as a path on a `surface` object or any time before that. The untransformed `vector<path_data_item>` shall be retained to ensure that a path can be properly recreated at any time. The steps for converting an untransformed `vector<path_data_item>` to transformed path data are found at 10.1.2.

13.11.14 surface immediate path modifiers [surface.modifiers.immediatepath]

```
experimental::io2d::path_factory& immediate() noexcept;
```

1 *Returns:* A reference to the Immediate Path (Table 21).

13.11.15 surface render modifiers [surface.modifiers.render]

```
void fill();
void fill(error_code& ec) noexcept;
void fill(const rgba_color& c);
void fill(const rgba_color& c, error_code& ec) noexcept;
void fill(const experimental::io2d::brush& b);
void fill(const experimental::io2d::brush& b, error_code& ec) noexcept;
void fill(const surface& s, const matrix_2d& m = matrix_2d::init_identity(),
    extend e = extend::none, filter f = filter::good);
void fill(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
```

1 *Effects:* Performs the Filling rendering and composing operation as specified by 13.11.8.

2 The meanings of the parameters are specified by 13.11.4.

3 *Throws:* As specified in Error reporting (3).

4 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void fill_immediate();
void fill_immediate(error_code& ec) noexcept;
void fill_immediate(const rgba_color& c);
void fill_immediate(const rgba_color& c, error_code& ec) noexcept;
void fill_immediate(const experimental::io2d::brush& b);
void fill_immediate(const experimental::io2d::brush& b, error_code& ec)
    noexcept;
void fill_immediate(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(),
    extend e = extend::none, filter f = filter::good);
void fill_immediate(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(),
    extend e = extend::none, filter f = filter::good) noexcept;
```

5 *Effects:* Performs the Filling rendering and composing operation as specified by 13.11.8.

6 The meanings of the parameters are specified by 13.11.4.

- 7 *Throws:* As specified in Error reporting (3).
- 8 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```

void paint();
void paint(error_code& ec) noexcept;
void paint(const rgba_color& c);
void paint(const rgba_color& c, error_code& ec) noexcept;
void paint(const experimental::io2d::brush& b);
void paint(const experimental::io2d::brush& b, error_code& ec) noexcept;
void paint(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void paint(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
void paint(double alpha);
void paint(double alpha, error_code& ec) noexcept;
void paint(const rgba_color& c, double alpha);
void paint(const rgba_color& c, double alpha, error_code& ec) noexcept;
void paint(const experimental::io2d::brush& b, double alpha);
void paint(const experimental::io2d::brush& b, double alpha,
    error_code& ec) noexcept;
void paint(const surface& s, double alpha,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void paint(const surface& s, double alpha, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;

```

- 9 *Effects:* Performs the Painting rendering and composing operation as specified by 13.11.7.
- 10 The meanings of the parameters are specified by 13.11.4.
- 11 *Throws:* As specified in Error reporting (3).
- 12 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```

void stroke();
void stroke(error_code& ec) noexcept;
void stroke(const rgba_color& c);
void stroke(const rgba_color& c, error_code& ec) noexcept;
void stroke(const experimental::io2d::brush& b);
void stroke(const experimental::io2d::brush& b, error_code& ec) noexcept;
void stroke(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void stroke(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;

```

- 13 *Effects:* Performs the Stroking rendering and composing operation as specified by 13.11.9.
- 14 The meanings of the parameters are specified by 13.11.4.
- 15 *Throws:* As specified in Error reporting (3).
- 16 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```

void stroke_immediate();
void stroke_immediate(error_code& ec) noexcept;
void stroke_immediate(const rgba_color& c);
void stroke_immediate(const rgba_color& c, error_code& ec) noexcept;
void stroke_immediate(const experimental::io2d::brush& b);
void stroke_immediate(const experimental::io2d::brush& b, error_code& ec)
    noexcept;
void stroke_immediate(const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void stroke_immediate(const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;

```

- 17 *Effects:* Performs the Stroking rendering and composing operation as specified by 13.11.9.
- 18 The meanings of the parameters are specified by 13.11.4.
- 19 *Throws:* As specified in Error reporting (3).
- 20 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

13.11.16 surface mask render modifiers

[surface.modifiers.maskrender]

```

void mask(const experimental::io2d::brush& mb);
void mask(const experimental::io2d::brush& mb, error_code& ec)
    noexcept;
void mask(const experimental::io2d::brush& mb, const rgba_color& c);
void mask(const experimental::io2d::brush& mb, const rgba_color& c,
    error_code& ec) noexcept;
void mask(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b);
void mask(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b, error_code& ec) noexcept;
void mask(const experimental::io2d::brush& mb, const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void mask(const experimental::io2d::brush& mb, const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;
void mask(surface& ms,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask(surface& ms, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask(surface& ms, const rgba_color& c,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask(surface& ms, const rgba_color& c, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask(surface& ms, const experimental::io2d::brush& b,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask(surface& ms, const experimental::io2d::brush& b, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;

```



```

void mask(surface& ms, const surface& s,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good);
void mask(surface& ms, const surface& s, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good) noexcept;

```

- 1 *Effects:* Performs the Masking rendering and composing operation as specified by [13.11.10](#).
- 2 The meanings of the parameters are specified by [13.11.4](#).
- 3 *Throws:* As specified in Error reporting (3).
- 4 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```

void mask_immediate(const experimental::io2d::brush& mb);
void mask_immediate(const experimental::io2d::brush& mb,
    error_code& ec) noexcept;
void mask_immediate(const experimental::io2d::brush& mb,
    const rgba_color& c);
void mask_immediate(const experimental::io2d::brush& mb,
    const rgba_color& c, error_code& ec) noexcept;
void mask_immediate(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b);
void mask_immediate(const experimental::io2d::brush& mb,
    const experimental::io2d::brush& b, error_code& ec) noexcept;
void mask_immediate(const experimental::io2d::brush& mb, const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good);
void mask_immediate(const experimental::io2d::brush& mb, const surface& s,
    const matrix_2d& m = matrix_2d::init_identity(), error_code& ec,
    extend e = extend::none, filter f = filter::good) noexcept;
void mask_immediate(surface& ms,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask_immediate(surface& ms, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask_immediate(surface& ms, const rgba_color& c,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask_immediate(surface& ms, const rgba_color& c, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask_immediate(surface& ms, const experimental::io2d::brush& b,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good);
void mask_immediate(surface& ms, const experimental::io2d::brush& b,
    error_code& ec, const matrix_2d& msMatrix = matrix_2d::init_identity(),
    extend msExtend = extend::none, filter msFilter = filter::good) noexcept;
void mask_immediate(surface& ms, const surface& s,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,

```



```

    filter msFilter = filter::good, filter f = filter::good);
void mask_immediate(surface& ms, const surface& s, error_code& ec,
    const matrix_2d& msMatrix = matrix_2d::init_identity(),
    const matrix_2d& m = matrix_2d::init_identity(),
    extend msExtend = extend::none, extend e = extend::none,
    filter msFilter = filter::good, filter f = filter::good) noexcept;

```

- 5 *Effects:* Performs the Masking rendering and composing operation as specified by [13.11.10](#).
- 6 The meanings of the parameters are specified by [13.11.4](#).
- 7 *Throws:* As specified in Error reporting (3).
- 8 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

13.11.17 surface text render modifiers [surface.modifiers.textrender]

```

vector_2d render_text(const string& utf8, const vector_2d& pos);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    error_code& ec) noexcept;
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const rgba_color& c);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const rgba_color& c, error_code& ec) noexcept;
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const experimental::io2d::brush& b);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const experimental::io2d::brush& b, error_code& ec) noexcept;
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const surface& s, const matrix_2d& m = matrix_2d::init_identity(),
    extend e = extend::none, filter f = filter::good);
vector_2d render_text(const string& utf8, const vector_2d& pos,
    const surface& s, error_code& ec,
    const matrix_2d& m = matrix_2d::init_identity(), extend e = extend::none,
    filter f = filter::good) noexcept;

```

- 1 *Effects:* Performs the Typesetting rendering and composing operation as specified by [13.11.11](#).
- 2 The meanings of the parameters are specified by [13.11.4](#).
- 3 *Returns:* A `vector_2d` object that provides the origin point for the next character of text that would have been drawn if `utf8` had possessed an additional character of text.
- 4 *Throws:* As specified in Error reporting (3).
- 5 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

13.11.18 surface transformation modifiers [surface.modifiers.transform]

```

void matrix(const matrix_2d& m);
void matrix(const matrix_2d& m, error_code& ec) noexcept;

```

- 1 *Effects:* Sets Transformation Matrix (Table 21) to the value of `m`.
- 2 *Throws:* As specified in Error reporting (3).
- 3 *Error conditions:* `io2d_error::invalid_matrix` if calling `!m.is_invertible()`.

13.11.19 surface font modifiers [surface.modifiers.font]

```

void font_face(const string& typeface, font_slant sl, font_weight w);
void font_face(const string& typeface, font_slant sl, font_weight w,
    error_code& ec) noexcept;

```

1 *Effects:* Sets Font Face (Table 21) to the result of constructing a `simple_font_face` object using the arguments to this function.

2 If an error occurs in constructing the `simple_font_face` object, Font Face shall retain the value it had prior to the execution of this function and that error shall be propagated back.

3 *Throws:* As specified in Error reporting (3).

4 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void font_face(const experimental::io2d::font_face& f);
void font_face(const experimental::io2d::font_face& f, error_code& ec)
    noexcept;
```

5 *Effects:* Sets Font Face (Table 21) to `f`.

6 *Throws:* As specified in Error reporting (3).

7 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void font_size(double s);
void font_size(double s, error_code& ec) noexcept;
```

8 *Effects:* Sets Font Matrix (Table 21) to the value of `matrix_2d::init_scale{ { s, s } }`.

9 *Throws:* As specified in Error reporting (3).

10 *Error conditions:* `errc::invalid_argument` if `s <= 0.0`.

```
void font_matrix(const matrix_2d& m);
void font_matrix(const matrix_2d& m, error_code& ec) noexcept;
```

11 *Effects:* Sets Font Matrix (Table 21) to the value of `m`.

12 *Throws:* As specified in Error reporting (3).

13 *Error conditions:* `io2d_error::invalid_matrix` if calling `!m.is_invertible()`.

```
void font_options(const font_options& fo) noexcept;
```

14 *Effects:* Sets Font Options (Table 21) to `fo`.

13.11.20 surface state observers

[surface.observers.state]

```
bool is_finished() const noexcept;
```

1 *Returns:* If a call to `surface::finished` has previously been made, returns `true`; otherwise returns `false`.

```
experimental::io2d::content content() const noexcept;
```

2 *Returns:* The `content` enumerator (13.2.3) that appropriately describes the underlying graphics data graphics resource.

```
experimental::io2d::brush brush() const noexcept;
```

3 *Returns:* The Current Brush (Table 21).

```
experimental::io2d::antialias antialias() const noexcept;
```

4 *Returns:* The value of General Antialiasing (Table 21).

```
experimental::io2d::dashes dashes() const;
experimental::io2d::dashes dashes(error_code& ec) const noexcept;
```

5 *Returns:* The Dash Pattern (Table 21).

6 *Throws:* As specified in Error reporting (3).

7 *Remarks:* See 13.11.9 for more information about the `dashes` type.

8 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.

`experimental::io2d::fill_rule fill_rule() const noexcept;`

9 *Returns:* The value of Fill Rule (Table 21).

`experimental::io2d::line_cap line_cap() const noexcept;`

10 *Returns:* The value of Line Cap (Table 21).

`experimental::io2d::line_join line_join() const noexcept;`

11 *Returns:* The value of Line Join (Table 21).

`double line_width() const noexcept;`

12 *Returns:* The value of Line Width (Table 21).

`double miter_limit() const noexcept;`

13 *Returns:* The value of Miter Limit (Table 21).

`experimental::io2d::compositing_operator compositing_operator() const
noexcept;`

14 *Returns:* The value of Composition Operator (Table 21).

`rectangle clip_extents() const noexcept;`

15 *Returns:* A `rectangle` that specifies the smallest bounding box which contains the Clip Area (Table 21).

`bool in_clip(const vector_2d& pt) const noexcept;`

16 *Returns:* If the point `pt` is outside of the Clip Area (Table 21), returns `false`; otherwise returns `true`.

`vector<rectangle> clip_rectangles() const;`
`vector<rectangle> clip_rectangles(error_code& ec) const noexcept;`

17 *Returns:* A `vector<rectangle>` object which contains the rectangles which make up the Clip Area (Table 21).

18 If an `error_code&` argument is passed and the error `io2d_error::clip_not_representable` occurs, returns an empty `vector<rectangle>` object.

19 *Throws:* As specified in Error reporting (3).

20 *Error conditions:* `io2d_error::clip_not_representable` if the Clip Area contains one or more areas that cannot be represented using rectangles. [*Example:* This error would occur if the Clip Area contains arcs or curves. — *end example*]

21 `errc::not_enough_memory` if a failure to allocate memory occurs.

13.11.21 surface render observers

[surface.observers.render]

```
rectangle fill_extents() const noexcept;
```

1 *Returns:* A **rectangle** that specifies the smallest bounding box which contains all areas in which a Filling operation (13.11.8) can have an effect using the Current Path (Table 21) and the value of Fill Rule. The Clip Area and the bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.

2 For purposes of calculating the return value, the coordinates of the Current Path shall be transformed using **surface::user_to_surface** before any other calculations are performed. When the final values for the return value are calculated, they shall be transformed using **surface::surface_to_user** and the **rectangle** that is returned shall be composed of those transformed values.

3 *Notes:* The resulting bounding box is not the same as the area that would be changed if **surface::fill** was called since it is calculated without regard to the Current Brush, the Composition Operator, and the Clip Area and thus could contain areas that would not actually be affected by a call to **surface::fill**.

```
rectangle fill_extents_immediate() const;
```

```
rectangle fill_extents_immediate(error_code& ec) const noexcept;
```

4 *Returns:* A **rectangle** that specifies the smallest bounding box which contains all areas in which a Filling operation (13.11.8) can have an effect using the Immediate Path (Table 21) and the value of Fill Rule. The Clip Area and the bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.

5 For purposes of calculating the return value, the processed coordinates of the Immediate Path shall be transformed using **surface::user_to_surface** before any other calculations are performed. When the final values for the return value are calculated, they shall be transformed using **surface::surface_to_user** and the **rectangle** that is returned shall be composed of those transformed values.

6 *Throws:* As specified in Error reporting (3).

7 *Remarks:* This function requires that the Immediate Path be processed into a usable form as if in the manner specified in 10.1.2.

8 *Error conditions:* **io2d_error::no_current_point** if, when processing the Immediate Path's path geometries, an operation was encountered which required a current point and the current path geometry had no current point.

9 **io2d_error::invalid_matrix** if, when processing the Immediate Path's path geometries, an operation was encountered which required the current transformation matrix to be invertible and the matrix was not invertible.

10 Other errors, if any, produced by this function are implementation-defined.

11 *Notes:* The resulting bounding box is not the same as the area that would be changed if **surface::fill_immediate** was called since it is calculated without regard to the Current Brush, the Composition Operator, and the Clip Area and thus could contain areas that would not actually be affected by a call to **surface::fill_immediate**.

```
bool in_fill(const vector_2d& pt) const noexcept;
```

12 *Returns:* If the point **pt** is not within any of the areas in which a Filling operation (13.11.8) can have an effect using the Current Path (Table 21) and the value of Fill Rule, this function returns **false**; otherwise it returns **true**. The Clip Area and bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.

13 For purposes of calculating the return value, the coordinates of the Current Path shall be transformed using **surface::user_to_surface** before any other calculations are performed. The value of **pt** shall

also be transformed using `surface::user_to_surface` prior to determining whether or not it is within any of the Filling operation areas.

14 *Notes:* The result does not mean that the content at the point `pt` would be changed if `surface::fill` was called since the areas are calculated without regard to the Current Brush, the Composition Operator, the Clip Area, and the Transformation Matrix and thus could contain areas that would not actually be affected by a call to `surface::fill`.

```
bool in_fill_immediate(const vector_2d& pt) const;
bool in_fill_immediate(const vector_2d& pt, error_code& ec) const noexcept;
```

15 *Returns:* If the point `pt` is not within any of the areas in which a Filling operation (13.11.8) can have an effect using the Immediate Path (Table 21) and the value of Fill Rule, this function returns `false`; otherwise it returns `true`. The Transformation Matrix, Clip Area, and the bound of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.

16 For purposes of calculating the return value, the processed coordinates of the Immediate Path shall be transformed using `surface::user_to_surface` before any other calculations are performed. The value of `pt` shall also be transformed using `surface::user_to_surface` prior to determining whether or not it is within any of the Filling operation areas.

17 *Throws:* As specified in Error reporting (3).

18 *Remarks:* This function requires that the Immediate Path be processed into a usable form as if in the manner specified in 10.1.2.

19 *Error conditions:* `io2d_error::no_current_point` if, when processing the Immediate Path's path geometries, an operation was encountered which required a current point and the current path geometry had no current point.

20 `io2d_error::invalid_matrix` if, when processing the Immediate Path's path geometries, an operation was encountered which required the current transformation matrix to be invertible and the matrix was not invertible.

21 Other errors, if any, produced by this function are implementation-defined.

22 *Notes:* The result does not mean that the point `pt` would be changed if `surface::fill` was called since the areas are calculated without regard to the Current Brush, the Composition Operator, the Clip Area, and the Transformation Matrix and thus could contain areas that would not actually be affected by a call to `surface::fill`.

```
rectangle stroke_extents() const noexcept;
```

23 *Returns:* A `rectangle` that specifies the smallest bounding box which contains all areas in which a Stroking operation (13.11.9) can have an effect using the Current Path (Table 21), the Line Cap, the Line Join, the Line Width, the Miter Limit, and the Dash Pattern. The Clip Area and the bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.

24 For purposes of calculating the return value, the coordinates of the Current Path shall be transformed using `surface::user_to_surface` before any other calculations are performed. When the final values for the return value are calculated, they shall be transformed using `surface::surface_to_user` and the `rectangle` that is returned shall be composed of those transformed values.

25 *Notes:* The resulting bounding box is not the same as the area that would be changed if `surface::stroke` was called since it is calculated without regard to the Current Brush, the Composition Operator, and the Clip Area and thus could contain areas that would not actually be affected by a call to `surface::stroke`.

```
rectangle stroke_extents_immediate() const;
rectangle stroke_extents_immediate(error_code& ec) const noexcept;
```

- 26 *Returns:* A **rectangle** that specifies the smallest bounding box which contains all areas in which a Stroking operation (13.11.9) can have an effect using the Current Path (Table 21), the Line Cap, the Line Join, the Line Width, the Miter Limit, and the Dash Pattern. The Clip Area and the bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.
- 27 For purposes of calculating the return value, the processed coordinates of the Immediate Path shall be transformed using **surface::user_to_surface** before any other calculations are performed. When the final values for the return value are calculated, they shall be transformed using **surface::surface_to_user** and the **rectangle** that is returned shall be composed of those transformed values.
- 28 *Throws:* As specified in Error reporting (3).
- 29 *Remarks:* This function requires that the Immediate Path be processed into a usable form as if in the manner specified in 10.1.2.
- 30 *Error conditions:* **io2d_error::no_current_point** if, when processing the Immediate Path's path geometries, an operation was encountered which required a current point and the current path geometry had no current point.
- 31 **io2d_error::invalid_matrix** if, when processing the Immediate Path's path geometries, an operation was encountered which required the current transformation matrix to be invertible and the matrix was not invertible.
- 32 Other errors, if any, produced by this function are implementation-defined.
- 33 *Notes:* The resulting bounding box is not the same as the area that would be changed if **surface::stroke_immediate** was called since it is calculated without regard to the Current Brush, the Composition Operator, and the Clip Area and thus could contain areas that would not actually be affected by a call to **surface::stroke_immediate**.

```
bool in_stroke(const vector_2d& pt) const noexcept;
```

- 34 *Returns:* If the point **pt** is not within any of the areas in which a Stroking operation (13.11.9) can have an effect using the Current Path (Table 21), the Line Cap, the Line Join, the Line Width, the Miter Limit, and the Dash Pattern, this function returns **false**; otherwise it returns **true**. The Clip Area and bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.
- 35 For purposes of calculating the return value, the coordinates of the Current Path shall be transformed using **surface::user_to_surface** before any other calculations are performed. The value of **pt** shall also be transformed using **surface::user_to_surface** prior to determining whether or not it is within any of the Stroking operation areas.
- 36 *Notes:* The result does not mean that the content at the point **pt** would be changed if **surface::stroke** was called since the areas are calculated without regard to the Current Brush, the Composition Operator, and the Clip Area and thus could contain areas that would not actually be affected by a call to **surface::stroke**.

```
bool in_stroke_immediate(const vector_2d& pt) const;
bool in_stroke_immediate(const vector_2d& pt, error_code& ec) const noexcept;
```

- 37 *Returns:* If the point **pt** is not within any of the areas in which a Stroking operation (13.11.9) can have an effect using the Immediate Path (Table 21), the Line Cap, the Line Join, the Line Width, the Miter Limit, and the Dash Pattern, this function returns **false**; otherwise it returns **true**. The Clip

Area and bounds of the underlying graphics data graphics resource shall be disregarded for purposes of calculating the return value.

38 For purposes of calculating the return value, the processed coordinates of the Immediate Path shall be transformed using `surface::user_to_surface` before any other calculations are performed. The value of `pt` shall also be transformed using `surface::user_to_surface` prior to determining whether or not it is within any of the Stroking operation areas.

39 *Throws:* As specified in Error reporting (3).

40 *Remarks:* This function requires that the Immediate Path be processed into a usable form as if in the manner specified in 10.1.2.

41 *Error conditions:* `io2d_error::no_current_point` if, when processing the Immediate Path's path geometries, an operation was encountered which required a current point and the current path geometry had no current point.

42 `io2d_error::invalid_matrix` if, when processing the Immediate Path's path geometries, an operation was encountered which required the current transformation matrix to be invertible and the matrix was not invertible.

43 Other errors, if any, produced by this function are implementation-defined.

44 *Notes:* The result does not mean that the content at the point `pt` would be changed if `surface::stroke_immediate` was called since the areas are calculated without regard to the Current Brush, the Composition Operator, and the Clip Area and thus could contain areas that would not actually be affected by a call to `surface::stroke_immediate`.

```
experimental::io2d::font_extents font_extents() const noexcept;
```

45 *Returns:* A `font_extents` object which specifies metrics for Font Face (Table 21). See the specification of the `font_extents` class (11.7) for more information.

46 The Transformation Matrix is disregarded for purposes of calculating the metrics.

```
experimental::io2d::text_extents text_extents(const string& utf8) const;
experimental::io2d::text_extents text_extents(const string& utf8,
error_code& ec) const noexcept;
```

47 *Returns:* A `text_extents` object which specifies metrics for `utf8` if it was rendered using Font Face (Table 21). See the specification of the `text_extents` class (11.8) for more information.

48 The Transformation Matrix is disregarded for purposes of calculating the metrics.

49 *Throws:* As specified in Error reporting (3).

50 *Error conditions:* `errc::invalid_argument` if `utf8` is not a valid UTF-8 string.

51 Other errors, if any, produced by this function are implementation-defined.

13.11.22 surface transformation observers [surface.observers.transform]

```
matrix_2d matrix() const noexcept;
```

1 *Returns:* The Transformation Matrix (Table 21).

```
vector_2d user_to_surface(const vector_2d& pt) const noexcept;
```

2 *Returns:* The result of calling `matrix_2d::transform_point` on Transformation Matrix (Table 21) with `pt` as the argument to that function.

```
vector_2d user_to_surface_distance(const vector_2d& dpt) const noexcept;
```


- 3 *Returns:* The result of calling `matrix_2d::transform_distance` on Transformation Matrix (Table 21) with `dpt` as the argument to that function.

```
vector_2d surface_to_user(const vector_2d& pt) const noexcept;
```

- 4 *Returns:* The result of creating a copy of Transformation Matrix (Table 21), calling `matrix_2d::invert` on that copy, and then calling `matrix_2d::transform_point` on the copy with `pt` as the argument to that function.

```
vector_2d surface_to_user_distance(const vector_2d& dpt) const noexcept;
```

- 5 *Returns:* The result of creating a copy of Transformation Matrix (Table 21), calling `matrix_2d::invert` on that copy, and then calling `matrix_2d::transform_distance` on the copy with `dpt` as the argument to that function.

13.11.23 surface font observers

[surface.observers.font]

```
matrix_2d font_matrix() const noexcept;
```

- 1 *Returns:* The Font Matrix (Table 21).

```
experimental::io2d::font_options font_options() const noexcept;
```

- 2 *Returns:* The Font Options (Table 21).

```
experimental::io2d::font_face font_face() const;
```

```
experimental::io2d::font_face font_face(error_code& ec) const noexcept;
```

- 3 *Returns:* The Font Face (Table 21). *Throws:* As specified in Error reporting (3).

- 5 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

13.12 Class image_surface

[imagesurface]

13.12.1 image_surface synopsis

[imagesurface.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class image_surface : public surface {
    public:
        // 13.12.3, construct/copy/move/destroy:
        image_surface() = delete;
        image_surface(const image_surface&) = delete;
        image_surface& operator=(const image_surface&) = delete;
        image_surface(image_surface&& other) noexcept;
        image_surface& operator=(image_surface&& other) noexcept;
        image_surface(experimental::io2d::format fmt, int width, int height);
        image_surface(experimental::io2d::format fmt, int width, int height,
            error_code& ec) noexcept;
        image_surface(vector<unsigned char>& data, experimental::io2d::format fmt,
            int width, int height);
        image_surface(vector<unsigned char>& data, experimental::io2d::format fmt,
            int width, int height, error_code& ec) noexcept;
        virtual ~image_surface();

        // 13.12.4, modifiers:
        void data(const vector<unsigned char>& data);
        void data(const vector<unsigned char>& data, error_code& ec) noexcept;
        vector<unsigned char> data();
        vector<unsigned char> data(error_code& ec) noexcept;
    };
}}}}
```



```

// 13.12.5, observers:
experimental::io2d::format format() const noexcept;
int width() const noexcept;
int height() const noexcept;
int stride() const noexcept;
};
} } } }

```

13.12.2 image_surface Description

[imagesurface.intro]

- 1 The class `image_surface` derives from the `surface` class and provides an interface to a raster graphics data graphics resource.
- 2 [*Note*: Because of the functionality it provides and what it can be used for, it is expected that developers familiar with other graphics technologies will think of the `image_surface` class as being a form of *render target*. This is intentional, though this Technical Specification does not formally define or use that term to avoid any minor ambiguities and differences in its meaning between the various graphics technologies that do use it. — *end note*]

13.12.3 image_surface constructors and assignment operators

[imagesurface.cons]

```

image_surface(experimental::io2d::format fmt, int width, int height);
image_surface(experimental::io2d::format fmt, int width, int height,
    error_code& ec) noexcept;

```

- 1 *Effects*: Constructs an object of type `image_surface`.
- 2 *Postconditions*: `this->format() == fmt`.
- 3 `this->width() == width`.
- 4 `this->height() == height`.
- 5 *Throws*: As specified in Error reporting (3).
- 6 *Remarks*: The result of calling `this->data()` shall be 0 for all bits that are defined by the specification of that function. [*Note*: Given implementation-specific details, it is possible that not all bits of the `image_surface` object's underlying raster graphics data graphics resource will be used to determine its pixel data. The values of those unused bits are irrelevant and the above paragraph makes it clear that only the bits that matter in determining pixel data have defined values, which are specified to have the same value as the bits of `data`; the value of the other bits, if any, do not have any defined value. — *end note*]
- 7 *Error conditions*: The errors, if any, produced by this function are implementation-defined.

```

image_surface(vector<unsigned char>& data, experimental::io2d::format fmt,
    int width, int height);
image_surface(vector<unsigned char>& data, experimental::io2d::format fmt,
    int width, int height, error_code& ec) noexcept;

```

- 8 *Effects*: Constructs an object of type `image_surface`.
- 9 *Postconditions*: `this->format() == fmt`.
- 10 `this->width() == width`.
- 11 `this->height() == height`.
- 12 `this->data() == data` for all bits that are defined by the specification of that function. [*Note*: Given implementation-specific details, it is possible that not all bits of the `image_surface` object's underlying raster graphics data graphics resource will be used to determine its pixel data. The values of those

unused bits are irrelevant and the above paragraph makes it clear that only the bits that matter in determining pixel data have defined values, which are specified to have the same value as the bits of **data**; the value of the other bits, if any, do not have any defined value. — *end note*]

13 *Throws:* As specified in Error reporting (3).

14 *Error conditions:* `io2d_error::invalid_stride` if `format_stride_for_width(fmt, width) * height != data.size()`.

15 Other errors, if any, produced by this function are implementation-defined.

```
virtual ~image_surface();
```

16 *Effects:* Destroys an object of type `image_surface`.

13.12.4 `image_surface` modifiers

[`imagesurface.modifiers`]

```
void data(const vector<unsigned char>& data);
```

```
void data(const vector<unsigned char>& data, error_code& ec) noexcept;
```

1 *Effects:* Any pending rendering and composing operations (13.11.4) shall be performed.

2 *Postconditions:* `this->data() == data` for all bits that are defined by the specification of that function. [*Note:* Given implementation-specific details, it is possible that not all bits of the `image_surface` object's underlying raster graphics data graphics resource will be used to determine its pixel data. The values of those unused bits are irrelevant and the above paragraph makes it clear that only the bits that matter in determining pixel data have defined values, which are specified to have the same value as the bits of **data**; the value of the other bits, if any, do not have any defined value. — *end note*]

3 *Throws:* As specified in Error reporting (3).

4 *Error conditions:* `io2d_error::invalid_stride` if `format_stride_for_width(fmt, width) * height != data.size()`.

5 Other errors, if any, produced by this function are implementation-defined.

```
vector<unsigned char> data();
```

```
vector<unsigned char> data(error_code& ec) noexcept;
```

6 *Effects:* Any pending rendering and composing operations (13.11.4) shall be performed.

7 *Returns:* A `vector<unsigned char>` containing the byte values of the pixel data of the underlying raster graphics data graphics resource. Where the result of `this->format()` is a `format` value which denotes a multi-byte pixel format, the pixel data shall be in native-endian order.

8 *Throws:* As specified in Error reporting (3).

9 *Error conditions:* `errc::not_enough_memory` if there was a failure to allocate memory.

10 *Notes:* This would normally be an observer function but the requirement that "[a]ny pending rendering and composing operations (13.11.4) shall be performed" means that calling this function might modify the underlying raster graphics data graphics resource. As such this function cannot be marked `const` and thus cannot strictly be classified as an observer function.

11 Developers using this function are cautioned that in many graphics technologies that implementers might use to implement this functionality, the effects of this function will typically cause a large performance degradation and as such it should be used with care and avoided where possible.

13.12.5 image_surface observers**[imagesurface.observers]**

```
experimental::io2d::format format() const noexcept;
```

1 *Returns:* The pixel format of the `image_surface` object.

2 *Remarks:* If the `image_surface` object is invalid, this function shall return `experimental::io2d::format::invalid`.

```
int width() const noexcept;
```

3 *Returns:* The number of pixels per horizontal line of the `image_surface` object.

4 *Remarks:* This function shall return the value 0 if `this->format() == experimental::io2d::format::unknown` || `this->format() == experimental::io2d::format::invalid`.

```
int height() const noexcept;
```

5 *Returns:* The number of horizontal lines of pixels in the `image_surface` object.

6 *Remarks:* This function shall return the value 0 if `this->format() == experimental::io2d::format::unknown` || `this->format() == experimental::io2d::format::invalid`.

```
int stride() const noexcept;
```

7 *Returns:* The length, in bytes, of a horizontal line of the `image_surface` object. [*Note:* This value is at least as large as the width in pixels of a horizontal line multiplied by the number of bytes per pixel but may be larger as a result of padding. — *end note*]

8 *Remarks:* This function shall return the value 0 if `this->format() == experimental::io2d::format::unknown` || `this->format() == experimental::io2d::format::invalid`.

13.13 Class display_surface**[displaysurface]****13.13.1 display_surface synopsis****[displaysurface.synopsis]**

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class display_surface : public surface {
    public:
        // 13.13.4, construct/copy/move/destroy:
        display_surface() = delete;
        display_surface(const display_surface&) = delete;
        display_surface& operator=(const display_surface&) = delete;
        display_surface(display_surface&& other) noexcept;
        display_surface& operator=(display_surface&& other) noexcept;

        display_surface(int preferredWidth, int preferredHeight,
            experimental::io2d::format preferredFormat,
            experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
            experimental::io2d::refresh_rate rr =
                experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0);
        display_surface(int preferredWidth, int preferredHeight,
            experimental::io2d::format preferredFormat, error_code& ec,
            experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
            experimental::io2d::refresh_rate rr =
                experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0)
            noexcept;

        display_surface(int preferredWidth, int preferredHeight,
            experimental::io2d::format preferredFormat,
            int preferredDisplayWidth, int preferredDisplayHeight,
```

```

    experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
    experimental::io2d::refresh_rate rr =
    experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0);
display_surface(int preferredWidth, int preferredHeight,
    experimental::io2d::format preferredFormat,
    int preferredDisplayWidth, int preferredDisplayHeight, error_code& ec,
    experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
    experimental::io2d::refresh_rate rr =
    experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0)
noexcept;

virtual ~display_surface();

// 13.13.5, modifiers:
virtual void save() override;
virtual void save(error_code& ec) noexcept override;
virtual void restore() override;
virtual void restore(error_code& ec) noexcept override;

void draw_callback(const function<void(display_surface& sfc)>& fn) noexcept;
void size_change_callback(const function<void(display_surface& sfc)>& fn)
    noexcept;
void width(int w);
void width(int w, error_code& ec) noexcept;
void height(int h);
void height(int h, error_code& ec) noexcept;
void display_width(int w);
void display_width(int w, error_code& ec) noexcept;
void display_height(int h);
void display_height(int h, error_code& ec) noexcept;
void dimensions(int w, int h);
void dimensions(int w, int h, error_code& ec) noexcept;
void display_dimensions(int dw, int dh);
void display_dimensions(int dw, int dh, error_code& ec) noexcept;
void scaling(experimental::io2d::scaling scl) noexcept;
void user_scaling_callback(const function<experimental::io2d::rectangle(
    const display_surface&, bool)>& fn) noexcept;
void letterbox_brush(experimental::nullopt_t) noexcept;
void letterbox_brush(const rgba_color& c);
void letterbox_brush(const rgba_color& c, error_code& ec) noexcept;
void letterbox_brush(const experimental::io2d::brush& b);
void letterbox_brush(const experimental::io2d::brush& b, error_code& ec)
    noexcept;
void auto_clear(bool val) noexcept;
void refresh_rate(experimental::io2d::refresh_rate rr) noexcept;
bool desired_frame_rate(double fps) noexcept;
void redraw_required() noexcept;
int show();
int show(error_code& ec);
void exit_show(int milliseconds);
void exit_show(int milliseconds, error_code& ec);

// 13.13.6, observers:
experimental::io2d::format format() const noexcept;
int width() const noexcept;

```

```

    int height() const noexcept;
    int display_width() const noexcept;
    int display_height() const noexcept;
    tuple<int, int> dimensions() const noexcept;
    tuple<int, int> display_dimensions() const noexcept;
    experimental::io2d::scaling scaling() const noexcept;
    function<experimental::io2d::rectangle(const display_surface&,
        bool&> user_scaling_callback() const;
    function<experimental::io2d::rectangle(const display_surface&,
        bool&> user_scaling_callback(error_code& ec) const noexcept;
    experimental::io2d::brush letterbox_brush() const noexcept;
    bool auto_clear() const noexcept;
    experimental::io2d::refresh_rate refresh_rate() const noexcept;
    double desired_frame_rate() const noexcept;
    double elapsed_draw_time() const noexcept;
};
} } } }

```

13.13.2 display_surface Description

[displaysurface.intro]

- ¹ The class `display_surface` derives from the `surface` class and provides an interface to a raster graphics data graphics resource called the **Back Buffer** and to a second raster graphics data graphics resource called the **Display Buffer**.
- ² The pixel data of the Display Buffer can never be accessed by the user except through a native handle, if one is provided. As such, its pixel format need not equate to any of the pixel formats described by the `experimental::io2d::format` enumerators. This is meant to give implementors more flexibility in trying to display the pixels of the Back Buffer in a way that is visually as close as possible to the colors of those pixels.
- ³ The Draw Callback (Table 24) is called by `display_surface::show` as required by the Refresh Rate and when otherwise needed by the implementation in order to update the pixel content of the Back Buffer.
- ⁴ After each execution of the Draw Callback, the contents of the Back Buffer are transferred using sampling with an unspecified filter to the Display Buffer. The Display Buffer is then shown to the user via the *output device*. [*Note*: The filter is unspecified to allow implementations to achieve the best possible result, including by changing filters at runtime depending on factors such as whether scaling is required and by using specialty hardware if available, while maintaining a balance between quality and performance that the implementer deems acceptable.

In the absence of specialty hardware, implementers are encouraged to use a filter that is the equivalent of a nearest neighbor interpolation filter if no scaling is required and otherwise to use a filter that produces results that are at least as good as those that would be obtained by using a bilinear interpolation filter. — *end note*]
- ⁵ What constitutes an output device is implementation-defined, with the sole constraint being that an output device shall allow the user to see the dynamically-updated contents of the Display Buffer. [*Example*: An output device might be a window in a windowing system environment or the usable screen area of a smart phone or tablet. — *end example*]
- ⁶ Implementations need not support the simultaneous existence of multiple `display_surface` objects.
- ⁷ All functions inherited from `surface` that affect its underlying graphics data graphics resource shall operate on the Back Buffer.

13.13.3 display_surface state**[displaysurface.state]**

- ¹ Table 24 specifies the name, type, function, and default value for each item of a display surface's observable state.
- ² Because the `display_surface` class publicly derives from the `surface` class, the observable state of a display surface also includes the observable state of a surface, as specified at 13.11.3.1.

Table 24 — Display surface observable state

Name	Type	Function	Default value
<i>Letterbox Brush</i>	<code>brush</code>	This is the brush that shall be used as specified by <code>scaling::letterbox</code> (Table 19)	<code>brush{ { rgba_color::black() } }</code>
<i>Scaling Type</i>	<code>scaling</code>	When the User Scaling Callback is equal to its default value, this is the type of scaling that shall be used when transferring the Back Buffer to the Display Buffer	<code>antialias::default_antialias</code>
<i>Draw Width</i>	<code>int</code>	The width in pixels of the Back Buffer. The minimum value is 1. The maximum value is unspecified. Because users can only request a preferred value for the Draw Width when setting and altering it, the maximum value may be a run-time determined value. If the preferred Draw Width exceeds the maximum value, then if a preferred Draw Height has also been supplied then implementations should provide a Back Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred Draw Width and the preferred Draw Height otherwise implementations should provide a Back Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred Draw Width and the current Draw Height	<i>N/A</i> [<i>Note</i> : It is impossible to create a <code>display_surface</code> object without providing a preferred Draw Width value; as such a default value cannot exist. — <i>end note</i>]

Table 24 — Display surface observable state (continued)

Name	Type	Function	Default value
<i>Draw Height</i>	int	The height in pixels of the Back Buffer. The minimum value is 1. The maximum value is unspecified. Because users can only request a preferred value for the Draw Height when setting and altering it, the maximum value may be a run-time determined value. If the preferred Draw Height exceeds the maximum value, then if a preferred Draw Width has also been supplied then implementations should provide a Back Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred Draw Width and the preferred Draw Height otherwise implementations should provide a Back Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the current Draw Width and the preferred Draw Height	<i>N/A</i> [<i>Note:</i> It is impossible to create a display_surface object without providing a preferred Draw Height value; as such a default value cannot exist. — <i>end note</i>]
<i>Draw Format</i>	format	The pixel format of the Back Buffer. When a display_surface object is created, a preferred pixel format value is provided. If the implementation does not support the preferred pixel format value as the value of Draw Format, the resulting value of Draw Format is implementation-defined	<i>N/A</i> [<i>Note:</i> It is impossible to create a display_surface object without providing a preferred Draw Format value; as such a default value cannot exist. — <i>end note</i>]

Table 24 — Display surface observable state (continued)

Name	Type	Function	Default value
<i>Display Width</i>	int	The width in pixels of the Display Buffer. The minimum value is unspecified. The maximum value is unspecified. Because users can only request a preferred value for the Display Width when setting and altering it, both the minimum value and the maximum value may be run-time determined values. If the preferred Display Width is not within the range between the minimum value and the maximum value, inclusive, then if a preferred Display Height has also been supplied then implementations should provide a Display Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred Display Width and the preferred Display Height otherwise implementations should provide a Display Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred Display Width and the current Display Height	<i>N/A</i> [<i>Note</i> : It is impossible to create a display_surface object without providing a preferred Display Width value since in the absence of an explicit Display Width argument the mandatory preferred Draw Width argument is used as the preferred Display Width; as such a default value cannot exist. — <i>end note</i>]

Table 24 — Display surface observable state (continued)

Name	Type	Function	Default value
<i>Display Height</i>	<code>int</code>	The height in pixels of the Display Buffer. The minimum value is unspecified. The maximum value is unspecified. Because users can only request a preferred value for the Display Height when setting and altering it, both the minimum value and the maximum value may be run-time determined values. If the preferred Display Height is not within the range between the minimum value and the maximum value, inclusive, then if a preferred Display Width has also been supplied then implementations should provide a Display Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred Display Width and the preferred Display Height otherwise implementations should provide a Display Buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the current Display Width and the preferred Display Height	<i>N/A</i> [<i>Note</i> : It is impossible to create a <code>display_surface</code> object without providing a preferred Display Height value since in the absence of an explicit Display Height argument the mandatory preferred Draw Height argument is used as the preferred Display Height; as such a default value cannot exist. — <i>end note</i>]
<i>Draw Callback</i>	<code>function< void(display_surface&)></code>	This function shall be called in a continuous loop when <code>display_surface::show</code> is executing. It is used to draw to the Back Buffer, which in turn results in the display of the drawn content to the user	<code>nullptr</code>

Table 24 — Display surface observable state (continued)

Name	Type	Function	Default value
<i>Size Change Callback</i>	<code>function< void(display_surface&)></code>	If it exists, this function shall be called whenever the Display Buffer has been resized. Neither the Display Width nor the Display Height shall be changed by the Size Change Callback; no diagnostic is required [<i>Note</i> : This means that there has been a change to the Display Width, Display Height, or both. Its intent is to allow the user the opportunity to change other observable state, such as the Draw Width, Draw Height, or Scaling Type, in reaction to the change. — <i>end note</i>]	<code>nullptr</code>
<i>User Scaling Callback</i>	<code>function< experimental::io::rectangle(const display_surface&, bool&)></code>	If it exists, this function shall be called whenever the contents of the Back Buffer need to be copied to the Display Buffer. The function is called with the const reference to <code>display_surface</code> object and a reference to a <code>bool</code> variable which has the value <code>false</code> . If the value of the <code>bool</code> is <code>true</code> when the function returns, the Letterbox Brush shall be used as specified by <code>scaling::letterbox</code> (Table 19). The function shall return a <code>rectangle</code> object that defines the area within the Display Buffer to which the Back Buffer shall be transferred. The <code>rectangle</code> may include areas outside of the bounds of the Display Buffer, in which case only the area of the Back Buffer that lies within the bounds of the Display Buffer will ultimately be visible to the user	<code>nullptr</code>

Table 24 — Display surface observable state (continued)

Name	Type	Function	Default value
<i>Auto Clear</i>	bool	If <code>true</code> the implementation shall call <code>surface::clear</code> , which shall clear the Back Buffer, immediately before it executes the Draw Callback	<code>false</code>
<i>Refresh Rate</i>	refresh_rate	The <code>refresh_rate</code> value that determines when the Draw Callback shall be called while <code>display_surface::show</code> is being executed	<code>refresh_rate::as_fast_as_possible</code>
<i>Desired Frame Rate</i>	double	This value is the number of times the Draw Callback shall be called per second while <code>display_surface::show</code> is being executed when the value of Refresh Rate is <code>refresh_rate::fixed</code> , subject to the additional requirements documented in the meaning of <code>refresh_rate::fixed</code> (Table 20)	

13.13.4 display_surface constructors and assignment operators [displaysurface.cons]

```
display_surface(int preferredWidth, int preferredHeight,
  experimental::io2d::format preferredFormat,
  experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
  experimental::io2d::refresh_rate rr =
  experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0);
display_surface(int preferredWidth, int preferredHeight,
  experimental::io2d::format preferredFormat, error_code& ec,
  experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
  experimental::io2d::refresh_rate rr =
  experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0)
noexcept;
```

- 1 *Effects:* Constructs an object of type `display_surface`.
- 2 The `preferredWidth` parameter specifies the preferred width value for Draw Width and Display Width. The `preferredHeight` parameter specifies the preferred height value for Draw Height and Display Height. Draw Width and Display Width need not have the same value. Draw Height and Display Height need not have the same value.
- 3 The `preferredFormat` parameter specifies the preferred pixel format value for Draw Format.
- 4 The value of Scaling Type shall be the value of `scl`.
- 5 The value of Refresh Rate shall be the value of `rr`.
- 6 The value of Desired Frame Rate shall be as if `display_surface::desired_frame_rate` was called with `fps` as its argument. If `!is_finite(fps)`, then the value of Desired Frame Rate shall be its default value.

All other observable state data shall have their default values.

Throws: As specified in Error reporting (3).

Error conditions: `errc::invalid_argument` if `preferredWidth <= 0`, `preferredHeight <= 0`, or `preferredFormat == experimental::io2d::format::invalid`.

`io2d::device_error` if successful creation of the `display_surface` object would exceed the maximum number of simultaneous valid `display_surface` objects that the implementation supports.

Other errors, if any, produced by this function are implementation-defined.

```
display_surface(int preferredWidth, int preferredHeight,
    experimental::io2d::format preferredFormat,
    int preferredDisplayWidth, int preferredDisplayHeight,
    experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
    experimental::io2d::refresh_rate rr =
    experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0);
display_surface(int preferredWidth, int preferredHeight,
    experimental::io2d::format preferredFormat,
    int preferredDisplayWidth, int preferredDisplayHeight, error_code& ec,
    experimental::io2d::scaling scl = experimental::io2d::scaling::letterbox,
    experimental::io2d::refresh_rate rr =
    experimental::io2d::refresh_rate::as_fast_as_possible, double fps = 30.0)
noexcept;
```

Effects: Constructs an object of type `display_surface`.

The `preferredWidth` parameter specifies the preferred width value for Draw Width. The `preferredDisplayWidth` parameter specifies the preferred display width value for Display Width. The `preferredHeight` parameter specifies the preferred height value for Draw Height. The `preferredDisplayHeight` parameter specifies the preferred display height value for Display Height.

The `preferredFormat` parameter specifies the preferred pixel format value for Draw Format.

The value of Scaling Type shall be the value of `scl`.

The value of Refresh Rate shall be the value of `rr`.

The value of Desired Frame Rate shall be as if `display_surface::desired_frame_rate` was called with `fps` as its argument. If `!is_finite(fps)`, then the value of Desired Frame Rate shall be its default value.

All other observable state data shall have their default values.

Throws: As specified in Error reporting (3).

Error conditions: `errc::invalid_argument` if `preferredWidth <= 0`, `preferredHeight <= 0`, `preferredDisplayWidth <= 0`, `preferredDisplayHeight <= 0`, or `preferredFormat == experimental::io2d::format::invalid`.

`io2d::device_error` if successful creation of the `display_surface` object would exceed the maximum number of simultaneous valid `display_surface` objects that the implementation supports.

Other errors, if any, produced by this function are implementation-defined.

13.13.5 `display_surface` modifiers

[`displaysurface.modifiers`]

```
virtual void save() override;
virtual void save(error_code& ec) noexcept override;
```

Effects: Calls `surface::save`.

Implementations may save additional data provided that it does not alter the observable state of the `display_surface` object.

3 *Throws:* As specified in Error reporting (3).

4 *Error conditions:* `errc::not_enough_memory` if the state cannot be saved.

```
virtual void restore() override;
virtual void restore(error_code& ec) noexcept override;
```

5 *Effects:* Calls `surface::restore`.

6 If the implementation saved additional data as per `display_surface::save`, it should restore that data. Otherwise it shall discard that data.

7 *Throws:* As specified in Error reporting (3).

8 *Remarks:* Because this function is only restoring previously saved state, except where the conditions for `io2d_error::invalid_restore` are met, implementations should not generate errors.

9 *Error conditions:* `io2d_error::invalid_restore` if this function is called without a previous matching call to `surface::save`. Implementations shall not produce `io2d_error::invalid_restore` except under the conditions stated in this paragraph.

10 Excluding the previously specified error, any errors produced by calling this function are implementation-defined.

```
void draw_callback(const function<void(display_surface& sfc)>& fn) noexcept;
```

11 *Effects:* Sets the Draw Callback to `fn`.

```
void size_change_callback(const function<void(display_surface& sfc)>& fn)
    noexcept;
```

12 *Effects:* Sets the Size Change Callback to `fn`.

```
void width(int w);
void width(int w, error_code& ec) noexcept;
```

13 *Effects:* If the value of Draw Width is the same as `w`, this function does nothing.

14 Otherwise, Draw Width is set as specified by Table 24 with `w` treated as being the preferred Draw Width.

15 If the value of Draw Width changes as a result, the implementation shall attempt to create a new Back Buffer with the updated dimensions while retaining the existing Back Buffer. The implementation may destroy the existing Back Buffer prior to creating a new Back Buffer with the updated dimensions only if it can guarantee that in doing so it will either succeed in creating the new Back Buffer or will be able to create a Back Buffer with the previous dimensions in the event of failure.

16 [*Note:* The intent of the previous paragraph is to ensure that, no matter the result, a valid Back Buffer continues to exist. Sometimes implementations will be able to determine that the new dimensions are valid but that to create the new Back Buffer successfully the previous one must be destroyed. The previous paragraph gives implementors that leeway. It goes even further in that it allows implementations to destroy the existing Back Buffer even if they cannot determine in advance that creating the new Back Buffer will succeed, provided that they can guarantee that if the attempt fails they can always successfully recreate a Back Buffer with the previous dimensions. Regardless, there must be a valid Back Buffer when this call completes. — *end note*]

17 The value of the Back Buffer's pixel data shall be unspecified upon completion of this function regardless of whether it succeeded.

18 If an error occurs, the implementation shall ensure that the Back Buffer is valid and has the same dimensions it had prior to this call and that Draw Width shall retain its value prior to this call.

19 *Throws:* As specified in Error reporting (3).
 20 *Error conditions:* `errc::invalid_argument` if `w <= 0` or if the value of `w` is greater than the maximum value for Draw Width.

`errc::not_enough_memory` if there is insufficient memory to create a Back Buffer with the updated dimensions.

Other errors, if any, produced by this function are implementation-defined.

```
void height(int h);
void height(int h, error_code& ec) noexcept;
```

21 *Effects:* If the value of Draw Height is the same as `h`, this function does nothing.
 22 Otherwise, Draw Height is set as specified by Table 24 with `h` treated as being the preferred Draw Height.

23 If the value of Draw Height changes as a result, the implementation shall attempt to create a new Back Buffer with the updated dimensions while retaining the existing Back Buffer. The implementation may destroy the existing Back Buffer prior to creating a new Back Buffer with the updated dimensions only if it can guarantee that in doing so it will either succeed in creating the new Back Buffer or will be able to create a Back Buffer with the previous dimensions in the event of failure.

24 [*Note:* The intent of the previous paragraph is to ensure that, no matter the result, a valid Back Buffer continues to exist. Sometimes implementations will be able to determine that the new dimensions are valid but that to create the new Back Buffer successfully the previous one must be destroyed. The previous paragraph gives implementors that leeway. It goes even further in that it allows implementations to destroy the existing Back Buffer even if they cannot determine in advance that creating the new Back Buffer will succeed, provided that they can guarantee that if the attempt fails they can always successfully recreate a Back Buffer with the previous dimensions. Regardless, there must be a valid Back Buffer when this call completes. — *end note*]

25 The value of the Back Buffer's pixel data shall be unspecified upon completion of this function regardless of whether it succeeded.

26 If an error occurs, the implementation shall ensure that the Back Buffer is valid and has the same dimensions it had prior to this call and that Draw Height shall retain its value prior to this call.

27 *Throws:* As specified in Error reporting (3).

28 *Error conditions:* `errc::invalid_argument` if `h <= 0` or if the value of `h` is greater than the maximum value for Draw Height.

`errc::not_enough_memory` if there is insufficient memory to create a Back Buffer with the updated dimensions.

Other errors, if any, produced by this function are implementation-defined.

```
void display_width(int w);
void display_width(int w, error_code& ec) noexcept;
```

29 *Effects:* If the value of Display Width is the same as `w`, this function does nothing.

30 Otherwise, Display Width is set as specified by Table 24 with `w` treated as being the preferred Display Width.

31 If the value of Display Width changes as a result, the implementation shall attempt to create a new Display Buffer with the updated dimensions while retaining the existing Display Buffer. The implementation may destroy the existing Display Buffer prior to creating a new Display Buffer with the updated dimensions only if it can guarantee that in doing so it will either succeed in creating the

new Display Buffer or will be able to create a Display Buffer with the previous dimensions in the event of failure.

[*Note:* The intent of the previous paragraph is to ensure that, no matter the result, a valid Display Buffer continues to exist. Sometimes implementations will be able to determine that the new dimensions are valid but that to create the new Display Buffer successfully the previous one must be destroyed. The previous paragraph gives implementors that leeway. It goes even further in that it allows implementations to destroy the existing Display Buffer even if they cannot determine in advance that creating the new Display Buffer will succeed, provided that they can guarantee that if the attempt fails they can always successfully recreate a Display Buffer with the previous dimensions. Regardless, there must be a valid Display Buffer when this call completes. — *end note*]

The value of the Display Buffer's pixel data shall be unspecified upon completion of this function regardless of whether it succeeded.

If an error occurs, the implementation shall ensure that the Display Buffer is valid and has the same dimensions it had prior to this call and that Display Width shall retain its value prior to this call.

Throws: As specified in Error reporting (3).

Error conditions: `errc::invalid_argument` if the value of `w` is less than the minimum value for Display Width or if the value of `w` is greater than the maximum value for Display Width.

`errc::not_enough_memory` if there is insufficient memory to create a Display Buffer with the updated dimensions.

Other errors, if any, produced by this function are implementation-defined.

```
void display_height(int h);
void display_height(int h, error_code& ec) noexcept;
```

Effects: If the value of Display Height is the same as `h`, this function does nothing.

Otherwise, Display Height is set as specified by Table 24 with `h` treated as being the preferred Display Height.

If the value of Display Height changes as a result, the implementation shall attempt to create a new Display Buffer with the updated dimensions while retaining the existing Display Buffer. The implementation may destroy the existing Display Buffer prior to creating a new Display Buffer with the updated dimensions only if it can guarantee that in doing so it will either succeed in creating the new Display Buffer or will be able to create a Display Buffer with the previous dimensions in the event of failure.

[*Note:* The intent of the previous paragraph is to ensure that, no matter the result, a valid Display Buffer continues to exist. Sometimes implementations will be able to determine that the new dimensions are valid but that to create the new Display Buffer successfully the previous one must be destroyed. The previous paragraph gives implementors that leeway. It goes even further in that it allows implementations to destroy the existing Display Buffer even if they cannot determine in advance that creating the new Display Buffer will succeed, provided that they can guarantee that if the attempt fails they can always successfully recreate a Display Buffer with the previous dimensions. Regardless, there must be a valid Display Buffer when this call completes. — *end note*]

The value of the Display Buffer's pixel data shall be unspecified upon completion of this function regardless of whether it succeeded.

If an error occurs, the implementation shall ensure that the Display Buffer is valid and has the same dimensions it had prior to this call and that Display Height shall retain its value prior to this call.

Throws: As specified in Error reporting (3).

Error conditions: `errc::invalid_argument` if the value of `h` is less than the minimum value for Display Height or if the value of `h` is greater than the maximum value for Display Height.

`errc::not_enough_memory` if there is insufficient memory to create a Display Buffer with the updated dimensions.

Other errors, if any, produced by this function are implementation-defined.

```
void dimensions(int w, int h);
void dimensions(int w, int h, error_code& ec) noexcept;
```

45 *Effects:* If the value of Draw Width is the same as `w` and the value of Draw Height is the same as `h`, this function does nothing.

46 Otherwise, Draw Width is set as specified by Table 24 with `w` treated as being the preferred Draw Width and Draw Height is set as specified by Table 24 with `h` treated as being the preferred Draw Height.

47 If the value of Draw Width changes as a result or the value of Draw Height changes as a result, the implementation shall attempt to create a new Back Buffer with the updated dimensions while retaining the existing Back Buffer. The implementation may destroy the existing Back Buffer prior to creating a new Back Buffer with the updated dimensions only if it can guarantee that in doing so it will either succeed in creating the new Back Buffer or will be able to create a Back Buffer with the previous dimensions in the event of failure.

48 [*Note:* The intent of the previous paragraph is to ensure that, no matter the result, a valid Back Buffer continues to exist. Sometimes implementations will be able to determine that the new dimensions are valid but that to create the new Back Buffer successfully the previous one must be destroyed. The previous paragraph gives implementors that leeway. It goes even further in that it allows implementations to destroy the existing Back Buffer even if they cannot determine in advance that creating the new Back Buffer will succeed, provided that they can guarantee that if the attempt fails they can always successfully recreate a Back Buffer with the previous dimensions. Regardless, there must be a valid Back Buffer when this call completes. — *end note*]

49 The value of the Back Buffer's pixel data shall be unspecified upon completion of this function regardless of whether it succeeded.

50 If an error occurs, the implementation shall ensure that the Back Buffer is valid and has the same dimensions it had prior to this call and that Draw Width and Draw Height shall retain the values they had prior to this call.

51 *Throws:* As specified in Error reporting (3).

52 *Error conditions:* `errc::invalid_argument` if `w <= 0`, if the value of `w` is greater than the maximum value for Draw Width, if `h <= 0` or if the value of `h` is greater than the maximum value for Draw Height.

`errc::not_enough_memory` if there is insufficient memory to create a Back Buffer with the updated dimensions.

Other errors, if any, produced by this function are implementation-defined.

```
void display_dimensions(int dw, int dh);
void display_dimensions(int dw, int dh, error_code& ec) noexcept;
```

53 *Effects:* If the value of Display Width is the same as `w` and the value of Display Height is the same as `h`, this function does nothing.

54 Otherwise, Display Width is set as specified by Table 24 with `w` treated as being the preferred Display Height and Display Height is set as specified by Table 24 with `h` treated as being the preferred Display Height.

55 If the value of Display Width or the value of Display Height changes as a result, the implementation shall attempt to create a new Display Buffer with the updated dimensions while retaining the existing

Display Buffer. The implementation may destroy the existing Display Buffer prior to creating a new Display Buffer with the updated dimensions only if it can guarantee that in doing so it will either succeed in creating the new Display Buffer or will be able to create a Display Buffer with the previous dimensions in the event of failure.

56 [*Note:* The intent of the previous paragraph is to ensure that, no matter the result, a valid Display Buffer continues to exist. Sometimes implementations will be able to determine that the new dimensions are valid but that to create the new Display Buffer successfully the previous one must be destroyed. The previous paragraph gives implementors that leeway. It goes even further in that it allows implementations to destroy the existing Display Buffer even if they cannot determine in advance that creating the new Display Buffer will succeed, provided that they can guarantee that if the attempt fails they can always successfully recreate a Display Buffer with the previous dimensions. Regardless, there must be a valid Display Buffer when this call completes. — *end note*]

57 If an error occurs, the implementation shall ensure that the Display Buffer is valid and has the same dimensions it had prior to this call and that Display Width and Display Height shall retain the values they had prior to this call.

58 If the Display Buffer has changed, even if its width and height have not changed, the Draw Callback shall be called.

59 If the width or height of the Display Buffer has changed, the Size Change Callback shall be called if it's value is not its default value.

60 *Throws:* As specified in Error reporting (3).

61 *Error conditions:* `errc::invalid_argument` if the value of `w` is less than the minimum value for Display Width, if the value of `w` is greater than the maximum value for Display Width, if the value of `h` is less than the minimum value for Display Height, or if the value of `h` is greater than the maximum value for Display Height.

`errc::not_enough_memory` if there is insufficient memory to create a Display Buffer with the updated dimensions.

Other errors, if any, produced by this function are implementation-defined.

```
void scaling(experimental::io2d::scaling scl) noexcept;
```

62 *Effects:* Sets Scaling Type to the value of `scl`.

```
void user_scaling_callback(const function<experimental::io2d::rectangle(
    const display_surface&, bool&)>& fn) noexcept;
```

63 *Effects:* Sets the User Scaling Callback to `fn`.

```
void letterbox_brush(experimental::nullopt_t) noexcept;
```

64 *Effects:* Sets the Letterbox Brush to its default value.

```
void letterbox_brush(const rgba_color& c);
void letterbox_brush(const rgba_color& c, error_code& ec) noexcept;
```

65 *Effects:* Sets the Letterbox Brush to a value as if `experimental::io2d::brush{ solid_color_brush_factory{ } }`.

66 *Throws:* As specified in Error reporting (3).

67 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void letterbox_brush(const experimental::io2d::brush& b);
void letterbox_brush(const experimental::io2d::brush& b, error_code& ec)
    noexcept;
```

68 *Effects:* Sets the Letterbox Brush to `b`.

69 *Throws:* As specified in Error reporting (3).

70 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

`void auto_clear(bool val) noexcept;`

71 *Effects:* Sets Auto Clear to the value of `val`.

`void refresh_rate(experimental::io2d::refresh_rate rr) noexcept;`

72 *Effects:* Sets the Refresh Rate to the value of `rr`.

`bool desired_frame_rate(double fps) noexcept;`

73 *Effects:* If `!is_finite(fps)`, this function has no effects.

74 Sets the Desired Frame Rate to an implementation-defined minimum frame rate if `fps` is less than the minimum frame rate, an implementation-defined maximum frame rate if `fps` is greater than the maximum frame rate, otherwise to the value of `fps`.

75 *Returns:* `false` if the Desired Frame Rate was set to the value of `fps`; otherwise `true`.

`void redraw_required() noexcept;`

76 *Effects:* When `display_surface::show` is executing, informs the implementation that it shall call the Draw Callback as soon as possible.

`int show();`
`int show(error_code& ec);`

77 *Effects:* Performs the following actions in a continuous loop:

- 1) Handle any implementation and host environment matters. If there are no pending implementation or host environment matters to handle, proceed immediately to the next action.
- 2) Run the Size Change Callback if doing so is required by its specification and it does not have a value equivalent to its default value.
- 3) If the Refresh Rate requires that the Draw Callback be called then:
 - a) Evaluate Auto Clear and perform the actions required by its specification, if any.
 - b) Run the Draw Callback.
 - c) Ensure that all operations from the Draw Callback that can effect the Back Buffer have completed.
 - d) Transfer the contents of the Back Buffer to the Display Buffer using sampling with an unspecified filter. If the User Scaling Callback does not have a value equivalent to its default value, use it to determine the position where the contents of the Back Buffer shall be transferred to and whether or not the Letterbox Brush should be used. Otherwise use the value of Scaling Type to determine the position and whether the Letterbox Brush should be used.

78 If `display_surface::exit_show` is called from the Draw Callback, the implementation shall finish executing the Draw Callback and shall immediately cease to perform any actions in the continuous loop other than handling any implementation and host environment matters.

79 No later than when this function returns, the output device shall cease to display the contents of the Display Buffer.

80 What the output device shall display when it is not displaying the contents of the Display Buffer is unspecified.

81 *Returns:* The possible values and meanings of the possible values returned are implementation-defined.

82 *Throws:* As specified in Error reporting (3).

83 *Remarks:* Since this function calls the Draw Callback and can call the Size Change Callback and the User Scaling Callback, in addition to the errors documented below, any errors that the callback functions produce can also occur.

84 *Error conditions:* `errc::operation_would_block` if the value of Draw Callback is equivalent to its default value or if it becomes equivalent to its default value before this function returns.

86 Other errors, if any, produced by this function are implementation-defined.

```
void exit_show(int ms) noexcept;
```

87 *Requires:* This function shall only be called from the Draw Callback; no diagnostic is required.

88 *Effects:* The implementation shall initiate the process of exiting the `display_surface::show` function's continuous loop.

89 Implementations shall not wait until the `display_surface::show` function's continuous loop ends before returning from this function.

90 Implementations should follow any procedures that the host environment requires in order to cause the `display_surface::show` function's continuous loop to stop executing without error.

91 A *termination time duration* shall then be determined as follows:

(91.1) — If the value `ms` is negative, the termination time duration shall be an unspecified number of milliseconds.

(91.2) — Otherwise the termination time duration shall be `ms` milliseconds.

92 The implementation shall continue to execute the `display_surface::show` function until it returns or until termination time duration milliseconds have passed since the termination time duration was determined, whichever comes first.

93 If the `display_surface::show` function has not returned before termination time duration milliseconds have passed since the termination time duration was determined the implementation shall force the `display_surface::show` function's continuous loop to stop executing and shall then cause `display_surface::show` to return.

13.13.6 `display_surface` observers

[`displaysurface.observers`]

```
experimental::io2d::format format() const noexcept;
```

1 *Returns:* The value of Draw Format.

```
int width() const noexcept;
```

2 *Returns:* The Draw Width.

```
int height() const noexcept;
```

3 *Returns:* The Draw Height.

```
int display_width() const noexcept;
```

4 *Returns:* The Display Width.

```
int display_height() const noexcept;
```

5 *Returns:* The Display Height.

```
tuple<int, int> dimensions() const noexcept;
```

6 *Returns:* A tuple<int, int> where the first element is the Draw Width and the second element is the Draw Height.

```
tuple<int, int> display_dimensions() const noexcept;
```

7 *Returns:* A tuple<int, int> where the first element is the Display Width and the second element is the Display Height.

```
experimental::io2d::scaling scaling() const noexcept;
```

8 *Returns:* The Scaling Type.

```
function<experimental::io2d::rectangle(const display_surface&, bool&>>
    user_scaling_callback() const;
function<experimental::io2d::rectangle(const display_surface&, bool&>>
    user_scaling_callback(error_code& ec) const noexcept;
```

9 *Returns:* A copy of User Scaling Callback.

10 *Throws:* As specified in Error reporting (3).

11 *Error conditions:* `errc::not_enough_memory` if a failure to allocate memory occurs.

```
experimental::io2d::brush letterbox_brush() const noexcept;
```

12 *Returns:* The Letterbox Brush.

```
bool auto_clear() const noexcept;
```

13 *Returns:* The value of Auto Clear.

```
double desired_framerate() const noexcept;
```

14 *Returns:* The value of Desired Framerate.

```
double elapsed_draw_time() const noexcept;
```

15 *Returns:* If called from the Draw Callback during the execution of `display_surface::show`, the amount of time in milliseconds that has passed since the previous call to the Draw Callback by the current execution of `display_surface::show`; otherwise 0.0.

13.14 Class mapped_surface

[mappedsurface]

13.14.1 mapped_surface synopsis

[mappedsurface.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    class mapped_surface {
    public:
        // 13.14.3, construct/copy/move/destroy:
        mapped_surface() = delete;
        mapped_surface(const mapped_surface&) = delete;
        mapped_surface& operator=(const mapped_surface&) = delete;
        mapped_surface(mapped_surface&& other) = delete;
        mapped_surface& operator=(mapped_surface&& other) = delete;
        ~mapped_surface();

        // 13.14.4, modifiers:
        void commit_changes();
        void commit_changes(error_code& ec) noexcept;
```

```

void commit_changes(const rectangle& area);
void commit_changes(const rectangle& area, error_code& ec) noexcept;
unsigned char* data();
unsigned char* data(error_code& ec) noexcept;

// 13.14.5, observers:
const unsigned char* data() const;
const unsigned char* data(error_code& ec) const noexcept;
experimental::io2d::format format() const noexcept;
int width() const noexcept;
int height() const noexcept;
int stride() const noexcept;
};
} } } }

```

13.14.2 mapped_surface Description

[mappedsurface.intro]

- 1 The `mapped_surface` class provides access to inspect and modify the pixel data of a `surface` object's underlying graphics data graphics resource or a subsection thereof.
- 2 A `mapped_surface` can only be created by the `surface::map` function. It cannot be copied or moved.
- 3 The pixel data is presented as an array in the form of a pointer to (possibly `const`) `unsigned char`.
- 4 The actual format of the pixel data depends on the `format` enumerator returned by calling `mapped_surface::format` and is native-endian. For more information, see the description of the `format` enum class (13.7).
- 5 The pixel data array is presented as a series of horizontal rows of pixels with row 0 being the top row of pixels of the underlying graphics data graphics resource and the bottom row being the row at `mapped_surface::height() - 1`.
- 6 Each horizontal row of pixels begins with the leftmost pixel and proceeds right to `mapped_surface::width() - 1`.
- 7 The width in bytes of each horizontal row is provided by `mapped_surface::stride`. This value may be larger than the result of multiplying the width in pixels of each horizontal row by the size in bytes of the pixel's format (most commonly as a result of implementation-dependent memory alignment requirements).
- 8 Whether the pixel data array provides direct access to the underlying graphics data graphics resource's memory or provides indirect access as if through a proxy or a copy is unspecified.
- 9 Changes made to the pixel data array are considered to be *uncommitted* so long as those changes are not reflected in the underlying graphics data graphics resource.
- 10 Changes made to the pixel data array are considered to be *committed* once they are reflected in the underlying graphics data graphics resource.

13.14.3 mapped_surface constructors and assignment operators [mappedsurface.cons]

```
~mapped_surface();
```

- 1 *Effects:* Destroys an object of type `mapped_surface`.
- 2 *Remarks:* Whether any uncommitted changes are committed during destruction of the `mapped_surface` object is unspecified.
- 3 Uncommitted changes shall not be committed during destruction of the `mapped_surface` object if doing so would result in an exception.
- 4 *Notes:* It is recommended that users use the `mapped_surface::commit_changes` function to commit changes prior to the destruction of the `mapped_surface` object to ensure consistent behavior.

13.14.4 mapped_surface modifiers

[mappedsurface.modifiers]

```
void commit_changes();
void commit_changes(error_code& ec) noexcept;
```

- 1 *Effects:* Any uncommitted changes shall be committed.
- 2 *Throws:* As specified in Error reporting (3).
- 3 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
unsigned char* data();
unsigned char* data(error_code& ec) noexcept;
```

- 4 *Returns:* A native-endian pointer to the pixel data array. [*Example:* Given the following code:

```
image_surface imgsfc{ format::argb32, 100, 100 };
imgsfc.paint(rgba_color::red());
imgsfc.flush();
imgsfc.map([](mapped_surface& mapsfc) -> void {
    auto pixelData = mapsfc.data();
    auto p0 = static_cast<uint32_t>(pixelData[0]);
    auto p1 = static_cast<uint32_t>(pixelData[1]);
    auto p2 = static_cast<uint32_t>(pixelData[2]);
    auto p3 = static_cast<uint32_t>(pixelData[3]);
    printf("%X %X %X %X\n", p0, p1, p2, p3);
});
```

In a little-endian environment, p0 == 0x0, p1 == 0x0, p2 == 0xFF, and p3 == 0xFF.

In a big-endian environment, p0 == 0xFF, p1 == 0xFF, p2 == 0x0, p3 == 0x0. — *end example*]

- 5 *Throws:* As specified in Error reporting (3).
- 6 *Remarks:* The bounds of the pixel data array range from a, where a is the address returned by this function, to a + this->stride() * this->height(). Given a height h where h is any value from 0 to this->height() - 1, any attempt to read or write a byte with an address that is not within the range of addresses defined by a + this->stride() * h shall result in undefined behavior; no diagnostic is required.
- 7 *Error conditions:* io2d_error::null_pointer if this->format() == experimental::io2d::format::unknown || this->format() == experimental::io2d::format::invalid.

13.14.5 mapped_surface observers

[mappedsurface.observers]

```
const unsigned char* data() const;
const unsigned char* data(error_code& ec) const noexcept;
```

- 1 *Returns:* A const native-endian pointer to the pixel data array. [*Example:* Given the following code:

```
image_surface imgsfc{ format::argb32, 100, 100 };
imgsfc.paint(rgba_color::red());
imgsfc.flush();
imgsfc.map([](mapped_surface& mapsfc) -> void {
    auto pixelData = mapsfc.data();
    auto p0 = static_cast<uint32_t>(pixelData[0]);
    auto p1 = static_cast<uint32_t>(pixelData[1]);
    auto p2 = static_cast<uint32_t>(pixelData[2]);
    auto p3 = static_cast<uint32_t>(pixelData[3]);
    printf("%X %X %X %X\n", p0, p1, p2, p3);
});
```

In a little-endian environment, `p0 == 0x0`, `p1 == 0x0`, `p2 == 0xFF`, and `p3 == 0xFF`.

In a big-endian environment, `p0 == 0xFF`, `p1 == 0xFF`, `p2 == 0x0`, `p3 == 0x0`. — *end example*]

2 *Throws:* As specified in Error reporting (3).

3 *Remarks:* The bounds of the pixel data array range from `a`, where `a` is the address returned by this function, to `a + this->stride() * this->height()`. Given a height `h` where `h` is any value from 0 to `this->height() - 1`, any attempt to read a byte with an address that is not within the range of addresses defined by `a + this->stride() * h` shall result in undefined behavior; no diagnostic is required.

4 *Error conditions:* `io2d_error::null_pointer` if `this->format() == experimental::io2d::format::unknown`
|| `this->format() == experimental::io2d::format::invalid`.

```
experimental::io2d::format format() const noexcept;
```

5 *Returns:* The pixel format of the mapped surface.

6 *Remarks:* If the mapped surface is invalid, this function shall return `experimental::io2d::format::invalid`.

```
int width() const noexcept;
```

7 *Returns:* The number of pixels per horizontal line of the mapped surface.

8 *Remarks:* This function shall return the value 0 if `this->format() == experimental::io2d::format::unknown`
|| `this->format() == experimental::io2d::format::invalid`.

```
int height() const noexcept;
```

9 *Returns:* The number of horizontal lines of pixels in the mapped surface.

10 *Remarks:* This function shall return the value 0 if `this->format() == experimental::io2d::format::unknown`
|| `this->format() == experimental::io2d::format::invalid`.

```
int stride() const noexcept;
```

11 *Returns:* The length, in bytes, of a horizontal line of the mapped surface. [*Note:* This value is at least as large as the width in pixels of a horizontal line multiplied by the number of bytes per pixel but may be larger as a result of padding. — *end note*]

12 *Remarks:* This function shall return the value 0 if `this->format() == experimental::io2d::format::unknown`
|| `this->format() == experimental::io2d::format::invalid`.

14 Standalone functions [io2d.standalone]

14.1 Standalone functions synopsis [io2d.standalone.synopsis]

```
namespace std { namespace experimental { namespace io2d { inline namespace v1 {
    int format_stride_for_width(format format, int width) noexcept;
    display_surface make_display_surface(int preferredWidth,
        int preferredHeight, format preferredFormat,
        scaling scl = scaling::letterbox,
        refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0);
    display_surface make_display_surface(int preferredWidth,
        int preferredHeight, format preferredFormat, error_code& ec,
        scaling scl = scaling::letterbox,
        refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0) noexcept;
    display_surface make_display_surface(int preferredWidth,
        int preferredHeight, format preferredFormat, int preferredDisplayWidth,
        int preferredDisplayHeight, scaling scl = scaling::letterbox,
        refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0);
    display_surface make_display_surface(int preferredWidth,
        int preferredHeight, format preferredFormat, int preferredDisplayWidth,
        int preferredDisplayHeight, ::std::error_code& ec,
        scaling scl = scaling::letterbox,
        refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0) noexcept;
    image_surface make_image_surface(format format, int width, int height);
    image_surface make_image_surface(format format, int width, int height,
        error_code& ec) noexcept;
} } } } // namespaces std::experimental::io2d::v1
```

14.2 format_stride_for_width [io2d.standalone.formatstrideforwidth]

```
int format_stride_for_width(format fmt, int width) noexcept;
```

- ¹ *Returns:* The size in bytes of a row of pixels with a visual data format of `fmt` that is `width` pixels wide. This value may be larger than the value obtained by multiplying the number of bytes specified by the format enumerator specified by `fmt` by the number of pixels specified by `width`.
- ² If `fmt == format::invalid`, this function shall return 0.

14.3 make_display_surface [io2d.standalone.makedisplaysurface]

```
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat,
    scaling scl = scaling::letterbox,
    refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0);
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat, error_code& ec,
    scaling scl = scaling::letterbox,
    refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0)
noexcept;
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat, int preferredDisplayWidth,
    int preferredDisplayHeight, scaling scl = scaling::letterbox,
    refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0);
```



```
display_surface make_display_surface(int preferredWidth,
    int preferredHeight, format preferredFormat, int preferredDisplayWidth,
    int preferredDisplayHeight, ::std::error_code& ec,
    scaling scl = scaling::letterbox,
    refresh_rate rr = refresh_rate::as_fast_as_possible, double fps = 30.0)
noexcept;
```

- 1 *Returns:* Returns a `display_surface` object that is exactly the same as if the equivalent `display_surface` constructor was called with the same arguments.
- 2 *Throws:* As specified in Error reporting (3).
- 3 *Error conditions:* The errors, if any, produced by this function are the same as the errors for the equivalent `display_surface` constructor (13.13.4).

14.4 make_image_surface

[io2d.standalone.makeimagesurface]

```
image_surface make_image_surface(int width, int height,
    format fmt = format::argb32);
image_surface make_image_surface(int width, int height,
    error_code& ec, format fmt = format::argb32) noexcept;
```

- 1 *Returns:* Returns an `image_surface` object that is exactly the same as if the `image_surface` constructor was called with the same arguments.
- 2 *Throws:* As specified in Error reporting (3).
- 3 *Error conditions:* The errors, if any, produced by this function are the same as the errors for the equivalent `display_surface` constructor (13.12.3).

Annex A (informative)

Bibliography

[bibliography]

- ¹ The following is a list of informative resources intended to assist in the understanding or use of this Technical Specification.
- (1.1) — Porter, Thomas and Duff, Tom, 1984, Compositing digital images. ACM SIGGRAPH Computer Graphics. 1984. Vol. 18, no. 3, p. 253-259. DOI 10.1145/964965.808606. Association for Computing Machinery (ACM)
 - (1.2) — Foley, James D. et al., *Computer graphics: principles and practice*. 2nd ed. Reading, Massachusetts : Addison-Wesley, 1996.

Index

2D graphics
 synopsis, [11–13](#)

additive color, [2](#)
aliasing, [2](#)
alpha, [2](#)
anti-aliasing, [2](#)
artifact, [2](#)
aspect ratio, [2](#)

C

 Unicode TR, [1](#)
channel, [1](#)
closed path geometry, [2](#)
color
 transparent black, [128](#)
color model, [2, 3](#)
 RGB, [3](#)
 RGBA, [3](#)
color space, [3](#)
 sRGB, [3](#)
color stop, [6](#)
compose, [6](#)
composing operation, [6](#)
composition algorithm, [6](#)
CSS Colors Specification, [1](#)
cubic Bézier curve, [3](#)
current point, [3](#)

definitions, [1–6](#)
degenerate path geometry, [3](#)
degenerate path segment, [3](#)

filter, [3](#)
final path segment, [4](#)

graphics data, [4](#)
 raster, [4](#)
graphics resource, [4](#)
 graphics data graphics resource, [4](#)
 path geometry graphics resource, [4](#)
graphics state data, [5](#)
graphics subsystem, [5](#)

initial path segment, [5](#)

last-move-to point, [5](#)

normalize, [5](#)

open path geometry, [5](#)

path geometry, [5](#)
path segment, [5](#)
pixel, [2](#)
pixmap, [4](#)
point, [4](#)
premultiplied format, [4](#)

references
 normative, [1](#)
render, [5](#)
rendering and composing operation, [6](#)
rendering operation, [6](#)

sample, [6](#)
scope, [1](#)
standard coordinate space, [1](#)

visual data, [1](#)
visual data element, [2](#)
visual data format, [2](#)

Index of implementation-defined behavior

The entries in this section are rough descriptions; exact specifications are at the indicated page in the general text.

- errc::argument_out_of_domain
 - what_arg value, [9](#)
- errc::invalid_argument
 - what_arg value, [9](#)
- io2d_error_category
 - equivalent, [17](#)
- numeric_limits<double>::is_iec559 evaluates
 - to false, [7](#)
- simple_font_face
 - typeface, [96](#)
- surface
 - Font Face, [148](#)
- antialias
 - subpixel, [122](#)
- antialiasing
 - best, [123](#)
 - default, [122](#)
 - fast, [123](#)
 - good, [123](#)
- color stop
 - maximum size, [103](#)
 - size_type, [103](#)
- Dash Pattern
 - offset value, [154](#)
- display_surface
 - constructor, [182](#)
 - dimensions, [186](#)
 - display_dimensions, [187](#)
 - display_height, [186](#)
 - display_width, [185](#)
 - height, [184](#)
 - letterbox_brush, [187–189](#)
 - maximum frame rate, [188](#)
 - minimum frame rate, [188](#)
 - restore, [183](#)
 - show return value, [189](#)
 - unsupported Draw Format, [177](#)
 - width, [184](#)
- filter
 - best, [109](#)
 - fast, [109](#)
 - good, [109](#)
- image_surface
 - constructor, [171, 172](#)
 - data, [172](#)
- io2d_error
 - device_error, [16](#)
 - invalid_status, [15](#)
 - null_pointer, [15](#)
- io2d_error_category
 - message, [16](#)
- mapped_surface
 - commit_changes, [192](#)
- Miter Limit
 - maximum, [158](#)
 - minimum, [158](#)
- other error codes
 - what_arg value, [9](#)
- output device, [175](#)
- presence and meaning of native_handle_type and
 - native_handle, [7](#)
- surface
 - brush, [157](#)
 - fill, [159](#)
 - fill_extents_immediate, [166](#)
 - fill_immediate, [160](#)
 - font_face, [164, 170](#)
 - in_fill_immediate, [167](#)
 - in_stroke_immediate, [169](#)
 - map, [157](#)
 - mark_dirty, [156](#)
 - mask, [162](#)
 - mask_immediate, [163](#)
 - paint, [160](#)
 - render_text, [163](#)
 - restore, [157](#)
 - stroke, [160](#)
 - stroke_extents_immediate, [168](#)

stroke_immediate, [161](#)
text_extents, [169](#)
surface::flush errors, [155](#)